A SNARK for (Non-)Subsequences with Text-Sub-Linear Proving Time

Dario Fiore[1], San Ling[2,3], Khai Hanh Tang[2](✉), Hong Hanh Tran[4], Huaxiong Wang[2], and Yingfei Yan[5]

[1] IMDEA Software Institute
dario.fiore@imdea.org
[2] Nanyang Technological University
{lingsan,khaihanh.tang,hxwang}@ntu.edu.sg
[3] VinUniversity
ling.s@vinuni.edu.vn
[4] FPT University
hanhth12@fe.edu.vn
[5] University Clermont Auvergne
yingfei.yan@uca.fr

Abstract. A keyword $\mathbf{s}$ is a subsequence of a text $\mathbf{t}$ if $\mathbf{s}$ can be obtained by deleting some characters from $\mathbf{t}$; otherwise, $\mathbf{s}$ is a non-subsequence of $\mathbf{t}$. (Non-)subsequence relationships arise in various fields, including genetic analysis, blockchains, and natural language processing. Recently, Ling et al. (SCN 2024) proposed a succinct argument for non-subsequences based on multivariate sumcheck (Lund et al., FOCS 1990) whose prover's running time is at least $\mathcal{O}(n + N + |\Sigma|)$, where n and N are respectively the lengths of strings $\mathbf{s}$ and $\mathbf{t}$, and Σ is the alphabet over which $\mathbf{s}$ and $\mathbf{t}$ are defined. As shown in their work, proving non-subsequence relationships is non-trivial since one needs to decompose such an argument into smaller components for sumcheck, permutation, and lookup.
We propose a subsequence scheme that separates proving (non-)subsequences into the following two phases: (i) a preprocessing phase and (ii) a (non-)subsequence proving phase, assuming $n \ll N$ (i.e., $|\mathbf{s}| \ll |\mathbf{t}|$). Specifically, we can generate a one-time preprocessing proof with inputs $\mathbf{t}$ and Σ, without any knowledge of $\mathbf{s}$. When $\mathbf{s}$ is known, we can determine whether $\mathbf{s}$ is a subsequence of $\mathbf{t}$ and prove the corresponding statement. Employing cached quotients (IACR ePrint 2022/1763), we achieve a running time quasi-linear in $N + |\Sigma|$ for preprocessing, while the running time of proving a (non-)subsequence relationship is $\mathcal{O}(n \log_2(N + |\Sigma|))$ for each query $\mathbf{s}$. Since $n \ll N$ and $\log_2(N + |\Sigma|)$ grows sub-linearly with the text size, this saves the prover's running time, assuming a preprocessing depending only on $\mathbf{t}$ is computed in advance. Hence, we achieve a *text-sub-linear* proving time.

Keywords: SNARK, lookup arguments, cached quotients

1 Introduction

String searching is a fundamental problem in computer science. One way to search is to check whether a keyword $\mathbf{s} = (s_i)_{i=1}^n$ appears in a text $\mathbf{t} = (t_i)_{i=1}^N$ in the sense that one can delete some characters from $\mathbf{t}$ to achieve $\mathbf{s}$. If such a case occurs, we say that $\mathbf{s}$ is a subsequence of $\mathbf{t}$, denoted $\mathbf{s} \lhd \mathbf{t}$. Otherwise $\mathbf{s}$ is a non-subsequence of $\mathbf{t}$, denoted $\mathbf{s} \not\lhd \mathbf{t}$. String searching algorithms have widespread applications, including genetic analysis, text processing, financial analysis, computer vision, natural language processing, etc. [28]. The most typical use cases involve a very large text $\mathbf{t}$ and relatively short queries $\mathbf{s}$, namely $n \ll N$.

In this work, we are interested in the problem of *proving* string-searching statements. Specifically, we consider a scenario in which a prover can convince a verifier, holding a short digest (e.g., a cryptographic commitment) of the large text $\mathbf{t}$, that a keyword $\mathbf{s}$ is, or not, a subsequence of $\mathbf{t}$. We aim to achieve two fundamental properties: security and efficiency. The former ensures that the prover cannot convince the verifier of a false statement. The latter guarantees that the verifier can quickly verify the proof, e.g., in time sub-linear in N.

Changelog is presented in page 23.

A candidate solution to securely and efficiently prove string-searching statements is to use succinct, non-interactive arguments of knowledge [29,30], dubbed SNARKs. They are a cryptographic primitive that allows a prover to convince a verifier of the validity of some public statement by producing and sending a non-interactive proof to the verifier for verification. The proof must be sound (i.e., a cheating prover cannot convince the verifier of a false statement) and succinct (i.e., the proof size is sub-linear in the size of the statement). In the recent years, SNARKs have gained a lot of attraction from the cryptographic research community with different developments including SNARKs for general statements [31,49,39,4,27,13,50,44,55,51,12], accumulation/folding schemes [9,36], post-quantum SNARKs [21,1,15,35], ...

SNARKs for (non-)subsequences. Constructing a SNARK for subsequences is straightforward, especially if one relies on the recent developments for lookup statements [53,52,40,25,19,10,56]. Proposed by Thakur [46], proving that $\mathbf{s}$ is a subsequence of $\mathbf{t}$ can be done by showing the existence of a sequence $(\mathsf{id}_i)_{i=1}^n$ satisfying (i) $\{(\mathsf{id}_i, s_i)\}_{i=1}^n \subseteq \{(i, t_i)\}_{i=1}^N$ and (ii) that $(\mathsf{id}_i)_{i=1}^n$ is increasing (reducible to a lookup $\{\mathsf{id}_i - \mathsf{id}_{i-1}\}_{i=2}^n \subseteq [N-1]$, i.e., each $\mathsf{id}_i - \mathsf{id}_{i-1}$ is positive).

In contrast, proving non-subsequence relationships is a non-trivial task. A naive approach is to use a brute-force search, involving checking all $\binom{N}{n}$ possible cases as potential subsequences of $\mathbf{t}$, making the prover time prohibitively large, i.e., superlinear in $|\mathbf{t}|$. Specifically, one can transform the algorithm that checks whether every subsequence of $\mathbf{t}$ is not equal to $\mathbf{s}$ into a SNARK, proving that this algorithm works correctly. This forces the prover to run in superlinear time on $|\mathbf{t}|$ since there are $\binom{N}{n}$ potential subsequences of $\mathbf{t}$. Hence, although the SNARKs now can support fast verification, generating a SNARK for checking all those $\binom{N}{n}$ potential subsequences of $\mathbf{t}$ requires the prover to do an extremely heavy task. A very recent work by Ling et al. [37] addressed this problem by proposing a SNARK for non-subsequences with linear proving time $\mathcal{O}(n + N + |\Sigma|)$, where Σ is the alphabet used by the strings. However, considering the situations mentioned above when $\mathbf{t}$ is extremely large, a limitation of the solution by Ling et al. [37] is the scalability of the prover, since its proving time is linear in N. Hence, we raise the following central question:

Can we achieve a fast non-subsequence argument in which, after a preprocessing, the proving time is sub-linear in N?

Indeed, our observation is that in most applications of string searching, the length N of the text is usually very large, yet one may reuse the same text to prove several queries. We take a few examples as follows.

- *Preventing plagiarism.* An institution may store a large database containing the contents of books, exams, etc. The institution can check whether a piece of text is plagiarized from the database by showing a (non-)subsequence relationship.
- *DNA matching.* A hospital has a large DNA database related to serious diseases, and can prove that a person's piece of DNA is a (non-)subsequence of the database to determine whether the person has a certain disease.
- *Video intelligence.* an authority keeps a database containing CCTV videos. This authority can indicate whether a person's piece of image belongs to the database, e.g., to check whether the person is related to some illegal activity.
- *Content searching in blockchains.* Since blockchains cannot store large chunks of data, one can store them outside of blockchains (off-chain) and keep their hash digests in the blockchains (on-chain). Then, to search for related documents via a sequence of keywords, one can provide a SNARK for (non-)subsequences w.r.t. the off-chain data.

The databases in the above examples are usually huge. Preprocessing for the database is necessary to produce the proofs faster.

1.1 Our Contributions

We answer the above question in the affirmative by proposing a (non-)subsequence argument in which the prover runs in time $\mathcal{O}(n \log_2(N + |\Sigma|))$. As $\mathcal{O}(n \log_2(N + |\Sigma|))$ is sub-linear in $N + |\Sigma|$, i.e., the combined size of the text $\mathbf{t}$ and alphabet Σ, we achieve a *text-sub-linear* proving time. Our SNARK can prove the correctness of the bit $b = \mathbf{s} \lhd \mathbf{t}$, that is $b = 1$ when $\mathbf{s} \lhd \mathbf{t}$, or $b = 0$ if $\mathbf{s} \not\lhd \mathbf{t}$. To the best of our knowledge, this is the first SNARK for (non-)subsequences achieving

	Ling et al. [37]	This work
Proof size (preprocessing)	-	$\mathcal{O}(1)$
$\mathcal{P}$ time (preprocessing)	-	$\widetilde{\mathcal{O}}(N + \lvert\Sigma\rvert)$
$\mathcal{V}$ time (preprocessing)	-	$\mathcal{O}(\log_2(N + \lvert\Sigma\rvert))$
Proof size (proving phase)	$\mathcal{O}(\log_2 n + \log_2 N + \log_2 \lvert\Sigma\rvert + \mathsf{aux}_{\mathsf{ps}})$	$\mathcal{O}(1)$
$\mathcal{P}$ time (proving phase)	$\mathcal{O}(n + N + \lvert\Sigma\rvert + \mathsf{aux}_{\mathsf{tp}})$	$\mathcal{O}(n \log_2(N + \lvert\Sigma\rvert))$
$\mathcal{V}$ time (proving phase)	$\mathcal{O}(\log_2 n + \log_2 N + \log_2 \lvert\Sigma\rvert + \mathsf{aux}_{\mathsf{tv}})$	$\mathcal{O}(\log_2 n)$

Table 1. Comparison between the results of Ling et al. [37] and ours. The notation $\widetilde{\mathcal{O}}(m)$, for some m, indicates $\mathcal{O}(m \log_2^c m)$ for some constant c.Note that the complexities of Ling et al. considered in this table employ a generic multivariate PCS rather than a specific instantiation. Regarding ours, we instantiate with KZG polynomial commitment scheme (PCS) and achieve better complexities than those in Ling et al. regardless of any multivariate PCS that Ling et al. instantiates. The highlighted complexity is achieved due to our new modeling, to be presented in Section 5. Here, $\mathsf{aux}_{\mathsf{ps}}$ is the total commitment size and evaluation proof size of multivariate PCSs to polynomials encoding n, N, and $|\Sigma|$ values. $\mathsf{aux}_{\mathsf{tp}}$ is the total prover time for committing and evaluating multivariate PCSs to polynomials encoding n, N, and $|\Sigma|$ values. $\mathsf{aux}_{\mathsf{tv}}$ is the total verifier for verifying evaluation proofs behind the multivariate PCSs to polynomials encoding n, N, and $|\Sigma|$ values.

such proving time. To achieve this fast proving time, our construction works by first preprocessing the large text $\mathbf{t}$ to produce some intermediate components. Then, when the query $\mathbf{s}$ is known, the prover can use them to prove that $b = \mathbf{s} \lhd \mathbf{t}$ in time sub-linear in $|\mathbf{t}| + |\Sigma|$. The efficiency of our result is in Table 1, where we also compare to the result by Ling et al. [37]. For the concrete costs of our solution, we refer to Table 4 in Section 7.

In brief, we define a sequence $\mathsf{id} = (\mathsf{id}_i)_{i=0}^n$ s.t., for $i \in [0, n]$, if $\mathbf{s}[1 \dots i] \lhd \mathbf{t}$, then id_i is the smallest index s.t. $\mathbf{s}[1 \dots i] \lhd \mathbf{t}[1 \dots \mathsf{id}_i]$; otherwise, $\mathsf{id}_i = N + 1$ (see Definition 2 for its formal definition). Then, we model the constraints capturing the correct formation of id to a set of lookup/permutation constraints. These constraints are nicely categorized into the two types below.

(i) **Lookups/permutations for preprocessing.** These are only based on the knowledge of $\mathbf{t}$ and are needed to convince the verifier, who only holds a commitment of $\mathbf{t}$, that (commitments) to other preprocessed vectors are correctly computed. These constraints are of the form $\{a_i\}_{i=1}^{N+|\Sigma|} \subseteq \{b_i\}_{i=1}^{N+|\Sigma|}$ suitable for preprocessing as both sides of this form are of large cardinality. We then use univariate sumcheck [5] to prove these constraints based on Haböck's lemmas [32] for lookup and permutation arguments (recalled in Section 2.4).

(ii) **Lookups for proving (non-)subsequence relationships.** These constraints are based on the knowledge of both $\mathbf{t}$ and $\mathbf{s}$, and of the form $\{a_i\}_{i=1}^{n} \subseteq \{b_i\}_{i=1}^{N+|\Sigma|}$ which are suitable with preprocessing-based SNARKs for lookups since the LHS of this form is of small cardinality, i.e., n. On the other hand, the RHS is of large cardinality and only constructed based on the knowledge of $\mathbf{t}$, which can be handled in the preprocessing together with the proof for the above lookup. Here, we use cached quotients (CQ) [19] (recalled in Section 2.4) to realize the preprocessing-based lookup arguments, hence making fast proving time.

We note that the approach of Ling et al. [37] is not suitable for preprocessing to achieve fast proving time as discussed above. Specifically, Ling et al. shows that $\mathbf{s} \not\lhd \mathbf{t}$ via a set of constraints containing lookups of the form $\{a_i\}_{i=1}^{N} \subseteq S$, for some set S. The LHS of this form contains N elements and is constructed based on the knowledge of both $\mathbf{s}$ and $\mathbf{t}$. Hence, the approach of Ling et al. is unfit for applying preprocessing-based lookup techniques like CQ [19] since, as assumed, N is large and the CQ technique [19] only efficiently supports lookups where the LHS is of small cardinality.

1.2 Related Work

As in the work of Ling et al. [37], proofs for regex searching (e.g., see [38,41,54,2]) are closely related to ours. Since a proof for regex searching can be transformed into a proof for non-subsequence (as discussed in [37]), the mentioned results for regex searching have prover time at least $\mathcal{O}(N)$, hence unable to outperform our prover time $\mathcal{O}(n \log_2(N + |\Sigma|))$ when $n \ll N$.

Another line of research relevant to our work is that on lookup arguments [7,8,26,32,45,48], which allow one to prove a statement that $\{s_i\}_{i=1}^{H} \subseteq \{t_i\}_{i=1}^{K}$ for some $H, K \in \mathbb{N}$, where s_i and t_i are values over some finite field $\mathbb{F}$. For instance, these arguments enable proving the correct machine executions [7,3,14,17,18] in which a prover shows that every instruction is selected correctly from the table of instructions specifying the respective machine.

Recently, advanced techniques for lookup arguments [53,52,40,25,19,10,56,11] focus on preprocessing to make prover time more independent from K, namely, the cardinality of $\{t_i\}_{i=1}^{K}$. Specifically, the prover can preprocess $\{t_i\}_{i=1}^{K}$ to obtain preprocessed components. Then, when $\{s_i\}_{i=1}^{H}$ is known, prover can prove the lookup argument $\{s_i\}_{i=1}^{H} \subseteq \{t_i\}_{i=1}^{K}$ utilizing those components. As a result, the prover's running time depends only on H while the dependence on K is small. Hence, by assuming that $H \ll K$, one achieves lookup arguments with fast prover time. Our result shows a new application of lookup arguments with preprocessing via our novel encoding of non-subsequence statements with lookups. To see the need for preprocessing, we note that our solution relies on generating a lookup table that depends on the text $\mathbf{t}$ and is, therefore, unstructured. This is what prevents us from using the recent lookup techniques of Lasso [45], which instead provides an efficient proving time when the tables have some structure that allows for a shorter, closed-form representation of them.

1.3 Structure of the Paper

Section 1 introduces the problem and our contributions. In Section 2, we present the necessary notations, conventions, and preliminaries (with additional material deferred to Appendix A). Section 3 is a technical overview, discussing an algorithm that enables a preprocessing based on $\mathbf{t}$; and when $\mathbf{s}$ is known, it can decide whether $\mathbf{s} \lhd \mathbf{t}$ efficiently with time loosely depending on $|\mathbf{t}|$. We define the notion of *subsequence scheme* to capture this construction paradigm in Section 4.Section 5 contains our permutation- and lookup-based modeling of (non-)subsequences; formulating the two sets of constraints corresponding to the preprocessing and proving phases. In Section 6, we transform these constraints into their polynomial forms. In Section 7, we describe our subsequence scheme following the syntax specified in Section 4.

2 Preliminaries

Notations and conventions. For any two sequences $\mathbf{a}$ and $\mathbf{b}$ of the same length, we write $\mathbf{a} \bowtie \mathbf{b}$ to denote that they are permutations of each other. For $a, b \in \mathbb{Z}$, if $a \leq b$, $[a, b]$ denotes $\{a, a+1, \ldots, b\}$. If $a \geq 1$, $[a]$ denotes $[1, a]$. For a string $\mathbf{x} = (x_i)_{i=1}^{|\mathbf{x}|}$ and indices $i, j \in [|\mathbf{x}|]$, $\mathbf{x}[i \ldots j]$ denotes $(x_k)_{k=i}^{j}$ if $i \leq j$. Otherwise, when $i > j$, we understand that $\mathbf{x}[i \ldots j] = \epsilon$, namely, the empty string. For an NP relation $\mathcal{R}$, we may write $(\mathsf{st}; \mathsf{wit}) \in \mathcal{R}$ where st and wit are respectively the statement and witness. Moreover, we may also write $\mathsf{st} \in \mathcal{L}(\mathcal{R})$ to indicate that there exists a witness wit s.t. $(\mathsf{st}; \mathsf{wit}) \in \mathcal{R}$. $\mathbb{F}$ denotes a finite field of prime order used globally in this paper. We employ an efficiently computable non-degenerate pairing $\langle \cdot, \cdot \rangle : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ where $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_T$ are groups of size $|\mathbb{F}|$ generated by $\mathfrak{g}_1, \mathfrak{g}_2$, and $\mathfrak{g}_T = \langle \mathfrak{g}_1, \mathfrak{g}_2 \rangle$, respectively. Throughout the paper, we use additive notation: for an $x \in \mathbb{F}$, we denote by $[\![x]\!]_1 = x \cdot \mathfrak{g}_1$ and $[\![x]\!]_2 = x \cdot \mathfrak{g}_2$. For $d \in \mathbb{N}$, $\mathbb{F}_{<d}[X]$ denotes the set of polynomials of degree less than d. Let $\mathbb{K}$ be a multiplicative subgroup of $\mathbb{F}$. We recall $z_{\mathbb{K}}(X) = X^{|\mathbb{K}|} - 1$, namely, the vanishing polynomial, to be the unique non-zero monic polynomial that vanishes at every element in $\mathbb{K}$, $z_{\mathbb{K}}(\alpha) = 0$ for all $\alpha \in \mathbb{K}$. $\widetilde{\mathcal{O}}(m)$ indicates $\mathcal{O}(m \log_2^c m)$ for some constant c.

2.1 Lagrange's Interpolation

Let $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}$ be a multiplicative subgroup of $\mathbb{F}$. For $i \in [K]$, the i-th Lagrange basis polynomial $\ell_{\mathbb{K}}^{(i)}(X)$ over $\mathbb{F}$ is $\ell_{\mathbb{K}}^{(i)}(X) = \prod_{j \neq i, j \in [K]} \frac{X - \omega_{\mathbb{K}}^{j-1}}{\omega_{\mathbb{K}}^{i-1} - \omega_{\mathbb{K}}^{j-1}}$. For $\mathbf{a} = (a_i)_{i=1}^{K} \in \mathbb{F}^K$, $f_{\mathbf{a}}(X) \in \mathbb{F}_{<K}[X]$ encodes $\mathbf{a}$ via $f_{\mathbf{a}}(X) = \sum_{i=1}^{K} a_i \cdot \ell_{\mathbb{K}}^{(i)}(X)$. It follows that, for $i \in [K]$, $f_{\mathbf{a}}(\omega_{\mathbb{K}}^{i-1}) = \sum_{i=1}^{K} a_i \cdot \ell_{\mathbb{K}}^{(i)}(\omega_{\mathbb{K}}^{i-1}) = a_i$. Here, $f_{\mathbf{a}}(X)$ is the unique polynomial of degree at most $K-1$ satisfying $f_a(\omega_{\mathbb{K}}^{i-1}) = a_i$ for $i \in [K]$ [47, Lemma 2.3].

Rotations. In some parts (c.f. Sections 6.1 and 6.2) of this result, we may need to rotate $\mathbf{a}$ either to the left or the right by one unit. Rotating $\mathbf{a}$ to the left by one unit, one obtains $\mathbf{a}' = (a_2, \ldots, a_K, a_1)$. The polynomial encoding $\mathbf{a}'$ is $f_{\mathbf{a}'}(X) = \left(\sum_{i=1}^{K-1} a_{i+1} \cdot \ell_{\mathbb{K}}^{(i)}(X)\right) + a_1 \cdot \ell_{\mathbb{K}}^{(K)}(X) = f_{\mathbf{a}}(\omega_{\mathbb{K}} \cdot X)$. Clearly, $f_{\mathbf{a}'}(\omega_{\mathbb{K}}^{i-1}) = a_{i+1}$ for each $i \in [K-1]$, and $f_{\mathbf{a}'}(\omega_{\mathbb{K}}^{K-1}) = a_1$.

Lemma 1 (Left rotation). $f_{\mathbf{a}'}(X) = f_{\mathbf{a}}(\omega_{\mathbb{K}} \cdot X)$.

The proof of Lemma 1 is deferred to Appendix A.2. For rotating $\mathbf{a}$ to the right, we have the following Lemma 2.

Lemma 2 (Right rotation). $f_{\mathbf{a}}(\omega_{\mathbb{K}}^{-1} \cdot X)$ *encodes the vector obtained by rotating* $\mathbf{a}$ *to the right by one unit. Equivalently, we can write* $f_{\mathbf{a}'}(\omega_{\mathbb{K}} \cdot X) = f_{\mathbf{a}}(X)$.

The proof of Lemma 2 is similar to that of Lemma 1.

2.2 Algebraic Group Model

We recall the definition of algebraic group model (AGM) [24] in Definition 1.

Definition 1 (AGM [24]). *Let* $\mathbb{G} = \langle \mathfrak{g} \rangle$ *be a group generated by* $\mathfrak{g}$ *and of order* $|\mathbb{F}|$*. An adversary* $\mathcal{A}$ *is algebraic if for every group element* $Z \in \mathbb{G} = \langle \mathfrak{g} \rangle$ *that it outputs, it is required to output a representation* $\mathbf{a} = (a_0, a_1, a_2, \ldots, a_m)$*, for some* $m \in \mathbb{N}$*, s.t.* $Z = a_0 \cdot [\![1]\!] + \sum_i a_i \cdot [\![y_i]\!]$*, where* $[\![y_1]\!], [\![y_2]\!], \ldots, [\![y_m]\!] \in \mathbb{G}$ *are group elements that* $\mathcal{A}$ *has seen thus far.*

Here, $(\mathfrak{g}, [\![\cdot]\!])$ can be either $(\mathfrak{g}_1, [\![\cdot]\!]_1)$ or $(\mathfrak{g}_2, [\![\cdot]\!]_2)$ introduced in notations and conventions in the beginning of this Section 2.

2.3 KZG Polynomial Commitments

Kate, Zaverucha, and Goldberg (KZG) [34] proposed a constant-sized polynomial commitment scheme for polynomials $f(X) \in \mathbb{F}_{<K}[X]$. (See the definition of polynomial commitment schemes (PCS) in Appendix A.6.) Importantly, an evaluation proof for any $f(\psi)$, where $\psi \in \mathbb{F}$, is constant-sized and constant-time to verify. Consider a group $\mathbb{G}_1$ generated by $\mathfrak{g}_1$. Let $K \in \mathbb{N}$. KZG requires public parameters $\{[\![x^{i-1}]\!]_1\}_{i=1}^{K}$ and $[\![x]\!]_2$ which hide the trapdoor x. KZG is computationally binding under K-SDH [6] and hiding under the discrete logarithm.

Committing. Let $f(X) = \sum_{i=1}^{K} f_i \cdot X^{i-1} \in \mathbb{F}_{<K}[X]$. A KZG commitment to $f(X)$ is $\sigma_1(f) = [\![f(x)]\!]_1 = \sum_{i=1}^{K} f_i \cdot [\![x^{i-1}]\!]_1$. Committing to $f(x)$ takes time $\mathcal{O}(K)$ $\mathbb{G}_1, \mathbb{F}$.

Remark 1. One may needs to commit to polynomials of degrees lower than $K-1$. Specifically, if $f'(X) = \sum_{i=1}^{m} f'_i \cdot X^{i-1}$ of degree $m-1 < K-1$. We simply ignore $[\![x^{i-1}]\!]_i$ for $i > m$. Hence, $[\![f'(x)]\!] = \sum_{i=1}^{m} f'_i \cdot [\![x^{i-1}]\!]_1$.

Remark 2. For a multiplicative sub-group $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}$ of $\mathbb{F}$, one can commit to $f(X) \in \mathbb{F}_{<K}[X]$ w.r.t the evaluations $f(\omega_{\mathbb{K}}^{i-1}) = \mathsf{val}_i \in \mathbb{F}$ for $i \in [K]$ by first forming the committed Lagrange basis $\left\{\left[\!\left[\ell_{\mathbb{K}}^{(i)}(x)\right]\!\right]_1\right\}_{i=1}^{K}$ and compute $[\![f(x)]\!]_1 := \sum_{i=1}^{K} \mathsf{val}_i \cdot \left[\!\left[\ell_{\mathbb{K}}^{(i)}(x)\right]\!\right]_1$ based on Section 2.1.

Proving single evaluation. To compute an evaluation proof that $f(\psi) = y$ for some $\psi, y \in \mathbb{F}$, KZG PCS views $f(X) - y = q(X)(X - \psi)$, i.e., $f(X) - y$ is divisible by $X - \psi$. The proof is then $\pi = [\![q(x)]\!]_1 = [\![(f(x) - y)/(x - \psi)]\!]_1$. Computing the proof takes time $\mathcal{O}(K)$. To verify π, one checks (in constant time) if

$$\langle \sigma_1(f) - y \cdot [\![1]\!]_1, [\![1]\!]_2 \rangle = \langle \pi, [\![x]\!]_2 - \psi \cdot [\![1]\!]_2 \rangle . \tag{1}$$

Proving batch evaluation. Given a set of points Ψ and their evaluations $\{f(\psi)\}_{\psi \in \Psi}$, KZG can prove all evaluations with a constant-sized batch proof rather than $|\Psi|$ single evaluation proofs.

The prover determines an accumulator polynomial $\mathsf{acc}(X) = \Pi_{\psi \in \Psi}(X - \psi)$ and computes the division $f(X)/\mathsf{acc}(X)$ into a quotient $\mathsf{quo}(X)$ and remainder $\mathsf{rem}(X)$. As noted by [34] that $\mathsf{rem}(\psi) = f(\psi)$ $\forall \psi \in \Psi$, we can determine $\mathsf{rem}(X)$ based on the evaluations $(f(\psi))_{\psi \in \Psi}$. The batch proof is $\pi_\psi = [\![\mathsf{quo}(x)]\!]_1$. To verify π_Ψ, the verifier first computes $[\![\mathsf{rem}(x)]\!]_1$ and $[\![\mathsf{acc}(x)]\!]_2$ which requires $([\![x^i]\!]_2)_{i=0}^{|\Psi|}$ as a part of the public parameters. The proof π_Ψ is valid if

$$\langle \sigma_1(f) - [\![\mathsf{rem}(X)]\!]_1, [\![1]\!]_2 \rangle = \langle \pi_\Psi, [\![\mathsf{acc}(x)]\!]_2 \rangle . \tag{2}$$

	Proof size	$\mathcal{P}$ time	$\mathcal{V}$ time
Committing	1 $\mathbb{G}_1$	$\mathcal{O}(K)$ $\mathbb{G}_1, \mathbb{F}$	-
Single Evaluation	1 $\mathbb{G}_1$	$\mathcal{O}(K)$ $\mathbb{G}_1, \mathbb{F}$	$\mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2, \mathbb{F}$, $2P$
Batch Evaluation	1 $\mathbb{G}_1$	$\widetilde{\mathcal{O}}(K)$ $\mathbb{G}_1, \mathbb{F}$	$\mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2, \mathbb{F}$, $2P$

Table 2. Efficiency of KZG PCS where "$2P$" means 2 pairings. In batch evaluation verification, $\mathsf{rem}(X)$ and $\mathsf{acc}(X)$ and their respective commitments $[\![\mathsf{rem}(x)]\!]_1$ and $[\![\mathsf{acc}(x)]\!]_2$ are assumed to be known in advance, hence achieving $\mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2, \mathbb{F}$ (see Remark 3).

Remark 3. The running time of prover, in proving batch evaluation, is $\widetilde{\mathcal{O}}(K)$ [20,23] while verifier's running time is $\mathcal{O}(K)$ since verifier needs to compute $[\![\mathsf{acc}(x)]\!]_2$. However, in this paper, $[\![\mathsf{rem}(x)]\!]_1$ and $[\![\mathsf{acc}(x)]\!]_2$ are determined in advance (as batch evaluation is only used in $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ in Figure 1 with $\mathsf{rem}(X)$ and $\mathsf{acc}(X)$ publicly known by both prover and verifier) and hence the verifier time is $\mathcal{O}(1)$. The sizes of commitment and evaluation proof are $\mathcal{O}(1)$.

Efficiency. The efficiencies are reported in Table 2. The efficiency for batch evaluation is based on Feist and Khovratovich [20].

2.4 SNARKs for Permutations and Lookups

Recall the protocols, denoted by $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$, $\Pi^{\mathbb{K}}_{\mathsf{perm}}$, and $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}} = (\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}, \Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}})$, for proving lookups, permutations, and lookups with preprocessing [19], respectively. More precisely, these protocols are for relations $\mathcal{R}_{\mathsf{lookup}}$, $\mathcal{R}_{\mathsf{perm}}$, and $\mathcal{R}_{\mathsf{cq}}$ having a common abstract relation

$$\mathcal{R}_{\mathsf{abstract}} = \left\{ \begin{array}{c} (\mathsf{pp}, \sigma_2(f_{\mathbf{t}}), \sigma_1(f_{\mathbf{s}}); \\ f_{\mathbf{t}}(X), f_{\mathbf{s}}(X)) \end{array} \middle| \begin{array}{l} \sigma_2(f_{\mathbf{t}}) = [\![f_{\mathbf{t}}(x)]\!]_2, \sigma_1(f_{\mathbf{s}}) = [\![f_{\mathbf{s}}(x)]\!]_1, \\ \mathsf{condition} \end{array} \right\} \tag{3}$$

where pp contains $H, K \in \mathbb{N}$, $\mathbb{H} = \{\omega_{\mathbb{H}}^{i-1}\}_{i=1}^{H}$, $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_1\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_2\}_{i=1}^{K+1}$, $z_{\mathbb{H}}(X) = X^H - 1$, $z_{\mathbb{K}}(X) = X^K - 1$, $\sigma_2(z_{\mathbb{K}}) = [\![z_{\mathbb{K}}(x)]\!]_2$, $\sigma_1(\ell_{\mathbb{K}}^{(i)}) = \left[\!\!\left[\ell_{\mathbb{K}}^{(i)}(x)\right]\!\!\right]_1$ $\forall i \in [K]$; $f_{\mathbf{s}}(X)$ and $f_{\mathbf{t}}(X)$ encode $\mathbf{s} = (s_i)_{i=1}^{H}$ and $\mathbf{t} = (t_j)_{j=1}^{K}$, respectively over $\mathbb{H}$ and $\mathbb{K}$, respectively. Here, $\mathsf{condition}$ is the condition specific for each relation in $\mathcal{R}_{\mathsf{lookup}}$, $\mathcal{R}_{\mathsf{perm}}$, and $\mathcal{R}_{\mathsf{cq}}$ as follows.

– $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ **for** $\mathcal{R}_{\mathsf{lookup}}$**.** This protocol proves that $\{s_i\}_{i=1}^{K} \subseteq \{t_i\}_{i=1}^{K}$, i.e., $H = K$. Adapted from $\mathcal{R}_{\mathsf{abstract}}$, $\mathcal{R}_{\mathsf{lookup}}$ has $\mathbb{H} = \mathbb{K}$ and $\mathsf{condition}$ is $\{s_i\}_{i=1}^{K} \subseteq \{t_i\}_{i=1}^{K}$.
– $\Pi^{\mathbb{K}}_{\mathsf{perm}}$ **for** $\mathcal{R}_{\mathsf{perm}}$**.** This protocol proves the permutation relationship $\mathbf{s} \bowtie \mathbf{t}$. Adapted from $\mathcal{R}_{\mathsf{abstract}}$, $\mathcal{R}_{\mathsf{perm}}$ has $\mathbb{H} = \mathbb{K}$ and $\mathsf{condition}$ replaced by $\mathbf{s} \bowtie \mathbf{t}$.
– $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}}$ **for** $\mathcal{R}_{\mathsf{cq}}$**.** This is the cached quotient protocol cq [19] that proves the lookup relationship $\{s_i\}_{i=1}^{H} \subseteq \{t_i\}_{i=1}^{K}$, replacing $\mathsf{condition}$ in $\mathcal{R}_{\mathsf{abstract}}$, assuming that $H \ll K$. Here, $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}}$ has two phases: (i) a preprocessing phase $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$ for preprocessing $\mathbf{t}$ to achieve some "cache", denoted by $\mathsf{prep}(f_{\mathbf{t}})$, and (ii) the proving phase $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$ for quickly proving the lookup condition leveraging $\mathsf{prep}(f_{\mathbf{t}})$.

The detailed descriptions of these protocols are presented in Appendix A.7. We briefly recall their ideas as follows.

Overview. We first recall the idea of CQ as follows. By Haböck's lookup argument [32], one can show $\{s_i\}_{i=1}^{H} \subseteq \{t_i\}_{i=1}^{K}$ via proving the existence of $\mathbf{m} = (m_i)_{i=1}^{K}$ s.t. $\sum_{i=1}^{K} \frac{m_i}{t_i+Y} = \sum_{i=1}^{H} \frac{1}{s_i+Y}$. Encod $\mathbf{m}$, $\mathbf{t}$, and $(m_i/(t_i+\beta))_{i=1}^{K}$, for some random challenge β given by verifier, into polynomials $f_{\mathbf{m}}(X)$, $f_{\mathbf{t}}(X)$, $f_{\mathbf{a}}(X)$, respectively, interpolated over $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}$. Similarly, encode $\mathbf{s}$ and $(1/(s_i+\beta))_{i=1}^{H}$ into $f_{\mathbf{s}}(X)$ and $f_{\mathbf{b}}(X)$, respectively, interpolated over $\mathbb{H} = \{\omega_{\mathbb{H}}^{i-1}\}_{i=1}^{H}$.

One needs to prove the sumcheck $\sum_{i=1}^{K} f_{\mathbf{a}}(\omega_{\mathbb{K}}^{i-1}) = \sum_{i=1}^{H} f_{\mathbf{b}}(\omega_{\mathbb{H}}^{i-1})$ (Haböck's lookup) and the well-formedness of $f_{\mathbf{a}}(X)$ and $f_{\mathbf{b}}(X)$. The sumcheck can be done by univariate sumcheck [5] via proving $K \cdot f_{\mathbf{a}}(0) = H \cdot f_{\mathbf{b}}(0)$. For well-formednesses of $f_{\mathbf{a}}(X)$ and $f_{\mathbf{b}}(X)$, one can prove by showing the existence of $q_{\mathbf{a}}(X)$ and $q_{\mathbf{b}}(X)$ satisfying $f_{\mathbf{a}}(X) \cdot (f_{\mathbf{t}}(X) + \beta) - f_{\mathbf{m}}(X) = q_{\mathbf{a}}(X) \cdot z_{\mathbb{K}}(X)$ and $f_{\mathbf{b}}(X) \cdot (f_{\mathbf{s}}(X) + \beta) - 1 = q_{\mathbf{b}}(X) \cdot z_{\mathbb{H}}(X)$ where $z_{\mathbb{K}}(X) = X^K - 1$ and $z_{\mathbb{H}}(X) = X^H - 1$. To enable preprocessing, computing $q_{\mathbf{a}}(X)$ is slow with time complexity depending on K.

To optimize this, [19] observes that we can write

$$\ell_{\mathbb{K}}^{(i)}(X) \cdot f_{\mathbf{t}}(X) = t_i \cdot \ell_{\mathbb{K}}^{(i)}(X) + z_{\mathbb{K}}(X) \cdot q_i(X)$$

for $q_i(X) \in \mathbb{F}_{<K}[X]$ $\forall i \in [K]$. Then, notice that $q_{\mathbf{a}}(X)$ can be computed from $\{q_i(X)\}_{i=1}^K$ as follows. We can establish

$$\underbrace{\sum_{i=1}^K \frac{m_i}{t_i+\beta} \cdot \ell_{\mathbb{K}}^{(i)}(X) \cdot f_{\mathbf{t}}(X)}_{f_{\mathbf{a}}(X) \cdot f_{\mathbf{t}}(X)} = \sum_{i=1}^K \frac{m_i}{t_i+\beta} \cdot t_i \cdot \ell_{\mathbb{K}}^{(i)}(X) + \sum_{i=1}^K \frac{m_i}{t_i+\beta} \cdot q_i(X) \cdot z_{\mathbb{K}}(X).$$

We see that

$$\begin{aligned}\sum_{i=1}^K \frac{m_i}{t_i+\beta} \cdot t_i \cdot \ell_{\mathbb{K}}^{(i)}(X) &= \sum_{i=1}^K \frac{m_i}{t_i+\beta} \cdot (t_i+\beta) \cdot \ell_{\mathbb{K}}^{(i)}(X) - \sum_{i=1}^K \frac{m_i}{t_i+\beta} \cdot \beta \cdot \ell_{\mathbb{K}}^{(i)}(X) \\ &= f_{\mathbf{m}}(X) - \beta \cdot f_{\mathbf{a}}(X).\end{aligned}$$

Hence, $\sum_{i=1}^K \frac{m_i}{t_i+\beta} \cdot q_i(X) = q_{\mathbf{a}}(X)$. As $q_{\mathbf{a}}(X)$ encodes $(m_i/(t_i+\beta))_{i=1}^K$, which is H-sparse since $(m_i)_{i=1}^K$ is H-spase. Hence, when committing into PCSs, one can achive commitment to $q_{\mathbf{a}}(X)$ from H commitments to $q_i(X)$ $\forall i \in [K]$. Hence, $q_i(X)$ $\forall i \in [K]$ and their commitments are contained in $\mathsf{prep}(f_{\mathbf{t}})$ as preprocessed components for fast recovering of commitment to $q_{\mathbf{a}}(X)$. Hence, the construction of $\Pi_{\mathsf{cq}}^{\mathbb{K}}$ can be performed via PCSs and their evaluation proofs/protocols.

Constructions of $\Pi_{\mathsf{lookup}}^{\mathbb{K}}$ and $\Pi_{\mathsf{perm}}^{\mathbb{K}}$ can be easily adapted from $\Pi_{\mathsf{cq}}^{\mathbb{H},\mathbb{K}}$ to equivalently prove (i) the existence of $\mathbf{m} = (m_i)_{i=1}^K$ s.t. $\sum_{i=1}^K \frac{m_i}{t_i+Y} = \sum_{i=1}^K \frac{1}{s_i+Y}$ and (ii) $\sum_{i=1}^K \frac{1}{t_i+Y} = \sum_{i=1}^K \frac{1}{s_i+Y}$, respectively, according to the Haböck's lemmas [32]. In these constructions, we do not need to consider the preprocessing to preprocess $q_i(X)$ $\forall i \in [K]$ in advance.

Efficiency. See Table 3.

Analysis. $\Pi_{\mathsf{cq}}^{\mathbb{H},\mathbb{K}}$ is knowledge-sound in AGM with error $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$.

The detailed constructions of these protocols, i.e., $\Pi_{\mathsf{lookup}}^{\mathbb{K}}$, $\Pi_{\mathsf{perm}}^{\mathbb{K}}$, and $\Pi_{\mathsf{cq}}^{\mathbb{H},\mathbb{K}}$ are recalled in Appendix A.7. Their efficiencies are reported in Table 3.

	Preprocessing	**Proving**		
	$\mathcal{P}$ **time**	**Proof size**	$\mathcal{P}$ **time**	$\mathcal{V}$ **time**
$\Pi_{\mathsf{lookup}}^{\mathbb{K}}$	-	8 $\mathbb{G}_1$, 3 $\mathbb{F}$	$\widetilde{\mathcal{O}}(K)$ $\mathbb{G}_1, \mathbb{F}$	$\mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2$, $\mathcal{O}(\log_2 K)$ $\mathbb{F}$, $4P$
$\Pi_{\mathsf{perm}}^{\mathbb{K}}$	-	8 $\mathbb{G}_1$, 3 $\mathbb{F}$	$\widetilde{\mathcal{O}}(K)$ $\mathbb{G}_1, \mathbb{F}$	$\mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2$, $\mathcal{O}(\log_2 K)$ $\mathbb{F}$, $4P$
$\Pi_{\mathsf{cq}}^{\mathbb{H},\mathbb{K}}$	$\widetilde{\mathcal{O}}(K)$ $\mathbb{G}_1, \mathbb{F}$	8 $\mathbb{G}_1$, 3 $\mathbb{F}$	$\mathcal{O}(H)$ $\mathbb{G}_1, \widetilde{\mathcal{O}}(H)$ $\mathbb{F}$	$\mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2$, $\mathcal{O}(\log_2 H)$ $\mathbb{F}$, $5P$

Table 3. Efficiency of $\Pi_{\mathsf{lookup}}^{\mathbb{K}}$, $\Pi_{\mathsf{perm}}^{\mathbb{K}}$, and $\Pi_{\mathsf{cq}}^{\mathbb{H},\mathbb{K}}$ where "$4P$" and "$5P$" respectively mean 4 and 5 pairings.

3 Technical Overview

Let $n, N \in \mathbb{N}$, $\mathbf{s} = (s_i)_{i=1}^n$, $\mathbf{t} = (t_i)_{i=1}^N$, and Σ be an alphabet capturing characters in $\mathbf{s}$ and $\mathbf{t}$. Assume that $n \ll N$. We now discuss the technical overview for our construction of fast SNARKs for (non-)subsequence arguments in Section 7 (w.r.t. the definition of subsequence scheme to be defined in Section 4). This section helps form the constraints (c.f. Section 5) suitable for constructing a subsequence scheme (c.f. Section 7), whose definition, including syntax, correctness, and knowledge soundness, can be found in Section 4, enabling fast proving time as claimed in Section 1.1. An initial attempt is presented in Section 3.1 while a more optimized attempt is in Section 3.2. For convenience purposes, let $K = N + |\Sigma|$ for this entire paper.

3.1 An Initial Attempt

Our initial attempt is to compute the sequence id in Definition 2 capturing $\mathbf{s} \lhd \mathbf{t} \iff \mathsf{id}_n < N+1$. This sequence can be computed by using a deterministic finite automaton (DFA) from Crochemore, Melichar, and Tronícek [16].

Definition 2 (Sequence id). $\mathsf{id} = (\mathsf{id}_i)_{i=0}^{n}$ *is defined as follows. For* $i \in [0, n]$, *if* $\mathbf{s}[1 \dots i] \lhd \mathbf{t}$, id_i *is the smallest index s.t.* $\mathbf{s}[1 \dots i] \lhd \mathbf{t}[1 \dots \mathsf{id}_i]$; *otherwise,* $\mathsf{id}_i = N+1$, *namely, the index larger than* $|\mathbf{t}| = N$, *to indicate that* $\mathbf{s}[1 \dots i] \not\lhd \mathbf{t}$.

We have the following Lemma 3 for capturing the properties of id.

Lemma 3 (Properties of id). *The following properties must hold.*

(i) If $\mathsf{id}_n \leq N$, *then* $\mathbf{s} \lhd \mathbf{t}$ *since* $\mathbf{s} = \mathbf{s}[1 \dots n]$; *otherwise,* $\mathbf{s} \not\lhd \mathbf{t}$. *Hence,*

$$(\mathbf{s} \lhd \mathbf{t}) = (\mathsf{id}_n \leq N). \tag{4}$$

(ii) For $i \in [n]$, *if* $\mathsf{id}_i < N+1$, *then* $s_i = t_{\mathsf{id}_i}$.
(iii) For $i \in [n]$, *if* $\mathsf{id}_{i-1} < N+1$, *then* $\mathsf{id}_i > \mathsf{id}_{i-1}$; *otherwise,* $\mathsf{id}_i = N+1$.

Proof. *Deciding $\mathbf{s} \lhd \mathbf{t}$ from id.* It is obvious from Definition 2 that id_n decides whether $\mathbf{s} \lhd \mathbf{t}$ as follows. If $\mathsf{id}_n \leq N$, then $\mathbf{s} = \mathbf{s}[1 \dots n] \lhd \mathbf{t}[1 \dots \mathsf{id}_n]$ implying $\mathbf{s} \lhd \mathbf{t}$ since $\mathbf{t}[1 \dots \mathsf{id}_n]$ is a prefix of $\mathbf{t}$; otherwise, when $\mathsf{id}_n = N+1$, it is obvious that $\mathbf{s} \not\lhd \mathbf{t}$.

$s_i = t_{\mathsf{id}_i}$ if $\mathsf{id}_i \leq N$. By definition, $\mathbf{s}[1 \dots i] \lhd \mathbf{t}[1 \dots \mathsf{id}_i]$. If $s_i \neq t_{\mathsf{id}_i}$, then it must be the case that $\mathbf{s}[1 \dots i] \lhd \mathbf{t}[1 \dots \mathsf{id}_i - 1]$, contradicting Definition 2 for id_i.

Non-decrement of id. For $i \in [n]$, if $\mathsf{id}_{i-1} < N+1$, then we know that $\mathsf{id}_i > \mathsf{id}_{i-1}$ according to the following two cases.

- If $\mathsf{id}_i \leq N$, then $\mathbf{s}[1 \dots i] \lhd \mathbf{t}[1 \dots \mathsf{id}_i]$ by Definition 2. Deleting the last element of each of these two sequences implies that $\mathbf{s}[1 \dots i-1] \lhd \mathbf{t}[1 \dots \mathsf{id}_i - 1] \lhd \mathbf{t}$. Therefore, according to Definition 2, id_{i-1} is the smallest index satisfying $\mathbf{s}[1 \cdots i-1] \lhd \mathbf{t}[1 \cdots \mathsf{id}_{i-1}]$. It follows that $\mathsf{id}_{i-1} \leq \mathsf{id}_i - 1 < \mathsf{id}_i$.
- If $\mathsf{id}_i = N+1$, then it is clear that $\mathsf{id}_i = N+1 > \mathsf{id}_{i-1}$.

If $\mathsf{id}_{i-1} = N+1$, it is straightforward that $\mathsf{id}_i = \mathsf{id}_{i-1} = N+1$ because $\mathbf{s}[1 \dots i-1] \not\lhd \mathbf{t}$ implies $\mathbf{s}[1 \dots i] \not\lhd \mathbf{t}$.

We thus conclude the proof. □

Computing id. Based on the above-discussed properties, we specify a way to compute id efficiently. Assume that we already determined $\mathsf{id}_{i-1} \leq N$. According to (ii) and (iii) in Lemma 3, we know that $s_{i-1} = t_{\mathsf{id}_{i-1}}$ and $\mathsf{id}_i > \mathsf{id}_{i-1}$. Therefore, one can determine id_i by looking for id_i as the smallest index in $[\mathsf{id}_{i-1} + 1, N+1]$ satisfying either $\mathsf{id}_i = N+1$ or $\mathsf{id}_i \leq N \wedge s_i = t_{\mathsf{id}_i}$. This hence leads us to use a DFA, denoted as dfa, in Definition 3, which is based on [16].

Definition 3. $\mathsf{dfa} : [0, N+1] \times \Sigma \to [N+1]$ *is defined as follows. For* $i \in [0, N]$ *and* $c \in \Sigma$, $\mathsf{dfa}(i, c)$ *is the smallest* $j \in [N+1]$ *s.t.* $j > i$ *and either* $j \in [N] \wedge t_j = c$ *or* $j = N+1$. *Additionally,* $\mathsf{dfa}(N+1, c) = N+1$ *for any* $c \in \Sigma$.

We see that $\mathsf{id}_0 = 0$ and $\mathsf{id}_i = \mathsf{dfa}(\mathsf{id}_{i-1}, s_i)$ for $i \in [n]$. Hence, following the above intuition, we can compute id based on Lemma 4.

Lemma 4. $\mathsf{id}_0 = 0$ *and* $\mathsf{id}_i = \mathsf{dfa}(\mathsf{id}_{i-1}, s_i)$ *for* $i \in [n]$.

The proof of Lemma 4 is deferred to Appendix B.1.

Transforming into lookup arguments. By Lemma 4, one can form the set

$$\mathcal{S}_{\mathsf{dfa}} = \left\{ (i, c, j) \in [0, N+1] \times \Sigma \times [N+1] \,\middle|\, \mathsf{dfa}(i, c) = j \right\}$$

and show the well-formedness of id via

$$\{(\mathsf{id}_{i-1}, s_i, \mathsf{id}_i)\}_{i=1}^{n} \subseteq \mathcal{S}_{\mathsf{dfa}}. \tag{5}$$

Having id well-formed, one can prove (non-)subsequence by checking whether $\mathsf{id}_n < N+1$ or $\mathsf{id}_n = N+1$. Notice that the LHS of (5) is of size n and $n \ll |\mathcal{S}_{\mathsf{dfa}}| = \mathcal{O}(N \cdot |\Sigma|)$. We can apply cached quotients (c.f. Section 2.4) to encode and preprocess $\mathcal{S}_{\mathsf{dfa}}$. Proving the well-formedness of id via (5) can be done in time $\widetilde{\mathcal{O}}(n)$ (c.f. Table 3).

Space inefficiency. Although the above way can help prove the well-formedness of id with CQ efficiently, maintaining dfa and encoding $\mathcal{S}_{\mathsf{dfa}}$ into polynomials (for the purpose of proving with CQ) incur a very huge cost, i.e., $|\mathcal{S}_{\mathsf{dfa}}| = \mathcal{O}(N \cdot |\Sigma|)$, especially when constructing a non-subsequence argument by proving correct formation of id relying on such a table. In other words, it took a large amount of time to preprocess the table dfa.

3.2 An Alternative Approach

As we follow the above dynamic programming method to determine id_i based on id_{i-1}, the index id_i is the smallest index s.t. $\mathsf{id}_i > \mathsf{id}_{i-1}$ and either $t_{\mathsf{id}_i} = s_i$ or $\mathsf{id}_i = N+1$, as specified by $\mathsf{dfa}(\mathsf{id}_{i-1}, s_i)$ for $i \in [n]$. We have the following idea.

Space inefficiency. We first observe that, for $i \in [n]$, if we already determine id_{i-1}, then $\mathsf{id}_i = \mathsf{dfa}(\mathsf{id}_{i-1}, s_i)$. However, by definition of dfa, one sees that $\mathsf{dfa}(\mathsf{id}_{i-1}, s_i) = \mathsf{dfa}(h, s_i)$ for any $h \in [\mathsf{id}_{i-1}, \mathsf{id}_i - 1]$ since $\mathsf{id}_i > \mathsf{id}_{i-1}$, if $\mathsf{id}_{i-1} < N+1$ (c.f. Lemma 3), and s_i does not appear in $\mathbf{t}[\mathsf{id}_{i-1}+1 \dots \mathsf{id}_i - 1]$. Hence, this way requires storing a lot of entries of dfa for jumping to id_i from id_{i-1}. We hence raise a question of whether we can save the space to store $\mathsf{dfa}(h, s_i)$ for all $h \in [\mathsf{id}_{i-1}, \mathsf{id}_i - 1]$. In other words, can we efficiently determine id_i from id_{i-1} and s_i without the need to use dfa.

Inverses of dfa and partitions of $[0, N+1]$. Fortunately, we have the following observation. For $c \in \Sigma$, let

$$\mathsf{dfa}_c : [0, N+1] \to [N+1] \text{ mapping } j \mapsto \mathsf{dfa}(j, c)\ \forall j \in [0, N+1]. \tag{6}$$

One sees that the set of all possible outputs of this function is

$$\left\{ j \in [N] \,\middle|\, t_j = c \right\} \cup \{N+1\}. \tag{7}$$

Let m_c be the number of appearances of c in $\mathbf{t}$, i.e., $m_c + 1$ is the cardinality of this set. Parse $\left\{ j \in [N] \,\middle|\, t_j = c \right\} \cup \{N+1\}$ as $\{j_i^{(c)}\}_{i=1}^{m_c+1}$ s.t. $1 \le j_1^{(c)} < j_2^{(c)} < \cdots < j_{m_c}^{(c)} < j_{m_c+1}^{(c)} = N+1$. Let $j_0^{(c)} = 0$. Then, one can observe that their corresponding pre-image intervals (namely, $\mathsf{dfa}_c^{-1}(j_i^{(c)}) = [j_{i-1}^{(c)}, j_i^{(c)} - 1]$, for $i \in [m_c]$, and $\mathsf{dfa}_c^{-1}(j_{m_c+1}^{(c)}) = [j_{m_c}^{(c)}, N+1]$) are consecutive intervals. Clearly, they are disjoint and form a partition of $[0, N+1]$according to the following Definition 4, which we just provide to formulate the constraints, for partitions of $[0, N+1]$ from consecutive intervals, that may appear at various places in this paper.

Definition 4 (Partition of an interval from consecutive intervals). *We say that a sequence of consecutive intervals $([x_i, y_i])_{i=1}^m$, for $m \in \mathbb{N}$, forms a partition of $[L, R]$, for $L, R \in \mathbb{Z}$ and $L \le R$, if (i) $x_{i+1} = y_i + 1\ \forall i \in [m-1]$, and (ii) $L = x_1 \le y_1 < x_2 \le y_2 < \cdots < x_m \le y_m = R$. In other words, $\{[x_i, y_i]\}_{i=1}^m$ are pairwise disjoint, $\bigcup_{i=1}^m [x_i, y_i] = [L, R]$ and $x_{i-1} < x_i\ \forall i \in [2, m]$.*

Hence, we have the following Lemma 5.

Lemma 5. *For $c \in \Sigma$, the consecutive intervals $\{\mathsf{dfa}_c^{-1}(j_i^{(c)})\}_{i=1}^{m_c+1}$ form a partition of $[0, N+1]$ according to Definition 4.*

Proof. The proof is straightforward from the above discussion. □

Therefore, when computing id as in Section 3.1, for $i \in [n]$, if $s_i = c$, to determine id_i from id_{i-1}, we use binary search (can be found in any fundamental algorithmic textbooks, e.g., [43], presented in detail in Appendix B.2) to search for $j_k^{(c)}$ s.t. $\mathsf{id}_{i-1} \in \mathsf{dfa}_c^{-1}(j_k^{(c)})$. Obviously, $\mathsf{id}_i = j_k^{(c)}$.

The above idea leads us to establish the set $\mathcal{S}_{\mathsf{intv}}$[6] in (8) as follows.

$$\mathcal{S}_{\mathsf{intv}} = \bigcup_{c \in \Sigma} \left(\left(\bigcup_{i=1}^{m_c} \{(c, j_{i-1}^{(c)}, j_i^{(c)} - 1)\} \right) \cup \{(c, j_{m_c}^{(c)}, N+1)\} \right) \tag{8}$$

[6] The naming of $\mathcal{S}_{\mathsf{intv}}$ aims to indicate "set of intervals".

as a set containing all triples corresponding to $\mathsf{dfa}_c^{-1}(j_i^{(c)})$ for $c \in \Sigma$ and $i \in [m_c+1]$. Obviously, each $(c, j, j') \in \mathcal{S}_{\mathsf{intv}}$ can be parsed as the following two sub-forms.

- $(c, j, j') = (c, j_{i-1}^{(c)}, j_i^{(c)} - 1)$ for $i \in [m_c]$: Here, we know that $\mathsf{dfa}_c^{-1}(j_i^{(c)}) = [j_{i-1}^{(c)}, j_i^{(c)} - 1]$. We can recognize this case via checking whether $j' < N$ since $j' = j_i^{(c)} - 1$ and $j_i^{(c)} \leq N$ for $i \in [m_c]$. Hence, one can directly understand that $\mathsf{dfa}_c^{-1}(j'+1) = \mathsf{dfa}_c^{-1}(j_i^{(c)}) = [j_{i-1}^{(c)}, j_i^{(c)} - 1] = [j, j']$.
- $(c, j, j') = (c, j_{m_c}^{(c)}, N+1)$: We know that $\mathsf{dfa}_c^{-1}(N+1) = [j_{m_c}^{(c)}, N+1]$. This sub-case is recognized via checking whether $j' = N+1$. Hence, we understand that $\mathsf{dfa}_c^{-1}(j') = \mathsf{dfa}_c^{-1}(N+1) = [j_{m_c}^{(c)}, N+1] = [j, j']$.

Consequently, we see that either $j' < N$ or $j' = N+1$. The case that $j' = N$ does not exist. We have the following Lemma 6.

Lemma 6. *For any* $(c, j, j') \in \Sigma \times [0, N+1] \times [0, N+1]$,

$$(c, j, j') \in \mathcal{S}_{\mathsf{intv}} \iff \begin{array}{l} (j' < N \wedge \mathsf{dfa}_c^{-1}(j'+1) = [j, j']) \\ \vee\ (j' = N+1 \wedge \mathsf{dfa}_c^{-1}(N+1) = [j, j']). \end{array} \tag{9}$$

We defer the proof of this lemma to Appendix B.3.

Computing id based on $\mathcal{S}_{\mathsf{intv}}$. For $i \in [n]$, one can compute id_i based on s_i and id_{i-1} by finding $(s_i, u_i, v_i) \in \mathcal{S}_{\mathsf{intv}}$ s.t. $\mathsf{id}_{i-1} \in [u_i, v_i]$ and setting id_i as follows.

- If $v_i < N$, $\mathsf{id}_i := v_i + 1$ since $\mathsf{id}_{i-1} \in [u_i, v_i] = \mathsf{dfa}_{s_i}^{-1}(v_i+1)$ implying $\mathsf{dfa}(\mathsf{id}_{i-1}, s_i) = v_i + 1$ according to Lemma 6.
- Otherwise, $\mathsf{id}_i := N+1$ since $\mathsf{id}_{i-1} \in [u_i, v_i] = \mathsf{dfa}_{s_i}^{-1}(N+1)$ by Lemma 6.

Hence, we see that $(v_i < N \wedge \mathsf{id}_i = v_i + 1) \vee (v_i = N+1 \wedge \mathsf{id}_i = N+1)$.

For notation, we wrap all u_i and v_i, for $i \in [n]$, into

$$\mathbf{u} = (u_i)_{i=1}^n \text{ and } \mathbf{v} = (v_i)_{i=1}^n. \tag{10}$$

Notice that the searches for all (s_i, u_i, v_i) described above constitute a form

$$\{(s_i, u_i, v_i)\}_{i=1}^n \subseteq \mathcal{S}_{\mathsf{intv}}. \tag{11}$$

This approach has a RHS of much smaller cardinality ($|\mathcal{S}_{\mathsf{intv}}| = N + |\Sigma| = K$) than that of the previous one ($|\mathcal{S}_{\mathsf{dfa}}| = \mathcal{O}(N \cdot |\Sigma|)$ in Section 3.1). However, we are not done yet. When transforming into lookup arguments (c.f. Section 2.4), the LHS of (11) can be captured easily by polynomials encoding $\mathbf{s}$, $\mathbf{u}$, and $\mathbf{v}$ while the RHS is not clearly specified. Here, we need a representation, via some polynomials, that can capture $\mathcal{S}_{\mathsf{intv}}$. Additionally, as discussed above, we need to find $(s_i, u_i, v_i) \in \mathcal{S}_{\mathsf{intv}}$ for each i. Therefore, the representation also needs to enable us to search all (s_i, u_i, v_i) efficiently in an acceptable time, e.g., $\mathcal{O}(\log_2 K)$ time for each (s_i, u_i, v_i). To this end, we will discuss the modeling capturing such a representation and the well-formedness of id in Section 5.

4 Definition of Subsequence Scheme

Let $\mathcal{C} = (\mathsf{C.Setup}, \mathsf{C.Com}, \mathsf{C.Verify})$ be a commitment scheme, whose definition is recalled in Appendix A.5. We discuss an overview for defining Subsequence Scheme in Section 4.1 and formally define it in Section 4.2.

4.1 Overview

Let $n, N \in \mathbb{N}$, $\mathbf{s} = (s_i)_{i=1}^n$, $\mathbf{t} = (t_i)_{i=1}^N$, and Σ be an alphabet capturing characters in $\mathbf{s}$ and $\mathbf{t}$. We denote by $\mathbf{s} \lhd \mathbf{t}$ if $\mathbf{s}$ is a subsequence of $\mathbf{t}$, i.e., one can delete some positions in $\mathbf{t}$ to achieve $\mathbf{s}$; otherwise, we denote by $\mathbf{s} \not\lhd \mathbf{t}$ to indicate that $\mathbf{s}$ is a non-subsequence of $\mathbf{t}$. We hence define the following relation $\mathcal{R}_{\mathsf{ssq}}$ to capture the subsequence relationship between committed $\mathbf{s}$ and $\mathbf{t}$.

$$\mathcal{R}_{\mathsf{ssq}} = \left\{ \mathsf{pp}, c_t, c_s, b;\ \mathbf{t}, \mathbf{s} \middle| c_t = \mathsf{C.Com}_{\mathsf{ck}_t}(\mathbf{t}) \wedge c_s = \mathsf{C.Com}_{\mathsf{ck}_s}(\mathbf{s}) \wedge b = \mathbf{s} \lhd \mathbf{t} \right\} \tag{12}$$

where ck_t and ck_s are contained in pp, c_t and c_s denote the commitments to the strings $\mathbf{t}$ and $\mathbf{s}$, respectively, under commitment keys ck_t and ck_s, respectively.

In this paper, we are only interested in the commitments for polynomials, namely, the polynomial commitments (recalled in Section 2.3 and Appendix A.6). This, hence, follows the paradigm of commit-and-prove SNARKs. In this section, we provide the definition of *Subsequence Scheme* for relation $\mathcal{R}_{\mathsf{ssq}}$.

Intuition. We briefly discuss the intuition for the definition of a subsequence scheme. The scheme aims to show that $b = \mathbf{s} \lhd \mathbf{t}$, where $|\mathbf{t}| \gg |\mathbf{s}|$, according to relation $\mathcal{R}_{\mathsf{ssq}}$. To save proving time, we follow the paradigm as from other schemes [19,10], assuming that $\mathbf{t}$ is known to the prover in advance, to propose a preprocessing phase. In particular, when $\mathbf{t}$ is known, we can compute some components, captured by a tuple of witnesses $\mathsf{wit}_{\mathsf{off}}$, from $\mathbf{t}$. With these components in hand, when $\mathbf{s}$ is known, one can quickly determine a bit b deciding $b = \mathbf{s} \lhd \mathbf{t}$ with time depends on $|\mathbf{s}|$ and loosely depends on $|\mathbf{t}|$, e.g., $\mathcal{O}(|\mathbf{s}| \cdot \log_2 |\mathbf{t}|)$. With this idea, the proof is hence also constructed in two phases as follows.

- *Preprocessing phase.* This phase shows that $\mathsf{wit}_{\mathsf{off}}$ is well-formed w.r.t $\mathbf{t}$. Here, we capture the well-formedness of $\mathsf{wit}_{\mathsf{off}}$ via the predicate $\mathsf{pred}^{(\mathsf{off})}$ defined to be

$$\begin{aligned} &\mathsf{pred}^{(\mathsf{off})}(\mathbf{t}, \mathsf{wit}_{\mathsf{off}}) = 1 \text{ iff } \mathsf{wit}_{\mathsf{off}} \text{ is well-formed w.r.t. } \mathbf{t}, \\ &\text{otherwise } \mathsf{pred}^{(\mathsf{off})}(\mathbf{t}, \mathsf{wit}_{\mathsf{off}}) = 0. \end{aligned} \tag{13}$$

- *Proving phase.* This phase shows that b is correctly computed from $\mathsf{wit}_{\mathsf{off}}$ and $\mathbf{s}$. Here, we capture the correct computation of b via the predicate $\mathsf{pred}^{(\mathsf{on})}$ defined to be

$$\begin{aligned} &\mathsf{pred}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, \mathbf{s}, b) = 1 \text{ iff } b \text{ is correctly computed from } \mathsf{wit}_{\mathsf{off}} \text{ and } \mathbf{t}, \\ &\text{otherwise } \mathsf{pred}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, \mathbf{s}, b) = 0. \end{aligned} \tag{14}$$

The predicates defined above are defined abstractively. Their detailed specification will be specified in the construction of our subsequence scheme (c.f. Section 7). Finally, we capture that $b = \mathbf{s} \lhd \mathbf{t}$ is correctly computed iff those mentioned in the preprocessing and proving phases hold. Specifically,

$$b = \mathbf{s} \lhd \mathbf{t} \iff \exists \mathsf{wit}_{\mathsf{off}} \text{ s.t. } \mathsf{pred}^{(\mathsf{off})}(\mathbf{t}, \mathsf{wit}_{\mathsf{off}}) = 1 \wedge \mathsf{pred}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, \mathbf{s}, b) = 1. \tag{15}$$

Design overview. With the above intuition, we define subsequence schemes as follows.

Setup algorithm Setup. This algorithm simply returns a public paratemer pp used for the entire scheme.

Preprocessing phase covered by $\mathsf{OffProve}$ *and* $\mathsf{OffVerify}$. Taking as inputs the public parameter pp, a string $\mathbf{t}$, and a commitment c_t to $\mathbf{t}$, the algorithm $\mathsf{OffProve}$ is run by prover to generate $\mathsf{wit}_{\mathsf{off}}$, an additional public parameter $\mathsf{pp}_{\mathsf{off}}$, a proof π_{off}, and some additional prover's auxiliary witness $\mathsf{aux}_{\mathsf{off}}$ for computing $\mathsf{OnProve}$ (to be discussed below). We can understand in the following sense. When realizing in constructions, the public parameter $\mathsf{pp}_{\mathsf{off}}$ contains a constant number of commitments to the additional components in $\mathsf{wit}_{\mathsf{off}}$. These commitments are succinct, e.g., sublinear in N, s.t. the verifier does not need to read $\mathsf{wit}_{\mathsf{off}}$. Instead, the verifier simply needs to read $\mathsf{pp}_{\mathsf{off}}$ and verify to validity of the proof π_{off}, via $\mathsf{OffVerify}$, capturing (i) $c_t = \mathsf{C.Com}_{\mathsf{ck}_t}(\mathbf{t})$, (ii) the validity of these commitments w.r.t. the additional components, which we denote by $\mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}_{\mathsf{off}}$, and (iii) $\mathsf{wit}_{\mathsf{off}}$ is well-formed w.r.t. $\mathbf{t}$. Therefore, π_{off} is a proof for

$$\mathcal{R}_{\mathsf{ssq\text{-}off}} = \left\{ \mathsf{pp}, c_t, \mathsf{pp}_{\mathsf{off}};\ \mathbf{t}, \mathsf{wit}_{\mathsf{off}} \;\middle|\; \begin{array}{l} c_t = \mathsf{C.Com}_{\mathsf{ck}_t}(\mathbf{t}) \wedge \mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}_{\mathsf{off}} \\ \wedge \mathsf{pred}^{(\mathsf{off})}(\mathbf{t}, \mathsf{wit}_{\mathsf{off}}) = 1 \end{array} \right\}. \tag{16}$$

Regarding the notation "$\leftrightarrows_{\mathsf{pp}}$", by the discussed intuition, it must satisfy a binding property s.t., following the binding property of commitment schemes (e.g., [33]), it is hard to find $\mathsf{wit}'_{\mathsf{off}} \neq \mathsf{wit}_{\mathsf{off}}$ s.t. $\mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}'_{\mathsf{off}}$.

Proving phase covered by $\mathsf{OnProve}$ *and* $\mathsf{OnVerify}$. Taking as inputs pp, $\mathsf{pp}_{\mathsf{off}}$, $\mathsf{wit}_{\mathsf{off}}$, $\mathsf{aux}_{\mathsf{off}}$, a string $\mathbf{s}$, and a commitment $c_s = \mathsf{C.Com}_{\mathsf{ck}_s}(\mathbf{s})$, this algorithm returns a bit b and a proof π_{on} validating that (i) $c_s = \mathsf{C.Com}_{\mathsf{ck}_s}(\mathbf{s})$, (ii) $\mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}_{\mathsf{off}}$, and (iii) b is correctly computed from

$\mathsf{wit}_{\mathsf{off}}$ and $\mathbf{s}$. Then, the verifier can read pp, $\mathsf{pp}_{\mathsf{off}}$, $c_s = \mathsf{C.Com}_{\mathsf{ck}_s}(\mathbf{s})$, and b, and run $\mathsf{OnVerify}$ to check the validity of π_{on} w.r.t

$$\mathcal{R}_{\mathsf{ssq\text{-}on}} = \left\{ \begin{matrix} \mathsf{pp}, \mathsf{pp}_{\mathsf{off}}, c_s, b; \\ \mathsf{wit}_{\mathsf{off}}, \mathbf{s} \end{matrix} \middle| \begin{matrix} c_s = \mathsf{C.Com}_{\mathsf{ck}_s}(\mathbf{s}) \wedge \mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}_{\mathsf{off}} \\ \wedge \mathsf{pred}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, \mathbf{s}, b) = 1 \end{matrix} \right\}. \tag{17}$$

Security discussion. As "$\leftrightarrows_{\mathsf{pp}}$" satisfies the binding property, we know that $\mathsf{wit}_{\mathsf{off}}$ employed in $\mathcal{R}_{\mathsf{ssq\text{-}off}}$ and $\mathcal{R}_{\mathsf{ssq\text{-}on}}$ is unique, since, otherwise, a cheating prover can break the binding property. As π_{off} and π_{on} are proofs for $\mathcal{R}_{\mathsf{ssq\text{-}off}}$ and $\mathcal{R}_{\mathsf{ssq\text{-}on}}$, respectively, by (knowledge) soundness property, we know that $\mathsf{pred}^{(\mathsf{off})}(\mathbf{t}, \mathsf{wit}_{\mathsf{off}}) = 1$ and $\mathsf{pred}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, \mathbf{s}, b) = 1$. By equivalence (15), we deduce that $b = \mathbf{s} \lhd \mathbf{t}$ and $(\mathsf{pp}, c_t, c_s, b;\ \mathbf{t}, \mathbf{s}) \in \mathcal{R}_{\mathsf{ssq}}$ as desired.

4.2 Definition of a Subsequence Scheme

Syntax. We now specify the syntax of a subsequence scheme in Definition 5.

Definition 5 (Syntax). *Let $\mathcal{C} = (\mathsf{C.Setup}, \mathsf{C.Com}, \mathsf{C.Verify})$ be a secure commitment scheme. Let $\mathcal{R}_{\mathsf{ssq}}$, $\mathcal{R}_{\mathsf{ssq\text{-}on}}$, and $\mathcal{R}_{\mathsf{ssq\text{-}off}}$ be defined as in (12), (16), and (17), respectively, s.t. the underlying predicates $\widetilde{\mathsf{pred}}^{(\mathsf{off})}$ and $\widetilde{\mathsf{pred}}^{(\mathsf{on})}$ in $\mathcal{R}_{\mathsf{ssq\text{-}on}}$ and $\mathcal{R}_{\mathsf{ssq\text{-}off}}$, respectively, satisfy (15). Then, a subsequence scheme $\mathcal{SSQ}$ for the relation $\mathcal{R}_{\mathsf{ssq}}$ is a tuple $\mathcal{SSQ} = (\mathsf{Setup}, \mathsf{OffProve}, \mathsf{OnProve}, \mathsf{OffVerify}, \mathsf{OnVerify})$.*

$\mathsf{Setup}(1^\lambda, N, n, \Sigma) \to \mathsf{pp}$**:** *Return a public parameter pp, given the lengths N and n of $\mathbf{t}$ and $\mathbf{s}$, respectively, and alphabet Σ capturing characters of $\mathbf{t}$ and $\mathbf{s}$. We assume that pp is implicit in the subsequent algorithms and contains the keys ck_t and ck_s (output from $\mathsf{C.Setup}$) for committing $\mathbf{t}$ and $\mathbf{s}$.*

$\mathsf{OffProve}_{\mathsf{pp}}(\mathbf{t}, c_t) \to (\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}})$**:** *On inputs $\mathbf{t}$ and a commitment c_t to $\mathbf{t}$, return an additional public parameter $\mathsf{pp}_{\mathsf{off}}$, a tuple of witnesses $\mathsf{wit}_{\mathsf{off}}$, a proof π_{off} w.r.t. $\mathcal{R}_{\mathsf{ssq\text{-}off}}$, and an additional prover's auxiliary witness $\mathsf{aux}_{\mathsf{off}}$.*

$\mathsf{OffVerify}_{\mathsf{pp}}(c_t, \mathsf{pp}_{\mathsf{off}}, \pi_{\mathsf{off}}) \to \{0,1\}$**:** *Return 1 if π_{off} is valid w.r.t. $\mathcal{R}_{\mathsf{ssq\text{-}off}}$; otherwise, return 0.*

$\mathsf{OnProve}_{\mathsf{pp}}(\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}}, \mathbf{s}, c_s) \to (\{0,1\}, \pi_{\mathsf{on}})$**:** *On inputs $\mathsf{pp}_{\mathsf{off}}$, $\mathsf{wit}_{\mathsf{off}}$, $\mathsf{aux}_{\mathsf{off}}$, $\mathbf{s}$, and a commitment c_s to $\mathbf{s}$, return a bit b and a proof π_{on} w.r.t. $\mathcal{R}_{\mathsf{ssq\text{-}on}}$.*

$\mathsf{OnVerify}_{\mathsf{pp}}(\mathsf{pp}_{\mathsf{off}}, c_s, b, \pi_{\mathsf{on}}) \to \{0,1\}$**:** *Return 1 if π_{on} is valid w.r.t. $\mathcal{R}_{\mathsf{ssq\text{-}on}}$; otherwise, return 0.*

Remark 4. The above syntax aims to split the preprocessing and proving phases into two separate steps. However, one can merge both $\mathsf{OffProve}$ and $\mathsf{OnProve}$ into a single algorithm Prove, taking all inputs and returning all outputs of $\mathsf{OffProve}$ and $\mathsf{OnProve}$. Similarly, one can merge $\mathsf{OffVerify}$ and $\mathsf{OnVerify}$ into Verify.

Correctness. We capture the correctness of $\mathcal{SSQ}$ by the following Definition 6.

Definition 6 (Correctness). *Let $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, N, n, \Sigma)$. For any $\mathbf{t}$, $\mathbf{s}$, c_t, and c_s s.t. $c_t = \mathsf{C.Com}_{\mathsf{ck}_t}(\mathbf{t})$ and $c_s = \mathsf{C.Com}_{\mathsf{ck}_s}(\mathbf{s})$, if $(\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}}) \leftarrow \mathsf{OffProve}_{\mathsf{pp}}(\mathbf{t}, c_t)$ and $(b, \pi_{\mathsf{on}}) \leftarrow \mathsf{OnProve}_{\mathsf{pp}}(\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}}, \mathbf{s}, c_s)$, then*

$$\mathsf{OffVerify}_{\mathsf{pp}}(c_t, \mathsf{pp}_{\mathsf{off}}, \pi_{\mathsf{off}}) = 1 \text{ and } \mathsf{OnVerify}_{\mathsf{pp}}(\mathsf{pp}_{\mathsf{off}}, c_s, b, \pi_{\mathsf{on}}) = 1.$$

Binding of $\leftrightarrows_{\mathsf{pp}}$. The relationship via "$\leftrightarrows_{\mathsf{pp}}$" (underlying $\mathcal{R}_{\mathsf{ssq\text{-}off}}$ and $\mathcal{R}_{\mathsf{ssq\text{-}on}}$ in (16) and (17), respectively) satisfies the binding property as in the following sense. For PPT adversary $\mathcal{A}$, with a negligible probability, $\mathcal{A}$ can output $\mathsf{pp}_{\mathsf{off}}$ and distinct $\mathsf{wit}_{\mathsf{off}}$ and $\mathsf{wit}'_{\mathsf{off}}$ s.t. $\mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}_{\mathsf{off}}$ and $\mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}'_{\mathsf{off}}$.

Knowledge soundness. We define the knowledge soundness of $\mathcal{SSQ}$, in the following Definition 7, in AGM [24] (c.f. Section 2.2). Here, $\mathcal{A}$ must be algebraic, i.e., for every group element generated by an algebraic adversary $\mathcal{A}$, $\mathcal{A}$ must additionally return a representation relative to the group element.

Intuitively, we define knowledge soundness as follows. When $\mathcal{A}$ outputs a proof passing $\mathsf{OffVerify}$ or $\mathsf{OnVerify}$, $\mathcal{A}$ must additionally provide repr containing all representations relative to all group elements output by $\mathcal{A}$. Then, $\mathcal{SSQ}$ is knowledge-sound if there exists an extractor $\mathcal{E}$ able to extract the corresponding witnesses satisfying $\mathcal{R}_{\mathsf{ssq\text{-}off}}$ or $\mathcal{R}_{\mathsf{ssq\text{-}on}}$ w.r.t. $\mathsf{OffVerify}$ or $\mathsf{OnVerify}$, respectively.

Definition 7 (Knowledge soundness). *A subsequence scheme $\mathcal{SSQ}$ satisfies knowledge soundness if, for any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$, taking as inputs the public parameters, group elements, and the representations output by $\mathcal{A}$, s.t., for* $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, N, n, \Sigma)$, *the probabilities*

$$\Pr\left[\begin{matrix}\mathsf{OffVerify}_{\mathsf{pp}}(c_t, \mathsf{pp}_{\mathsf{off}}, \pi_{\mathsf{off}}) = 1 \\ \wedge(\mathsf{pp}, c_t, \mathsf{pp}_{\mathsf{off}}; \mathbf{t}', \mathsf{wit}'_{\mathsf{off}}) \notin \mathcal{R}_{\mathsf{ssq\text{-}off}}\end{matrix}\middle|\begin{matrix}(c_t, \mathsf{pp}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{repr}) \leftarrow \mathcal{A}(\mathsf{pp}), \\ (\mathbf{t}', \mathsf{wit}'_{\mathsf{off}}) \leftarrow \mathcal{E}(\mathsf{pp}, c_t, \mathsf{pp}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{repr})\end{matrix}\right] \text{ and}$$

$$\Pr\left[\begin{matrix}\mathsf{OnVerify}_{\mathsf{pp}}(\mathsf{pp}_{\mathsf{off}}, c_s, b, \pi_{\mathsf{on}}) = 1, \\ \wedge(\mathsf{pp}, \mathsf{pp}_{\mathsf{off}}, c_s, b; \mathsf{wit}'_{\mathsf{off}}, \mathbf{s}') \notin \mathcal{R}_{\mathsf{ssq\text{-}on}}\end{matrix}\middle|\begin{matrix}(\mathsf{pp}_{\mathsf{off}}, c_s, b, \pi_{\mathsf{on}}, \mathsf{repr}) \leftarrow \mathcal{A}(\mathsf{pp}), \\ (\mathsf{wit}'_{\mathsf{off}}, \mathbf{s}') \leftarrow \mathcal{E}(\mathsf{pp}, \mathsf{pp}_{\mathsf{off}}, c_s, b, \pi_{\mathsf{on}}, \mathsf{repr})\end{matrix}\right]$$

are both negligible, i.e., at most $\mathsf{negl}(\lambda)$.

5 Modeling for Subsequence Scheme

Following Section 3, in Section 5.1, we show a representation of $\mathcal{S}_{\mathsf{intv}}$ and model the constraints capturing its well-formedness. In Section 5.2, given the well-formedness of this representation, we model the well-formedness of id so that we can apply a lookup with preprocessing (e.g., CQ) to save the prover time.

5.1 Modeling Preprocessing

As explained in Section 3.2, the set $\mathcal{S}_{\mathsf{intv}}$ is introduced as an intermediate component to help determine id and show its well-formedness efficiently. Recall that $K = N + |\Sigma|$. We provide a representation of $\mathcal{S}_{\mathsf{intv}}$ via the sequences $\mathsf{ch} = (\mathsf{ch}_i)_{i=1}^K$, $\mathsf{lft} = (\mathsf{lft}_i)_{i=1}^K$, and $\mathsf{rgt} = (\mathsf{rgt}_i)_{i=1}^K$[7] defined to satisfy

$$\begin{cases} \{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=1}^K = \mathcal{S}_{\mathsf{intv}}, & \text{(representing } \mathcal{S}_{\mathsf{intv}}) \\ \mathsf{ch}_{i-1} < \mathsf{ch}_i \vee (\mathsf{ch}_{i-1} = \mathsf{ch}_i \wedge \mathsf{lft}_{i-1} < \mathsf{lft}_i)\ \forall i \in [2, K]. & \text{(ordered)} \end{cases} \tag{18}$$

Intuitively, as $(\mathsf{ch}, \mathsf{lft}, \mathsf{rgt})$ is a representation of $\mathcal{S}_{\mathsf{intv}}$, the first condition in (18) aims to capture such a representation. Consequently, according to the RHS of (9) in Lemma 6, each $(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)$ corresponds to an interval in the set of intervals $\{\mathsf{dfa}_{s_j}^{-1}(j)\}_{j=1}^N \cup \{\mathsf{dfa}_c^{-1}(N+1)\}_{c\in\Sigma}$ determined by the inverse of dfa_c defined in (6). The second condition in (18) orders the sequence $(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)_{i=1}^K$ to enable binary search (detailed in Appendix B.6). This allows us to determine id_i from id_{i-1} efficiently, with the total time complexity $\mathcal{O}(n \log_2 K)$ for computing the entire id.

These sequences, i.e., ch, lft, and rgt, will be used as intermediate components to support fast proving of our (non-)subsequence arguments in the following sense. When designing our subsequence scheme, we run a preprocessing phase which aims to show that ch, lft, and rgt are well-formed, i.e., satisfying (18) via constraints formulated in this section (c.f. Lemma 7). Notice that ch, lft, and rgt are constructed based on $\mathbf{t}$ only since they are defined based on $\mathcal{S}_{\mathsf{intv}}$ (c.f. (18)) and $\mathcal{S}_{\mathsf{intv}}$ is defined based on $\mathbf{t}$ (c.f. (8)). Eventually, the proving phase of our scheme simply assumes the well-formedness of these sequences to proceed proving the well-formedness of id via constraints formulated in Section 5.2, and hence concluding whether a string $\mathbf{s}$ is a (non-)subsequence of a preprocessed string $\mathbf{t}$ via exploiting the intermediate components ch, lft, and rgt.

We now discuss the simple constraints on ch, lft, and rgt that are equivalent to (18) and suitable with lookup, permutation, and sumcheck arguments.

Relationship between ch_i and ch_{i+1} for $i \in [K-1]$. From the second condition in (18), we know that ch is a non-decreasing sequence. Hence, $\mathsf{ch}_{i+1} - \mathsf{ch}_i \geq 0$. However, we know that every $c \in \Sigma$ appears in ch since $\mathcal{S}_{\mathsf{intv}}$ contains $(c, j_{m_c}^{(c)}, N+1)$ for all $c \in \Sigma$ (see (8)). Hence, without loss of generality, we assume that $\Sigma = [|\Sigma|] = \{1, \ldots, |\Sigma|\}$. Consequently, $\mathsf{ch}_{i+1} - \mathsf{ch}_i \in \{0, 1\}$ for $i \in [K-1]$. We hence deduce that, for each $c \in \Sigma$, there exist $l_c, r_c \in [K]$ s.t. $l_c \leq r_c$ and, for any $i \in [K]$, $\mathsf{ch}_i = c$ iff $i \in [l_c, r_c]$.

The arrangement from the second condition in (18) also hints us some properties of lft and rgt, to be discussed as follows.

[7] The names ch, lft, and rgt aim to indicate "character", "left", and "right", respectively.

For each $c \in \Sigma$, since $\{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=1}^{K} = \mathcal{S}_{\mathsf{intv}}$ and the above discussion about l_c and r_c, we know that

$$\{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=l_c}^{r_c} = \{(c, j_{i-1}^{(c)}, j_i^{(c)} - 1)\}_{i=1}^{m_c} \cup \{(c, j_{m_c}^{(c)}, N+1)\}$$

where m_c and $\{j_i^{(c)}\}_{i=1}^{m_c}$ are defined in Section 3.2. By the second condition in (18) and Lemmas 5 and 6, we deduce that $([\mathsf{lft}_i, \mathsf{rgt}_i])_{i=l_c}^{r_c}$ is a sequence of consecutive intervals partitioning $[0, N+1]$ (c.f. Definition 4). Hence, by Definition 4, we deduce that, for each $c \in \Sigma$,

$$\begin{cases} \mathsf{lft}_{i+1} = \mathsf{rgt}_i + 1 \ \forall i \in [l_c, r_c - 1], \\ 0 = \mathsf{lft}_{l_c} \le \mathsf{rgt}_{l_c} < \mathsf{lft}_{l_c+1} \le \mathsf{rgt}_{l_c+1} < \cdots < \mathsf{lft}_{r_c} \le \mathsf{rgt}_{r_c} = N+1. \end{cases} \tag{19}$$

Hence, if $\mathsf{ch}_{i+1} - \mathsf{ch}_i = 0$, for $i \in [K-1]$, then $\mathsf{lft}_{i+1} - \mathsf{rgt}_i = 1$. Otherwise, when $\mathsf{ch}_{i+1} - \mathsf{ch}_i = 1$, we know that $\mathsf{rgt}_i = N+1$ and $\mathsf{lft}_{i+1} = 0$ according to (19). Therefore, $\mathsf{lft}_{i+1} - \mathsf{rgt}_i = -(N+1)$.

Putting cases $\mathsf{ch}_{i+1} - \mathsf{ch}_i = 0$ *and* $\mathsf{ch}_{i+1} - \mathsf{ch}_i = 1$ *together.* As we assumed that $\Sigma = [|\Sigma|]$ and analyzed above, (i) $\mathsf{ch}_{i+1} - \mathsf{ch}_i \in \{0, 1\}$, (ii) $\mathsf{ch}_{i+1} - \mathsf{ch}_i = 0 \Rightarrow \mathsf{lft}_{i+1} - \mathsf{rgt}_i = 1$, and (iii) $\mathsf{ch}_{i+1} - \mathsf{ch}_i = 1 \Rightarrow \mathsf{lft}_{i+1} - \mathsf{rgt}_i = -(N+1)$. We can capture these three constraints by the lookup

$$\{(\mathsf{ch}_{i+1} - \mathsf{ch}_i, \mathsf{lft}_{i+1} - \mathsf{rgt}_i)\}_{i=1}^{K-1} \subseteq \{(0, 1), (1, -(N+1))\}. \tag{20}$$

Here, $\mathsf{lft}_{i+1} - \mathsf{rgt}_i = -(N+1)$ only implies $\mathsf{lft}_{i+1} = 0$ and $\mathsf{rgt}_i = N+1$ only if $\mathsf{lft}_{i+1}, \mathsf{rgt}_i \in [0, N+1]$. This is due to the fact that, by some simple manipulation, $\mathfrak{a}, \mathfrak{b} \in [0, N+1], \mathfrak{a} - \mathfrak{b} = -(N+1)$ implying $\mathfrak{a} = 0 \wedge \mathfrak{b} = N+1$ only happens when $\mathfrak{a}$ and $\mathfrak{b}$ are the smallest and largest, respectively, numbers in $[0, N+1]$. Hence, one must additionally constrain

$$\{\mathsf{lft}_i\}_{i=1}^{K} \subseteq [0, N+1] \text{ and } \{\mathsf{rgt}_i\}_{i=1}^{K} \subseteq [0, N+1].$$

$\mathsf{lft}_1 = 0$ *and* $\mathsf{rgt}_K = N+1$. We must enforce $\mathsf{lft}_1 = 0$ and $\mathsf{rgt}_K = N+1$ since these values are not considered in (20). However, the enforcement of $\mathsf{rgt}_K = N+1$ can be skipped since $\mathsf{rgt}_{l_{|\Sigma|}} \le \mathsf{rgt}_{l_{|\Sigma|}+1} \le \cdots \le \mathsf{rgt}_{r_{|\Sigma|}} = \mathsf{rgt}_K$ (c.f. (19)), $\{\mathsf{rgt}_i\}_{i=1}^{K} \subseteq [0, N+1]$, and later we have (21) capturing that $(|\Sigma|, N+1) \in \{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=1}^{K}$. Hence, it must be the case that $\mathsf{rgt}_K = N+1$.

$\mathsf{ch}_1 = 1$ *and* $\mathsf{ch}_K = |\Sigma|$. We note that ch only contains characters in Σ. Constraining $\mathsf{ch}_1 = 1$ and $\mathsf{ch}_K = |\Sigma|$ implies that $1 = \mathsf{ch}_1 \le \mathsf{ch}_2 \le \cdots \le \mathsf{ch}_K = |\Sigma|$, i.e., enforcing both ends of ch, as we already capture $\mathsf{ch}_{i+1} - \mathsf{ch}_i \in \{0, 1\}$ in (20).

Validity of intervals. Each interval $[\mathsf{lft}_i, \mathsf{rgt}_i]$ must be valid, i.e., $0 \le \mathsf{lft}_i \le \mathsf{rgt}_i \le N+1$. Hence, $\mathsf{rgt}_i - \mathsf{lft}_i \in [0, N+1]$ since $\mathsf{lft}_i, \mathsf{rgt}_i \in [0, N+1]$.

Enforcing entries in lft and rgt. The partition of $[0, N+1]$ from $([\mathsf{lft}_i, \mathsf{rgt}_i])_{i=l_c}^{r_c}$ according to Definition 4 is insufficient to capture the first condition in (18) since there is no enforcement on the values of lft_i and rgt_i. To this end, we enforce

$$\{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=1}^{K} = \{(t_i, i-1)\}_{i=1}^{N} \cup \{(c, N+1)\}_{c \in \Sigma} \tag{21}$$

where the RHS of this equation is formed based on (8). By doing so, we can enforce rgt_i to have the correct values. Regarding lft_i, one simply sees that lft_i is automatically determined based on (20).

Putting all together. Based on the above discussion, we have the following Lemma 7, whose proof is deferred to Appendix B.4, capturing the well-formedness of ch, lft, and rgt w.r.t. (18).

Lemma 7. *[Well-formedness of* ch, lft, *and* rgt*] Let* $N \in \mathbb{N}$ *and* $\Sigma = [|\Sigma|] \subseteq \mathbb{F}$ *be an alphabet. Let* $\mathbf{t} = (t_i)_{i=1}^{N} \in \Sigma^N$. *Then,* ch, lft, *and* rgt *determined from* $\mathbf{t}$ *(c.f.* (18)*) are well-formed iff*

$$\begin{cases} \mathsf{ch}_1 = 1, \mathsf{ch}_K = |\Sigma|, \ \mathsf{lft}_1 = 0, \\ \{(\mathsf{ch}_{i+1} - \mathsf{ch}_i, \mathsf{lft}_{i+1} - \mathsf{rgt}_i)\}_{i=1}^{K-1} \subseteq \{(0, 1), (1, -(N+1))\}, \\ \{\mathsf{lft}_i\}_{i=1}^{K} \subseteq [0, N+1], \{\mathsf{rgt}_i\}_{i=1}^{K} \subseteq [0, N+1], \\ \{\mathsf{rgt}_i - \mathsf{lft}_i\}_{i=1}^{K} \subseteq [0, N+1], \\ \{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=1}^{K} = \{(t_i, i-1)\}_{i=1}^{N} \cup \{(c, N+1)\}_{c \in \Sigma}. \end{cases} \tag{22}$$

When constructing our subsequence scheme in Section 7, these sequences, namely, ch, lft, and rgt, are then included in $\mathsf{wit}_{\mathsf{off}}$ as specified in detail in Section 7.1 to help the prover quickly compute the sequence id and show id's well-formedness to the verifier.

5.2 Modeling Proving Phase

We assume that ch, lft, and rgt are already determined. Then, we view them as intermediate components to quickly determine id, deciding whether a length-n string $\mathbf{s} = (s_i)_{i=1}^n$ is a subsequence of $\mathbf{t}$.

From Section 3.2, we know that the sequence $\mathsf{id} = (\mathsf{id}_i)_{i=0}^n$ (c.f. Definition 2) is computed by looking for $\mathbf{u} = (u_i)_{i=1}^n$ and $\mathbf{v} = (v_i)_{i=1}^n$ satisfying

$$\begin{cases} \{(s_i, u_i, v_i)\}_{i=1}^n \subseteq \mathcal{S}_{\mathsf{intv}} = \{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=1}^K \quad \text{(c.f. (11) and (18))}, \\ \mathsf{id}_0 = 0 \quad \wedge \quad \mathsf{id}_{i-1} \in [u_i, v_i] \ \forall i \in [n], \\ (v_i < N \wedge \mathsf{id}_i = v_i + 1) \vee (v_i = N + 1 \wedge \mathsf{id}_i = N + 1) \ \forall i \in [n]. \end{cases} \tag{23}$$

To capture $\mathsf{id}_{i-1} \in [u_i, v_i]$, we note that the provided lookup and sumcheck arguments (c.f. Section 2) are inconvenient with mismatching indices, i.e., accessing id at index $i-1$ while accessing $\mathbf{u}$ and $\mathbf{v}$ at index i. Therefore, we define

$$\widetilde{\mathsf{id}} = (\widetilde{\mathsf{id}}_1, \ldots, \widetilde{\mathsf{id}}_n) := (\mathsf{id}_0, \mathsf{id}_1, \ldots, \mathsf{id}_{n-1}). \tag{24}$$

Then, one can equivalently show $\mathsf{id}_{i-1} \in [u_i, v_i]$ via showing $\widetilde{\mathsf{id}}_i \in [u_i, v_i]$. This can be done by lookup arguments $\{\widetilde{\mathsf{id}}_i - u_i\}_{i=1}^n \subseteq [0, N+1]$ and $\{v_i - \widetilde{\mathsf{id}}_i\}_{i=1}^n \subseteq [0, N+1]$ since $\widetilde{\mathsf{id}}_i, u_i, v_i \in [0, N+1]$.

Regarding $(v_i < N \wedge \mathsf{id}_i = v_i + 1) \vee (v_i = N + 1 \wedge \mathsf{id}_i = N + 1)$, we can capture via $\{(v_i, \mathsf{id}_i)\} \subseteq \{(i-1, i)\}_{i=1}^N \cup \{(N+1, N+1)\}$.

Recall that $\mathbf{s} \lhd \mathbf{t}$ if $\mathsf{id}_n \leq N$. We establish the following Lemma 8 (deferred to Appendix B.5) to capture whether $\mathbf{s} \lhd \mathbf{t}$.

Lemma 8. *Let $n, N \in \mathbb{N}$, and $\Sigma = [|\Sigma|] \subseteq \mathbb{F}$ be an alphabet. Let $\mathbf{s} = (s_i)_{i=1}^n \in \Sigma^n$ and $\mathbf{t} = (t_i)_{i=1}^N \in \Sigma^N$. Then,$\mathbf{s} \lhd \mathbf{t}$ (resp., $\mathbf{s} \not\lhd \mathbf{t}$) iff there exist ch, lft, rgt, id, $\widetilde{\mathsf{id}}$, $\mathbf{u}$, and $\mathbf{v}$ s.t. $(\mathsf{ch}, \mathsf{lft}, \mathsf{rgt})$ is well-formed w.r.t. $\mathbf{t}$ (i.e., (18) holds),*

$$\begin{cases} \mathsf{id}_0 = 0, \\ \{(s_i, u_i, v_i)\}_{i=1}^n = \{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=1}^K, \\ \{\widetilde{\mathsf{id}}_i - u_i\}_{i=1}^n \subseteq [0, N+1], \ \{v_i - \widetilde{\mathsf{id}}_i\}_{i=1}^n \subseteq [0, N+1], \\ \{(v_i, \mathsf{id}_i)\}_{i=1}^n \subseteq \{(i-1, i)\}_{i=1}^N \cup \{(N+1, N+1)\}, \end{cases} \tag{25}$$

and $\mathsf{id}_n \leq N$ (resp., $\mathsf{id}_n = N + 1$).

6 Polynomial Constraints for Subsequence Scheme

Our SNARK for (non-)subsequences requires manipulating polynomials as we use components in Sections 2.3 and 2.4. In this section, we formulate polynomial constraints for capturing those in Lemma 7 and Lemma 8. These polynomial constraints will help us construct our subsequence scheme conveniently in Section 7.

Since components in Lemmas 7 and 8 are bounded and we need to handle some form of tuples, in this section and Section 7, we use a global value B to combine tuples into a single value as follows. For any $(a_i)_{i=1}^c$ and $(b_i)_{i=1}^c$ for some constant c, B is chosen such that $(a_i)_{i=1}^c = (b_i)_{i=1}^c$ if and only if $\sum_{i=1}^c a_i \cdot B^{i-1} = \sum_{i=1}^c b_i \cdot B^{i-1}$. More precisely, we enforce $B \geq 2 \cdot |\Sigma| + N$ and $|\mathbb{F}| > 2 \cdot B^2 \cdot (N+1)$; the rationale is explained in Appendix E.

6.1 Polynomial Constraints for Preprocessing

We denote by $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^K$ the multiplicative subgroup of $\mathbb{F}$ of cardinality K generated by $\omega_{\mathbb{K}}$. For convenience, assume that K is a power of 2.

We formulate Theorem 1 below to capture the constraints in Lemma 7 via polynomials. We refer the readers to Appendix C.1 for a discussion leading to the formal version of Theorem 1, namely, Theorem 6. Then, in Section 7.2, we discuss and describe protocol $\Pi_{\mathsf{ssq\text{-}off}}^{\mathbb{K}}$ for proving the preprocessing based on polynomial constraints in Theorem 6. Theorem 1 employs the following

polynomials, via Lagrange's interpolation (c.f. Section 2.1), encoding $\{B, 1-B\cdot(N+1)\}$ (by g_{diff}), Σ^8, $[0, N+1]$, t, ch, lft, and rgt.[9]

$$\begin{aligned}
&g_{\mathsf{diff}}(X) \text{ s.t. } \omega_{\mathbb{K}}^{0} \mapsto B \text{ and } \omega_{\mathbb{K}}^{i-1} \mapsto 1 - B\cdot(N+1)\ \forall i \in [2, K],\\
&g_{\Sigma}(X) \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto i\ \forall i \in [|\Sigma|] \text{ and } \omega_{\mathbb{K}}^{i-1} \mapsto |\Sigma|\ \ \forall i \in [|\Sigma|+1, K],\\
&g_{\mathsf{suf}\text{-}\Sigma}(X) \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto 0\ \forall i \in [N] \text{ and } \omega_{\mathbb{K}}^{i-1} \mapsto i - N\ \forall i \in [N+1, K],\\
&g_{\mathsf{rg}_N}(X) \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto i-1\ \forall i \in [N+2] \text{ and } \omega_{\mathbb{K}}^{i-1} \mapsto 0\ \forall i \in [N+3, K],\\
&f_{\mathsf{t}}(X) \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto t_i\ \forall i \in [N] \text{ and } \omega_{\mathbb{K}}^{i-1} \mapsto 0\ \forall i \in [N+1, K],\\
&f_{\mathsf{ch}}(X) \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto \mathsf{ch}_i\ \forall i \in [K],\\
&f_{\mathsf{lft}}(X) \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto \mathsf{lft}_i\ \forall i \in [K], \text{ and}\\
&f_{\mathsf{rgt}}(X) \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto \mathsf{rgt}_i\ \forall i \in [K].
\end{aligned} \tag{26}$$

Theorem 1 is now presented as follows.

Theorem 1 (Informal version of Theorem 6). *Let $\mathbb{F}_{<K}[X]$-polynomials $g_{\mathsf{diff}}(X)$, $g_{\Sigma}(X)$, $g_{\mathsf{suf}\text{-}\Sigma}(X)$, $g_{\mathsf{rg}_N}(X)$, $f_{\mathsf{t}}(X)$, $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, and $f_{\mathsf{rgt}}(X)$ be defined as in (26). Let B and $\mathbb{F}$ be appropriately set (detailed conditions for this setting are deferred to Theorem 6). Then, the constraints for well-formedness of ch, lft, and rgt, captured by (22), hold iff there exist $f_{\widehat{\mathsf{ch}}}(X), f_{\widehat{\mathsf{lft}}}(X) \in \mathbb{F}_{<K}[X]$ s.t.*

$$\begin{cases}
f_{\mathsf{t}}(\omega_{\mathbb{K}}^{i-1}) = 0\ \forall i \in [N+1, K],\\
f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{0}) = 1,\ f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{K-1}) = |\Sigma|\,,\ f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{0}) = 0,\\
\{(f_{\widehat{\mathsf{ch}}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1})) + B\cdot(f_{\widehat{\mathsf{lft}}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}))\}_{i=1}^{K}\\
\quad \subseteq \{g_{\mathsf{diff}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K},\\
f_{\widehat{\mathsf{ch}}}(X) = f_{\mathsf{ch}}(\omega_{\mathbb{K}}\cdot X) + |\Sigma|\cdot\ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}}\cdot X),\\
f_{\widehat{\mathsf{lft}}}(X) = f_{\mathsf{lft}}(\omega_{\mathbb{K}}\cdot X),\\
\{f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K},\\
\{f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K},\\
\{f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\Sigma}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K},\\
\{f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K},\\
(f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1}) + B\cdot f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}))_{i=1}^{K}\\
\quad \bowtie (f_{\mathsf{t}}(\omega_{\mathbb{K}}^{i-1}) + g_{\mathsf{suf}\text{-}\Sigma}(\omega_{\mathbb{K}}^{i-1}) + B\cdot g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1}))_{i=1}^{K}
\end{cases} \tag{27}$$

where $\{\ell_{\mathbb{K}}^{(i)}(X)\}_{i=1}^{K}$ is the Lagrange basis over $\mathbb{K}$ (see Section 2.1).

Remark 5. $f_{\widehat{\mathsf{ch}}}(X)$ and $f_{\widehat{\mathsf{lft}}}(X)$ in Theorem 1 aim to respectively encode $\widehat{\mathsf{ch}}$ and $\widehat{\mathsf{lft}}$ in (28). We defer the detailed discussion to Appendix C.1 (c.f. (54)).

$$\begin{aligned}
\widehat{\mathsf{ch}} &= (\widehat{\mathsf{ch}}_1, \ldots, \widehat{\mathsf{ch}}_K) := (\underbrace{\mathsf{ch}_2, \mathsf{ch}_3, \ldots, \mathsf{ch}_K}_{\text{suffix of } \mathsf{ch}}, |\Sigma|+2), \text{ and}\\
\widehat{\mathsf{lft}} &= (\widehat{\mathsf{lft}}_1, \ldots, \widehat{\mathsf{lft}}_K) := (\underbrace{\mathsf{lft}_2, \mathsf{lft}_3, \ldots, \mathsf{lft}_K}_{\text{suffix of } \mathsf{lft}}, 0).
\end{aligned} \tag{28}$$

6.2 Polynomial Constraints for Proving Phase

We formulate Theorem 2 to capture the constraints in Lemma 8 via polynomials. We have $\mathsf{id} = (\mathsf{id}_i)_{i=0}^{n} \in \mathbb{F}^{n+1}$, $\widetilde{\mathsf{id}} = (\widetilde{\mathsf{id}}_i)_{i=1}^{n} \in \mathbb{F}^{n}$, $\mathbf{u} = (u_i)_{i=1}^{n}$, and $\mathbf{v} = (v_i)_{i=1}^{n} \in \mathbb{F}^{n}$. We use $\mathbb{H} = \{\omega_{\mathbb{H}}^{i-1}\}_{i=1}^{n}$ as

[8] We use $g_{\mathsf{suf}\text{-}\Sigma}(X)$ to encode Σ at a "suffix" of a sequence. Specifically, $g_{\mathsf{suf}\text{-}\Sigma}$ encodes $(0^N \| (c)_{c\in\Sigma})$ where 0^N is a length-N zero vector and $(c)_{c\in\Sigma} = (i-N)_{i=N+1}^{K}$.

[9] We denote by $g_{\mathsf{seq}}(X)$ to indicate an encoding to some common sequence seq, known to both prover and verifier, and $f_{\mathsf{seq}}(X)$ to indicate that seq is only known to prover.

a multiplicative subgroup of $\mathbb{F}$ and the polynomials in (29)interpolated over $\mathbb{H}$, to encode $\mathbf{s}$, id, $\widetilde{\mathsf{id}}$, $\mathbf{u}$, and $\mathbf{v}$, respectively, via encoding $\mathbf{a} = (a_i)_{i=1}^n$ as follows.

$$\begin{aligned}
&f_{\mathbf{s}}(X) \text{ s.t. } \omega_{\mathbb{H}}^{i-1} \mapsto s_i \ \forall i \in [n],\\
&f_{\mathsf{id}}(X) \text{ s.t. } \omega_{\mathbb{H}}^{i-1} \mapsto \mathsf{id}_i \ \forall i \in [n], \qquad f_{\widetilde{\mathsf{id}}}(X) \text{ s.t. } \omega_{\mathbb{H}}^{i-1} \mapsto \widetilde{\mathsf{id}}_i \ \forall i \in [n],\\
&f_{\mathbf{u}}(X) \text{ s.t. } \omega_{\mathbb{H}}^{i-1} \mapsto u_i \ \forall i \in [n], \text{ and} \qquad f_{\mathbf{v}}(X) \text{ s.t. } \omega_{\mathbb{H}}^{i-1} \mapsto v_i \ \forall i \in [n].
\end{aligned} \tag{29}$$

We capture system (25) in Lemma 8 via polynomial constraints as follows.

Polynomial constraints for the first line of (25)**.** This line requires showing that $\mathsf{id}_0 = 0$. Notice that $f_{\mathsf{id}}(X)$ (c.f. (29)) does not encode id_0. The only polynomial encoding id_0 is $f_{\widetilde{\mathsf{id}}}(X)$ which can be used to capture $\mathsf{id}_0 = 0$ via $f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}}^0) = 0$. However, verifying this requires running the protocol for polynomial evaluations (c.f. Section 2.3). Fortunately, since $\widetilde{\mathsf{id}}$ is determined based on id, one must show the well-formedness of $f_{\widetilde{\mathsf{id}}}(X)$. On the other hand, as discussed in Section 5.2, id_n must be known to verifier to determine whether $\mathbf{s} \lhd \mathbf{t}$. Let $\mathsf{eoid} := \mathsf{id}_n$[10]. Therefore, one must run an evaluation protocol for checking whether $f_{\mathsf{id}}(\omega_{\mathbb{H}}^{n-1}) = \mathsf{eoid}$. Then, one can verify both $\mathsf{id}_0 = f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}}^0) = 0$ and the well-formedness of $f_{\widetilde{\mathsf{id}}}(X)$ by checking whether

$$f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}} \cdot X) = f_{\mathsf{id}}(X) - \mathsf{eoid} \cdot \ell_{\mathbb{H}}^{(n)}(X). \tag{30}$$

This is explained as follows. $f_{\mathsf{id}}(X) - \mathsf{eoid} \cdot \ell_{\mathbb{H}}^{(n)}(X) = f_{\mathsf{id}}(X) - \mathsf{id}_n \cdot \ell_{\mathbb{H}}^{(n)}(X)$ encodes $(\mathsf{id}_1, \ldots, \mathsf{id}_{K-1}, 0)$. Then, $\widetilde{\mathsf{id}} = (0, \mathsf{id}_1, \ldots, \mathsf{id}_{K-1})$ is a right rotation of $(\mathsf{id}_1, \ldots, \mathsf{id}_{K-1}, 0)$. Hence, $f_{\mathsf{id}}(\omega_{\mathbb{H}}^{n-1}) = \mathsf{eoid}$ and (30) imply $f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}}^0) = 0$.

Since $\mathsf{eoid} = \mathsf{id}_n$, one sees that $b = (\mathsf{id}_n \le N) = (\mathsf{eoid} \le N)$ (c.f. Lemma 3).

Polynomial constraints for the second line of (25)**.** This line contains

$$\{(s_i, u_i, v_i)\}_{i=1}^n \subseteq \{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=1}^K. \tag{31}$$

Since $s_i \in \Sigma$ and $u_i, v_i \in [0, N+1]$, by setting sufficiently large B and $\mathbb{F}$, one can achieve the equivalence

$$(31) \iff \{s_i + B \cdot u_i + B^2 \cdot v_i\}_{i=1}^n \subseteq \{\mathsf{ch}_i + B \cdot \mathsf{lft}_i + B^2 \cdot \mathsf{rgt}_i\}_{i=1}^K. \tag{32}$$

The RHS of equivalence (32) is equivalent to

$$\{f_{\mathbf{s}}(\omega_{\mathbb{H}}^{i-1}) + B \cdot f_{\mathbf{u}}(\omega_{\mathbb{H}}^{i-1}) + B^2 \cdot f_{\mathbf{v}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^n \subseteq \{f_{\mathsf{comb}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K. \tag{33}$$

where $f_{\mathsf{comb}}(X)$[11] is defined in (34) to encode $\{\mathsf{ch}_i + B \cdot \mathsf{lft}_i + B^2 \cdot \mathsf{rgt}_i\}_{i=1}^K$.

$$f_{\mathsf{comb}}(X) := f_{\mathsf{ch}}(X) + B \cdot f_{\mathsf{lft}}(X) + B^2 \cdot f_{\mathsf{rgt}}(X). \tag{34}$$

However, (32) only holds when s_i, u_i, v_i, ch_i, lft_i, rgt_i are bounded in some ranges. In fact, the bounds of $\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i$ are already captured in Theorem 1 via the lookups $\{f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K \subseteq \{g_{\Sigma}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K$, $\{f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K$, $\{f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K$ where $g_{\Sigma}(X)$ and $g_{\mathsf{rg}_N}(X)$ respectively encode Σ and $[0, N+1]$ (c.f. (26)). Hence, the only remaining thing is to guarantee the bounds for values s_i, u_i, v_i, which are straightforward via

$$\begin{aligned}
&\{f_{\mathbf{s}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^n \subseteq \{g_{\Sigma}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K, \{f_{\mathbf{u}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^n \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K, \text{ and}\\
&\{f_{\mathbf{v}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^n \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K.
\end{aligned} \tag{35}$$

Polynomial constraints for the third line of (25)**.** This line contains

$$\{\widetilde{\mathsf{id}}_i - u_i\}_{i=1}^n \subseteq [0, N+1] \text{ and } \{v_i - \widetilde{\mathsf{id}}_i\}_{i=1}^n \subseteq [0, N+1]. \tag{36}$$

These can be done via the lookups

$$\begin{aligned}
&\{f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}}^{i-1}) - f_{\mathbf{u}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^n \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K,\\
&\{f_{\mathbf{v}}(\omega_{\mathbb{H}}^{i-1}) - f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^n \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^K
\end{aligned} \tag{37}$$

[10] The name eoid aims to indicate "end of id", i.e., id_n.

[11] The name $f_{\mathsf{comb}}(X)$ aims to indicate a "combined" encoding of ch, lft, and rgt.

where $g_{\mathsf{rg}_N}(X)$ is defined in (26) to encode $[0, N+1]$. Notice that (36) only holds when $\widetilde{\mathsf{id}}_i, u_i, v_i \in [0, N+1]$. System (35) already captures $u_i, v_i \in [0, N+1]$. For $\widetilde{\mathsf{id}}_i \in [0, N+1]$, we do not need to show via the lookup w.r.t. $g_{\mathsf{rg}_N}(X)$ as in the cases of u_i and v_i. Instead, in (38) below, we know that $\mathsf{id}_i \in [0, N+1]$. In addition, $\widetilde{\mathsf{id}}$ is formed based on id (c.f. (30)). Therefore, $\widetilde{\mathsf{id}}_i \in [0, N+1]$.

Polynomial constraints for the fourth line of (25)**.** This line contains

$$\{(v_i, \mathsf{id}_i)\}_{i=1}^{n} \subseteq \{(i-1, i)\}_{i=1}^{N} \cup \{(N+1, N+1)\}. \tag{38}$$

By setting appropriate B and $\mathbb{F}$, one can achieve the equivalence

$$(38) \iff \{v_i + B \cdot \mathsf{id}_i\}_{i=1}^{n} \subseteq \{i - 1 + B \cdot i\}_{i=1}^{N} \cup \{(B+1) \cdot (N+1)\}. \tag{39}$$

Notice that this equivalence holds assuming $v_i \in [0, N+1]$, which was captured in (35), and $\mathsf{id}_i \in [0, N+1]$. Therefore, to capture $\mathsf{id}_i \in [0, N+1]$, we simply show that

$$\{f_{\mathsf{id}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}. \tag{40}$$

The set $\{v_i + B \cdot \mathsf{id}_i\}_{i=1}^{n}$ in (39) can be encoded by $f_{\mathbf{v}}(X) + B \cdot f_{\mathsf{id}}(X)$. On the other hand, $\{i - 1 + B \cdot i\}_{i=1}^{N} \cup \{(B+1) \cdot (N+1)\}$ is encoded by $g_{\mathsf{inc}}(X)$[12] in (41).

$$g_{\mathsf{inc}}(X) \text{ maps } \omega_{\mathbb{K}}^{i-1} \mapsto \begin{cases} (i-1) + B \cdot i & \forall i \in [N], \\ (B+1) \cdot (N+1) & \forall i \in [N+1, K]. \end{cases} \tag{41}$$

Hence, the RHS of (39) is equivalence to

$$\{f_{\mathbf{v}}(\omega_{\mathbb{H}}^{i-1}) + B \cdot f_{\mathsf{id}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\mathsf{inc}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}. \tag{42}$$

Putting everything together. Based on the above discussions, we establish Theorem 2 below containing polynomial constraints capturing system (25) in Lemma 8.

Theorem 2. *Assume that $f_{\mathbf{t}}(X)$, $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, and $f_{\mathsf{rgt}}(X)$ (c.f. (26)) encode well-formed* ch, lft, *and* rgt, *respectively, w.r.t.* $\mathbf{t}$ *(c.f. (18)). Let $f_{\mathbf{s}}(X)$ and $f_{\mathsf{id}}(X)$ encode* $\mathbf{s}$ *and* $(\mathsf{id}_i)_{i=1}^{n}$, *over* $\mathbb{H}$, *as in* (29). *Let B and $\mathbb{F}$ be appropriately set s.t.* (32) *and* (39) *hold. Then, the constraints for well-formedness of* id, *captured by* (25), *hold iff there exist $f_{\widetilde{\mathsf{id}}}(X), f_{\mathbf{u}}(X), f_{\mathbf{v}}(X) \in \mathbb{F}_{<n}[X]$, and $\mathsf{eoid} \in \mathbb{F}$ satisfying*

$$\begin{cases} f_{\mathsf{id}}(\omega_{\mathbb{H}}^{n-1}) = \mathsf{eoid}, \\ f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}} \cdot X) = f_{\mathsf{id}}(X) - \mathsf{eoid} \cdot \ell_{\mathbb{H}}^{(n)}(X), \\ \{f_{\mathbf{s}}(\omega_{\mathbb{H}}^{i-1}) + B \cdot f_{\mathbf{u}}(\omega_{\mathbb{H}}^{i-1}) + B^2 \cdot f_{\mathbf{v}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{f_{\mathsf{comb}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\mathbf{s}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\Sigma}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\mathbf{u}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\mathbf{v}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}}^{i-1}) - f_{\mathbf{u}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\mathbf{v}}(\omega_{\mathbb{H}}^{i-1}) - f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\mathsf{id}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\mathbf{v}}(\omega_{\mathbb{H}}^{i-1}) + B \cdot f_{\mathsf{id}}(\omega_{\mathbb{H}}^{i-1})\}_{i=1}^{n} \subseteq \{g_{\mathsf{inc}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \end{cases} \tag{43}$$

where $g_{\Sigma}(X)$ and $g_{\mathsf{rg}_N}(X)$ are in (26).

The proof of this Theorem 2 directly follows the above discussion.

7 Our Subsequence Scheme

We construct our subsequence scheme, denoted by $\widetilde{\mathcal{SSQ}}$, following the definition in Section 4. We specify the relations required in Section 7.1, construct protocols $\Pi_{\mathsf{ssq\text{-}off}}^{\mathbb{K}}$ and $\Pi_{\mathsf{ssq\text{-}on}}^{\mathbb{H},\mathbb{K}}$ in Sections 7.2 and 7.3, and describe $\widetilde{\mathcal{SSQ}}$ in Section 7.4.

[12] The naming of $g_{\mathsf{inc}}(X)$ aims to indicate "increment" which applies to all $i \in [K-1]$ via the tuple $(i-1, i)$, i.e., i is achieved via increasing $i-1$ by 1.

7.1 Specifying Relations

According to Section 4, we need to specify the relations $\mathcal{R}_{\mathsf{ssq}}$, $\mathcal{R}_{\mathsf{ssq\text{-}off}}$, and $\mathcal{R}_{\mathsf{ssq\text{-}on}}$. Underlying $\mathcal{R}_{\mathsf{ssq\text{-}off}}$ and $\mathcal{R}_{\mathsf{ssq\text{-}on}}$ are the predicates $\mathsf{pred}^{(\mathsf{off})}$ and $\mathsf{pred}^{(\mathsf{on})}$, respectively, satisfying (15). Hence, in this section, we specify these required relations based on the polynomial constraints that we formulated in Section 6. Since we use polynomials to encode and capture the constraints, we hence specify these relations and predicates based on polynomials and use the relations denoted by $\widetilde{\mathcal{R}}_{\mathsf{ssq}}$, $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}off}}$, $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}on}}$, $\widetilde{\mathsf{pred}}^{(\mathsf{off})}$, and $\widetilde{\mathsf{pred}}^{(\mathsf{on})}$ as the corresponding polynomial versions of $\mathcal{R}_{\mathsf{ssq}}$, $\mathcal{R}_{\mathsf{ssq\text{-}off}}$, $\mathcal{R}_{\mathsf{ssq\text{-}on}}$, $\mathsf{pred}^{(\mathsf{off})}$, and $\mathsf{pred}^{(\mathsf{on})}$. We proceed as follows.

Subsequence relationship $b = \mathbf{s} \lhd \mathbf{t}$ via polynomials. As $f_{\mathbf{t}}(X)$ and $f_{\mathbf{s}}(X)$ encode $\mathbf{t}$ and $\mathbf{s}$, respectively, one sees that

$$b = \mathbf{s} \lhd \mathbf{t} \iff b = (f_{\mathbf{s}}(\omega_{\mathbb{H}}^{i-1}))_{i=1}^{n} \lhd (f_{\mathbf{t}}(\omega_{\mathbb{K}}^{i-1}))_{i=1}^{K}. \tag{44}$$

Commitments. For commitments to polynomials, we follow Remark 2 to prepare committed Lagrange bases $\{\sigma_1(\ell_{\mathbb{K}}^{(i)})\}_{i=1}^{K} = \left\{ \left[\!\left[\ell_{\mathbb{K}}^{(1)}(x) \right]\!\right]_1 \right\}_{i=1}^{K}$, $\{\sigma_2(\ell_{\mathbb{K}}^{(i)})\}_{i=1}^{K} = \left\{ \left[\!\left[\ell_{\mathbb{K}}^{(1)}(x) \right]\!\right]_2 \right\}_{i=1}^{K}$, and $\{\sigma_1(\ell_{\mathbb{H}}^{(i)})\}_{i=1}^{n} = \left\{ \left[\!\left[\ell_{\mathbb{H}}^{(1)}(x) \right]\!\right]_1 \right\}_{i=1}^{n}$ in pp and use them to commit polynomials based on Lagrange's interpolation (c.f. Section 2.1).

Hence, the variant $\widetilde{\mathcal{R}}_{\mathsf{ssq}}$ of $\mathcal{R}_{\mathsf{ssq}}$ can be defined as follows.

Relation $\widetilde{\mathcal{R}}_{\mathsf{ssq}}$. The variant $\widetilde{\mathcal{R}}_{\mathsf{ssq}}$ of $\mathcal{R}_{\mathsf{ssq}}$ (c.f. (12)) is defined as follows.

$$\widetilde{\mathcal{R}}_{\mathsf{ssq}} = \left\{ \begin{array}{l|l} \mathsf{pp}, \sigma_1(f_{\mathbf{t}}), \sigma_1(f_{\mathbf{s}}), b; & \sigma_1(f_{\mathbf{t}}) = [\![f_{\mathbf{t}}(x)]\!]_1, \sigma_1(f_{\mathbf{s}}) = [\![f_{\mathbf{s}}(X)]\!]_1, \\ f_{\mathbf{t}}(X), f_{\mathbf{s}}(X) & \text{and the RHS of (44) holds} \end{array} \right\}.$$

Inputs for $\widetilde{\mathcal{SSQ}}$. Before specifying $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}off}}$, $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}on}}$, $\widetilde{\mathsf{pred}}^{(\mathsf{off})}$, and $\widetilde{\mathsf{pred}}^{(\mathsf{on})}$, we additionally need the following inputs.

Public parameter $\underline{\mathsf{pp}}$ contains $n, N, B \in \mathbb{N}$, $\Sigma = [|\Sigma|] \subseteq \mathbb{F}$, $K = N + |\Sigma|$, $\{[\![x^{i-1}]\!]_1\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_2\}_{i=1}^{K+1}$, $\omega_{\mathbb{H}}, \omega_{\mathbb{K}}, \mathbb{H} = \{\omega_{\mathbb{H}}^{i-1}\}_{i=1}^{n}, \mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}, \ell_{\mathbb{K}}^{(1)}(X), \ell_{\mathbb{H}}^{(n)}, \mathsf{acc}(X) = \prod_{i=N+1}^{K}(\omega_{\mathbb{K}}^{i-1} - X)$, $z_{\mathbb{K}}(X) = X^K - 1$, $g_{\mathsf{diff}}(X)$, $g_{\Sigma}(X)$, $g_{\mathsf{suf}\text{-}\Sigma}(X)$, $g_{\mathsf{rg}_N}(X)$, $g_{\mathsf{inc}}(X)$, $\{\sigma_1(\ell_{\mathbb{K}}^{(i)})\}_{i=1}^{K} = \left\{ \left[\!\left[\ell_{\mathbb{K}}^{(1)}(x) \right]\!\right]_1 \right\}_{i=1}^{K}$, $\{\sigma_2(\ell_{\mathbb{K}}^{(i)})\}_{i=1}^{K} = \left\{ \left[\!\left[\ell_{\mathbb{K}}^{(1)}(x) \right]\!\right]_2 \right\}_{i=1}^{K}$, $\{\sigma_1(\ell_{\mathbb{H}}^{(i)})\}_{i=1}^{n} = \left\{ \left[\!\left[\ell_{\mathbb{H}}^{(1)}(x) \right]\!\right]_1 \right\}_{i=1}^{n}$, $\sigma_2(\mathsf{acc}) = [\![\mathsf{acc}(x)]\!]_2$, $\sigma_2(z_{\mathbb{K}}) = [\![z_{\mathbb{K}}(x)]\!]_2$, $\sigma_2(g_{\mathsf{diff}}) = [\![g_{\mathsf{diff}}(x)]\!]_2$, $\sigma_2(g_{\Sigma}) = [\![g_{\Sigma}(x)]\!]_2$, $\sigma_2(g_{\mathsf{suf}\text{-}\Sigma}) = [\![g_{\mathsf{suf}\text{-}\Sigma}(x)]\!]_2$, $\sigma_2(g_{\mathsf{rg}_N}) = [\![g_{\mathsf{rg}_N}(x)]\!]_2$, and $\sigma_2(g_{\mathsf{inc}}) = [\![g_{\mathsf{inc}}(x)]\!]_2$ where $g_{\mathsf{diff}}(X)$, $g_{\Sigma}(X)$, $g_{\mathsf{suf}\text{-}\Sigma}(X)$, $g_{\mathsf{rg}_N}(X)$, $f_{\mathbf{t}}(X)$, $g_{\mathsf{inc}}(X)$, and $\mathsf{acc}(x)$ are defined in (26), (41), and Section 2.3.

$\underline{\mathsf{wit}_{\mathsf{off}}}$ contains $f_{\mathbf{t}}(X)$, $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, $f_{\mathsf{rgt}}(X)$, $f_{\widehat{\mathsf{ch}}}(X)$, $f_{\widehat{\mathsf{lft}}}(X)$, $f_{\mathsf{comb}}(X)$.

$\underline{\mathsf{pp}_{\mathsf{off}}}$ has $\sigma_2(f_{\mathbf{t}}) = [\![f_{\mathbf{t}}(x)]\!]_2$, $\sigma_1(f_{\mathsf{ch}}) = [\![f_{\mathsf{ch}}(x)]\!]_1$, $\sigma_1(f_{\mathsf{lft}}) = [\![f_{\mathsf{lft}}(x)]\!]_1$, $\sigma_1(f_{\mathsf{rgt}}) = [\![f_{\mathsf{rgt}}(x)]\!]_1$, $\sigma_1(f_{\widehat{\mathsf{ch}}}) = [\![f_{\widehat{\mathsf{ch}}}(x)]\!]_1$, $\sigma_1(f_{\widehat{\mathsf{lft}}}) = [\![f_{\widehat{\mathsf{lft}}}(x)]\!]_1$, $\sigma_2(f_{\mathsf{comb}}) = [\![f_{\mathsf{comb}}(x)]\!]_2$.

Notice that pp contains $\sigma_2(f_{\mathbf{t}})$ which is committed under $\{[\![x^{i-1}]\!]_2\}_{i=1}^{K+1}$. This is convenient for us to run $\Pi_{\mathsf{cq}}^{\mathbb{H},\mathbb{K}}$ w.r.t. $\mathcal{R}_{\mathsf{cq}}$ as specified in Section 2.4.

Predicates $\widetilde{\mathsf{pred}}^{(\mathsf{off})}$ and $\widetilde{\mathsf{pred}}^{(\mathsf{on})}$. Since $\mathsf{wit}_{\mathsf{off}}$ is already determined, based on (27) in Theorem 1, we have relation $\widetilde{\mathsf{pred}}^{(\mathsf{off})}(f_{\mathbf{t}}(X), \mathsf{wit}_{\mathsf{off}})$ defined to be

$$\widetilde{\mathsf{pred}}^{(\mathsf{off})}(f_{\mathbf{t}}(X), \mathsf{wit}_{\mathsf{off}}) = 1 \iff f_{\mathbf{t}}(X) \text{ and } \mathsf{wit}_{\mathsf{off}} \text{ together satisfy Theorem 1.}$$

Theorem 1 can be satisfied because the required polynomials can be found in $\mathsf{wit}_{\mathsf{off}}$ and pp. Notice that polynomials in pp can be determined by any party. Similarly, we can define $\widetilde{\mathsf{pred}}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, f_{\mathbf{s}}(X), b)$ to be

$$\widetilde{\mathsf{pred}}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, f_{\mathbf{s}}(X), b) = 1 \iff \begin{array}{l} \mathsf{wit}_{\mathsf{off}} \text{ and } f_{\mathbf{s}}(X) \text{ satisfy Theorem 2 and} \\ b = (\mathsf{eoid} \leq N) \text{ where } \mathsf{eoid} \text{ is in Theorem 2.} \end{array}$$

Obviously,

$$b=(f_{\mathbf{s}}(\omega_{\mathbb{H}}^{i-1}))_{i=1}^{n} \lhd (f_{\mathbf{t}}(\omega_{\mathbb{K}}^{i-1}))_{i=1}^{N} \Longleftrightarrow \exists \mathsf{wit}_{\mathsf{off}} \text{ s.t. } \begin{cases} \widetilde{\mathsf{pred}}^{(\mathsf{off})}(f_{\mathbf{t}}(X), \mathsf{wit}_{\mathsf{off}}) = 1 \\ \widetilde{\mathsf{pred}}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, f_{\mathbf{s}}(X), b) = 1. \end{cases}$$

Notation "$\leftrightarrows_{\mathsf{pp}}$". Since $\mathsf{wit}_{\mathsf{off}}$ and $\mathsf{pp}_{\mathsf{off}}$ are already specified above, as required in Section 4, we can define "$\leftrightarrows_{\mathsf{pp}}$" to be

$$\mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}_{\mathsf{off}} \iff \text{commitments in } \mathsf{pp}_{\mathsf{off}} \text{ are valid w.r.t. those in } \mathsf{wit}_{\mathsf{off}}.$$

Clearly, "$\leftrightarrows_{\mathsf{pp}}$" satisfies the binding property defined in Section 4.1 according to the binding of the underlying commitments.

Relation $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}off}}$. This variant of $\mathcal{R}_{\mathsf{ssq\text{-}off}}$ (c.f. (16)) is trivially defined as follows.

$$\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}off}} = \left\{ \mathsf{pp}, \sigma_1(f_{\mathbf{t}}), \mathsf{pp}_{\mathsf{off}}; f_{\mathbf{t}}(X), \mathsf{wit}_{\mathsf{off}} \;\middle|\; \begin{array}{l} \sigma_1(f_{\mathbf{t}}) = [\![f_{\mathbf{t}}(x)]\!]_1 \wedge \mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}_{\mathsf{off}} \\ \wedge \widetilde{\mathsf{pred}}^{(\mathsf{off})}(f_{\mathbf{t}}(X), \mathsf{wit}_{\mathsf{off}}) = 1 \end{array} \right\}.$$

Relation $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}on}}$. This variant of $\mathcal{R}_{\mathsf{ssq\text{-}on}}$ (c.f. (17)) is trivially defined as follows.

$$\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}on}} = \left\{ \mathsf{pp}, \mathsf{pp}_{\mathsf{off}}, \sigma_1(f_{\mathbf{s}}), b; \mathsf{wit}_{\mathsf{off}}, f_{\mathbf{s}}(X) \;\middle|\; \begin{array}{l} \sigma_1(f_{\mathbf{s}}) = [\![f_{\mathbf{s}}(x)]\!]_1 \wedge \mathsf{pp}_{\mathsf{off}} \leftrightarrows_{\mathsf{pp}} \mathsf{wit}_{\mathsf{off}} \\ \wedge \widetilde{\mathsf{pred}}^{(\mathsf{on})}(\mathsf{wit}_{\mathsf{off}}, f_{\mathbf{s}}(X), b) = 1 \end{array} \right\}.$$

Setting of B and $\mathbb{F}$ for Theorems 1 and 2. Additionally, as $\widetilde{\mathsf{pred}}^{(\mathsf{off})}$ and $\widetilde{\mathsf{pred}}^{(\mathsf{on})}$ capture those in Theorems 1 and 2, respectively, we need to set B and $\mathbb{F}$ appropriately as required in theorems. Therefore, we set B and $\mathbb{F}$ s.t.

$$B \geq 2 \cdot |\Sigma| + N \text{ and } |\mathbb{F}| > 2 \cdot B^2 \cdot (N+1) \tag{45}$$

The explanation for such settings is discussed in Appendix E based on fundamental mathematical computations.

7.2 Protocol for Preprocessing

We describe $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ in Figure 1 for proving $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}off}}$. As underlying $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}off}}$ is $\widetilde{\mathsf{pred}}^{(\mathsf{off})}$ capturing (27), $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ mainly focuses on proving this system.

Regarding $f_{\widehat{\mathsf{ch}}}(X) = f_{\mathsf{ch}}(\omega_{\mathbb{K}} \cdot X) + |\Sigma| \cdot \ell^{(1)}_{\mathbb{K}}(\omega_{\mathbb{K}} \cdot X)$ and $f_{\widehat{\mathsf{lft}}}(X) = f_{\mathsf{lft}}(\omega_{\mathbb{K}} \cdot X)$ in (27), prover proves them by receiving $\rho \xleftarrow{\$} \mathbb{F}$, from verifier, and showing that

$$f_{\widehat{\mathsf{ch}}}(\rho) = f_{\mathsf{ch}}(\omega_{\mathbb{K}} \cdot \rho) + |\Sigma| \cdot \ell^{(1)}_{\mathbb{K}}(\omega_{\mathbb{K}} \cdot \rho) \text{ and } f_{\widehat{\mathsf{lft}}}(\rho) = f_{\mathsf{lft}}(\omega_{\mathbb{K}} \cdot \rho). \tag{46}$$

(46) holds with probability $1 - \mathcal{O}(K/|\mathbb{F}|)$ according to Schwartz-Zippel Lemma(recalled in Appendix A.4).

Besides running $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$, $\mathcal{P}$ also runs $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$ (c.f. Figure 5, deferred to Appendix A.7) to preprocess $g_{\mathsf{rg}_N}(X)$, $g_{\mathsf{inc}}(X)$, $g_{\Sigma}(X)$, and $f_{\mathsf{comb}}(X)$ using $\sigma_2(g_{\mathsf{rg}_N})$, $\sigma_2(g_{\mathsf{inc}})$, $\sigma_2(g_{\Sigma})$, and $\sigma_2(f_{\mathsf{comb}})$, respectively, where $f_{\mathsf{comb}}(X)$ is determined from $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, and $f_{\mathsf{rgt}}(X)$ (c.f. (34)), to obtain

$$\mathsf{aux}_{\mathsf{off}} = (\mathsf{prep}(g_{\mathsf{rg}_N}), \mathsf{prep}(g_{\mathsf{inc}}), \mathsf{prep}(g_{\Sigma}), \mathsf{prep}(f_{\mathsf{comb}})). \tag{47}$$

These preprocessed components are employed to run $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$ inside $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ (c.f. Figure 2), to be presented in Figure 2.

Analysis. The completeness of $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ is straightforward. Knowledge soundness of $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ follows Theorem 3 whose proof is deferred to Appendix C.2.

Theorem 3. *$\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ is knowledge-sound in AGM assuming the security of PCSs, $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$, and $\Pi^{\mathbb{K}}_{\mathsf{perm}}$. The soundness error is $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$.*

Protocol $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$. We assume that $\mathcal{P}$ already determined $\mathsf{wit}_{\mathsf{off}}$. **Common inputs** are pp, $\sigma_1(f_{\mathbf{t}})$, and $\mathsf{pp}_{\mathsf{off}}$. $\mathcal{P}$**'s inputs** contain $f_{\mathbf{t}}(X)$ and $\mathsf{wit}_{\mathsf{off}}$.
Execution:
1. $\mathcal{V}$ samples and sends $\rho \xleftarrow{\$} \mathbb{F}$ to $\mathcal{P}$.
2. Both parties run the following protocols:
(a) Run the evaluation protocol of PCS to evaluate and check whether
$f_{\widehat{\mathsf{ch}}}(\rho) = f_{\mathsf{ch}}(\omega_{\mathbb{K}} \cdot \rho) + |\Sigma| \cdot \ell^{(1)}_{\mathbb{K}}(\omega_{\mathbb{K}} \cdot \rho)$ using $\sigma_1(f_{\widehat{\mathsf{ch}}})$, $\sigma_1(f_{\mathsf{ch}})$, $\sigma_1(\ell^{(1)}_{\mathbb{K}})$;
$f_{\widehat{\mathsf{lft}}}(\rho) = f_{\mathsf{lft}}(\omega_{\mathbb{K}} \cdot \rho)$ using $\sigma_1(f_{\widehat{\mathsf{lft}}})$, $\sigma_1(f_{\mathsf{lft}})$;
$f_{\mathsf{ch}}(\omega^0_{\mathbb{K}}) = 1,\ f_{\mathsf{ch}}(\omega^{K-1}_{\mathbb{K}}) = |\Sigma|,$ and $f_{\mathsf{lft}}(\omega^0_{\mathbb{K}}) = 0$ using $\sigma_1(f_{\mathsf{ch}})$, $\sigma_1(f_{\mathsf{lft}})$;
evaluation of $f_{\mathbf{t}}(\rho)$ is the same for $f_{\mathbf{t}}(X)$ in both $\sigma_1(f_{\mathbf{t}})$, $\sigma_2(f_{\mathbf{t}})$;
batch evaluation for $f_{\mathbf{t}}(\omega^{i-1}_{\mathbb{K}}) = 0\ \forall i \in [N+1, K]$ using $\sigma_1(f_{\mathbf{t}})$, $\sigma_2(\mathsf{acc})$;
$f_{\mathsf{comb}}(\rho) = f_{\mathsf{ch}}(\rho) + B \cdot f_{\mathsf{lft}}(\rho) + B^2 \cdot f_{\mathsf{rgt}}(\rho)$ using $\sigma_2(f_{\mathsf{comb}}), \sigma_1(f_{\mathsf{ch}}), \sigma_1(f_{\mathsf{lft}}), \sigma_1(f_{\mathsf{rgt}})$.
(b) $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ for
- $\{(f_{\widehat{\mathsf{ch}}}(\omega^{i-1}_{\mathbb{K}}) - f_{\mathsf{ch}}(\omega^{i-1}_{\mathbb{K}})) + B \cdot (f_{\widehat{\mathsf{lft}}}(\omega^{i-1}_{\mathbb{K}}) - f_{\mathsf{rgt}}(\omega^{i-1}_{\mathbb{K}}))\}^K_{i=1} \subseteq \{g_{\mathsf{diff}}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\widehat{\mathsf{ch}}})$, $\sigma_1(f_{\mathsf{ch}})$, $\sigma_1(f_{\widehat{\mathsf{lft}}})$, $\sigma_1(f_{\mathsf{rgt}})$, $\sigma_2(g_{\mathsf{diff}})$;
- $\{f_{\mathsf{lft}}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1} \subseteq \{g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathsf{lft}})$, $\sigma_2(g_{\mathsf{rg}_N})$;
- $\{f_{\mathsf{rgt}}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1} \subseteq \{g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathsf{rgt}})$, $\sigma_2(g_{\mathsf{rg}_N})$;
- $\{f_{\mathsf{ch}}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1} \subseteq \{g_{\Sigma}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathsf{ch}})$, $\sigma_2(g_{\Sigma})$;
- $\{f_{\mathsf{rgt}}(\omega^{i-1}_{\mathbb{K}}) - f_{\mathsf{lft}}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1} \subseteq \{g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathsf{rgt}})$, $\sigma_1(f_{\mathsf{lft}})$, $\sigma_2(g_{\mathsf{rg}_N})$.

(c) $\Pi^{\mathbb{K}}_{\mathsf{perm}}$ for
- $(f_{\mathsf{ch}}(\omega^{i-1}_{\mathbb{K}}) + B \cdot f_{\mathsf{rgt}}(\omega^{i-1}_{\mathbb{K}}))^K_{i=1} \bowtie (f_{\mathbf{t}}(\omega^{i-1}_{\mathbb{K}}) + g_{\mathsf{suf}\text{-}\Sigma}(\omega^{i-1}_{\mathbb{K}}) + B \cdot g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}}))^K_{i=1}$ using $\sigma_1(f_{\mathsf{ch}})$, $\sigma_1(f_{\mathsf{rgt}})$, $\sigma_2(f_{\mathbf{t}})$, $\sigma_2(g_{\mathsf{suf}\text{-}\Sigma})$, $\sigma_2(g_{\mathsf{rg}_N})$.

Fig. 1. Protocol $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ for preprocessing of fast (non-)subsequence arguments.

Protocol $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$. Assume that $\mathcal{P}$ already determined id (c.f. Definition 2), $\widetilde{\mathsf{id}}$ (c.f. (24)), $\mathbf{u}$, and $\mathbf{v}$ (c.f. (10)). $\mathcal{P}$ also computed eoid and $f_{\mathbf{s}}(X)$, $f_{\mathsf{id}}(X)$, $f_{\widetilde{\mathsf{id}}}(X)$, $f_{\mathbf{u}}(X)$, and $f_{\mathbf{v}}(X)$. **Common inputs** are pp, $\sigma_1(f_{\mathbf{s}})$, and $\mathsf{pp}_{\mathsf{off}}$. $\mathcal{P}$**'s inputs** contain $f_{\mathbf{s}}(X)$, $\mathsf{wit}_{\mathsf{off}}$, $\mathsf{aux}_{\mathsf{off}}$(c.f. (47)), and $f_{\mathsf{id}}(X)$, $f_{\widetilde{\mathsf{id}}}(X)$, $f_{\mathbf{u}}(X)$, $f_{\mathbf{v}}(X)$.
Execution:
1. $\mathcal{P}$ computes and sends eoid, $\sigma_1(f_{\mathsf{id}}) = [\![f_{\mathsf{id}}(x)]\!]_1$, $\sigma_1(f_{\widetilde{\mathsf{id}}}) = [\![f_{\widetilde{\mathsf{id}}}(x)]\!]_1$, $\sigma_1(f_{\mathbf{u}}) = [\![f_{\mathbf{u}}(x)]\!]_1$, and $\sigma_1(f_{\mathbf{v}}) = [\![f_{\mathbf{v}}(x)]\!]_1$ to $\mathcal{V}$.
2. $\mathcal{V}$ samples and sends $\chi \xleftarrow{\$} \mathbb{F}$ to $\mathcal{P}$.
3. Both parties run the following invocations to $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$ (c.f. Figure 6, deferred to Appendix A.7).
(a) Run the evaluation protocol of PCS to prove $f_{\mathsf{id}}(\omega^{n-1}_{\mathbb{H}}) = \mathsf{eoid}$, and
$f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}} \cdot \chi) = f_{\mathsf{id}}(\chi) - \mathsf{eoid} \cdot \ell^{(n)}_{\mathbb{H}}(\chi)$.
(b) $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$ for
- $\{f_{\mathbf{s}}(\omega^{i-1}_{\mathbb{H}}) + B \cdot f_{\mathbf{u}}(\omega^{i-1}_{\mathbb{H}}) + B^2 \cdot f_{\mathbf{v}}(\omega^{i-1}_{\mathbb{H}})\}^n_{i=1} \subseteq \{f_{\mathsf{comb}}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathbf{s}})$, $\sigma_1(f_{\mathbf{u}})$, $\sigma_1(f_{\mathbf{v}})$, $\mathsf{prep}(f_{\mathsf{comb}})$;
- $\{f_{\mathbf{s}}(\omega^{i-1}_{\mathbb{H}})\}^n_{i=1} \subseteq \{g_{\Sigma}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathbf{s}})$, $\mathsf{prep}(g_{\Sigma})$;
- $\{f_{\mathbf{u}}(\omega^{i-1}_{\mathbb{H}})\}^n_{i=1} \subseteq \{g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathbf{u}})$, $\mathsf{prep}(g_{\mathsf{rg}_N})$;
- $\{f_{\mathbf{v}}(\omega^{i-1}_{\mathbb{H}})\}^n_{i=1} \subseteq \{g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathbf{v}})$, $\mathsf{prep}(g_{\mathsf{rg}_N})$;
- $\{f_{\widetilde{\mathsf{id}}}(\omega^{i-1}_{\mathbb{H}}) - f_{\mathbf{u}}(\omega^{i-1}_{\mathbb{H}})\}^n_{i=1} \subseteq \{g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\widetilde{\mathsf{id}}})$, $\sigma_1(f_{\mathbf{u}})$, $\mathsf{prep}(g_{\mathsf{rg}_N})$;
- $\{f_{\mathbf{v}}(\omega^{i-1}_{\mathbb{H}}) - f_{\widetilde{\mathsf{id}}}(\omega^{i-1}_{\mathbb{H}})\}^n_{i=1} \subseteq \{g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathbf{v}})$, $\sigma_1(f_{\widetilde{\mathsf{id}}})$, $\mathsf{prep}(g_{\mathsf{rg}_N})$;
- $\{f_{\mathsf{id}}(\omega^{i-1}_{\mathbb{H}})\}^n_{i=1} \subseteq \{g_{\mathsf{rg}_N}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathsf{id}})$, $\mathsf{prep}(g_{\mathsf{rg}_N})$;
- $\{f_{\mathbf{v}}(\omega^{i-1}_{\mathbb{H}}) + B \cdot f_{\mathsf{id}}(\omega^{i-1}_{\mathbb{H}})\}^n_{i=1} \subseteq \{g_{\mathsf{inc}}(\omega^{i-1}_{\mathbb{K}})\}^K_{i=1}$ using $\sigma_1(f_{\mathbf{v}})$, $\sigma_1(f_{\mathsf{id}})$, $\mathsf{prep}(g_{\mathsf{inc}})$.

Fig. 2. Protocol $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ for proving phase of fast (non-)subsequence arguments.

7.3 Protocol for Proving Phase

We construct $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ in Figure 2 for $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}on}}$. As underlying $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}on}}$ is $\widetilde{\mathsf{pred}}^{(\mathsf{on})}$ capturing (43), protocol $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ mainly focuses on proving this system.

Regarding $f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}} \cdot X) = f_{\mathsf{id}}(X) - \mathsf{eoid} \cdot \ell_{\mathbb{H}}^{(n)}(X)$, prover can prove it by receiving $\chi \xleftarrow{\$} \mathbb{F}$, from verifier, and showing that $f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}} \cdot \chi) = f_{\mathsf{id}}(\chi) - \mathsf{eoid} \cdot \ell_{\mathbb{H}}^{(n)}(\chi)$.

Analysis. The completeness of $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ is straightforward. Knowledge soundness of $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ follows Theorem 4 whose proof is deferred to Appendix D.1.

Theorem 4. *$\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ satisfies knowledge soundness, with soundness error bounded by $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$, in AGM assuming the security of the PCSs and $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}}$.*

7.4 Description

We describe our subsequence scheme $\widetilde{\mathcal{SSQ}}$ (based on $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ and $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ in Figures 1 and 2, respectively), following Definition 5. The description is as follows.

$\mathsf{Setup}(1^\lambda, N, n, \Sigma) \to \mathsf{pp}$**:** $\mathcal{P}$ works as follows.
- Abort if n and $K = N + |\Sigma|$ are not powers of 2.
- Determine B and $\mathbb{F}$ according to (45). Moreover, $\mathbb{F}$ must be sufficiently large to guarantee the security, and $n \mid |\mathbb{F}| - 1$ and $K \mid |\mathbb{F}| - 1$.
- Determine and return pp, as specified in Section 7.1.

$\mathsf{OffProve}_{\mathsf{pp}}(\sigma_1(f_{\mathbf{t}}), f_{\mathbf{t}}(X)) \to (\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}})$**:** Assume that $\mathcal{P}$ computed $f_{\mathbf{t}}(X)$ and $\sigma_1(f_{\mathbf{t}})$ in advance from $\mathbf{t}$. $\mathcal{P}$ runs as follows.
- Determine sequences ch, lft, and rgt (c.f. (18)). Then, additionally determine sequences $\widehat{\mathsf{ch}}$, $\widehat{\mathsf{lft}}$ (specified in (28)).
- Encode these sequences into polynomials in $\mathsf{wit}_{\mathsf{off}}$ and commit to them to obtain $\mathsf{pp}_{\mathsf{off}}$ as specified in Section 7.1.
- Run $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$ (c.f. Figure 5, deferred to Appendix A.7) to preprocess $g_{\mathsf{rg}_N}(X)$, $g_{\mathsf{inc}}(X)$, $g_\Sigma(X)$, $f_{\mathsf{comb}}(X)$ from using $\sigma_2(g_{\mathsf{rg}_N})$, $\sigma_2(g_{\mathsf{inc}})$, $\sigma_2(g_\Sigma)$, $\sigma_2(f_{\mathsf{comb}})$, respectively, in pp and $\mathsf{pp}_{\mathsf{off}}$ where $f_{\mathsf{comb}}(X)$ is determined from $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, $f_{\mathsf{rgt}}(X)$ (c.f. (34)), to obtain $\mathsf{aux}_{\mathsf{off}}$ (c.f. (47)).
- Run the non-interactive version (Fiat-Shamir transform [22]) of $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ w.r.t. the above determined inputs to obtain the proof π_{off}.
- Return $(\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}})$.

$\mathsf{OffVerify}_{\mathsf{pp}}(\sigma_1(f_{\mathbf{t}}), \mathsf{pp}_{\mathsf{off}}, \pi_{\mathsf{off}}) \to \{0,1\}$**:** If π_{off} is valid w.r.t. relation $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}off}}$ specified in Section 7.1, return 1; otherwise, return 0.

$\mathsf{OnProve}_{\mathsf{pp}}(\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}}, f_{\mathbf{s}}(X), \sigma(f_{\mathbf{s}})) \to (\{0,1\}, \pi_{\mathsf{on}})$**:** Assume that $\mathcal{P}$ computed $f_{\mathbf{s}}(X)$ and $\sigma_1(f_{\mathbf{s}})$ in advance from $\mathbf{s}$. $\mathcal{P}$ runs as follows.
- Determine id, $\widetilde{\mathsf{id}}$, $\mathbf{u}$, and $\mathbf{u}$ as specified in Section 3.2 and (24). Determine $\mathsf{eoid} := \mathsf{id}_n$, as specified in Section 6.2, and $b := (\mathsf{eoid} \le N)$ (c.f. (4)).
- Encode the id, $\widetilde{\mathsf{id}}$, $\mathbf{u}$, $\mathbf{v}$ into $f_{\mathsf{id}}(X)$, $f_{\widetilde{\mathsf{id}}}(X)$, $f_{\mathbf{u}}(X)$, $f_{\mathbf{v}}(X)$ (c.f. (29)).
- Run the non-interactive version (Fiat-Shamir transform [22]) of $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ w.r.t. the above determined components to obtain the proof π_{on}.
- Return (b, π_{on}).

$\mathsf{OnVerify}_{\mathsf{pp}}(\mathsf{pp}_{\mathsf{off}}, \sigma_1(f_{\mathbf{s}}), b, \pi_{\mathsf{on}}) \to \{0,1\}$**:** If π_{on} is valid w.r.t. relation $\widetilde{\mathcal{R}}_{\mathsf{ssq\text{-}on}}$ specified in Section 7.1, return 1; otherwise, return 0.

Efficiency. The efficiency of $\widetilde{\mathcal{SSQ}}$ is summarized in Table 4. Note that the efficiency is mainly dominated by the Lagrange interpolations, basic binary search to determine $\mathbf{u}, \mathbf{v}$ [43] (detailed in Appendix B.6), committing to polynomials and (batch) evaluations, and the invocations to $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$, $\Pi^{\mathbb{K}}_{\mathsf{perm}}$, and $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}} = (\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}, \Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}})$. A more detailed discussion is deferred to Appendix F.

Analysis. The security of $\widetilde{\mathcal{SSQ}}$ follows the below Theorem 5.

Theorem 5. *$\widetilde{\mathcal{SSQ}}$ in Section 7.4 satisfies the binding of "$\leftrightarrows_{\mathsf{pp}}$", completeness, and knowledge soundness defined in Section 4.2 in AGM assuming the completeness and knowledge soundness of $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ and $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$.*

Proof. The completeness and knowledge soundness trivially follows those of $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$ (c.f. Section 7.2) and $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$ (c.f. Section 7.3). The binding of "$\leftrightarrows_{\mathsf{pp}}$" is already discussed in Section 4.1. □

	Running time	Proof size
OffProve	$\widetilde{\mathcal{O}}(K)\ \mathbb{G}_1, \mathbb{F}, \mathcal{O}(K)\ \mathbb{G}_2$	$61\ \mathbb{G}_1, 2\ \mathbb{G}_2, 18\ \mathbb{F}$
OffVerify	$\mathcal{O}(1)\ \mathbb{G}_1, \mathbb{G}_2, \mathcal{O}(\log_2 K)\ \mathbb{F}, 12P$	-
OnProve	$\mathcal{O}(n)\ \mathbb{G}_1, \mathcal{O}(n \log_2 K)\ \mathbb{F}$	$72\ \mathbb{G}_1, 25\ \mathbb{F}$
OnVerify	$\mathcal{O}(1)\ \mathbb{G}_1, \mathbb{G}_2, \mathcal{O}(\log_2 n)\ \mathbb{F}, 12P$	-

Table 4. Efficiency of $\widetilde{\mathcal{SSQ}}$ described in Section 7.4. Here, "$12P$" means 12 pairings.

Changelog

We present the changelog as follows.

2026-Jan-08: We successfully uploaded the initial manuscript to Cryptology ePrint Archive (received on 2026-Jan-04 and approved on 2026-Jan-08).

2026-Jan-10: We revised the manuscript to improve editorial quality, and correct grammatical errors and typos. We also adjusted page layout to improve readability.

2026-Jan-11: We linked the protocols mentioned in the preliminaries to their detailed descriptions in the appendices.

References

1. Albrecht, M.R., Fenzi, G., Lapiha, O., Nguyen, N.K.: SLAP: Succinct Lattice-Based Polynomial Commitments from Standard Assumptions. In: EUROCRYPT 2024 (2024)
2. Angel, S., Ioannidis, E., Margolin, E., Setty, S., Woods, J.: Reef: Fast Succinct Non-Interactive Zero-Knowledge Regex Proofs. In: USENIX Security 2024 (2024)
3. Arun, A., Setty, S., Thaler, J.: Jolt: SNARKs for Virtual Machines via Lookups. In: EUROCRYPT 2024 (2024)
4. Ben-Sasson, E., Bentov, I., Horesh, Y., Riabzev, M.: Scalable Zero Knowledge with No Trusted Setup. In: CRYPTO 2019 (2019)
5. Ben-Sasson, E., Chiesa, A., Riabzev, M., Spooner, N., Virza, M., Ward, N.P.: Aurora: Transparent Succinct Arguments for R1CS. In: EUROCRYPT 2019 (2019)
6. Boneh, D., Boyen, X.: Short signatures without random oracles and the SDH assumption in bilinear groups. J. Cryptol. **21**(2), 149–177 (2008)
7. Bootle, J., Cerulli, A., Groth, J., Jakobsen, S., Maller, M.: Arya: Nearly Linear-Time Zero-Knowledge Proofs for Correct Program Execution. In: ASIACRYPT 2018 (2018)
8. Bowe, S., Grigg, J., Hopwood, D.: Recursive Proof Composition without a Trusted Setup. Cryptology ePrint Archive (2019), `https://ia.cr/2019/1021`
9. Bünz, B., Chiesa, A., Mishra, P., Spooner, N.: Recursive Proof Composition from Accumulation Schemes. In: TCC 2020 (2020)
10. Campanelli, M., Faonio, A., Fiore, D., Li, T., Lipmaa, H.: Lookup Arguments: Improvements, Extensions and Applications to Zero-Knowledge Decision Trees. In: PKC 2024 (2024)
11. Campanelli, M., Fiore, D., Gennaro, R.: Natively Compatible Super-Efficient Lookup Arguments and How to Apply Them. J. Cryptol. **38**(1), 14 (2025)
12. Chen, B., Bünz, B., Boneh, D., Zhang, Z.: HyperPlonk: Plonk with Linear-Time Prover and High-Degree Custom Gates. In: EUROCRYPT 2023 (2023)
13. Chiesa, A., Hu, Y., Maller, M., Mishra, P., Vesely, N., Ward, N.: Marlin: Preprocessing zkSNARKs with Universal and Updatable SRS. In: EUROCRYPT 2020 (2020)
14. Choudhuri, A.R., Garg, S., Goel, A., Sekar, S., Sinha, R.: SublonK: Sublinear Prover PlonK. In: PETS 2024 (2024)
15. Cini, V., Malavolta, G., Nguyen, N.K., Wee, H.: Polynomial Commitments from Lattices: Post-quantum Security, Fast Verification and Transparent Setup. In: CRYPTO 2024 (2024)
16. Crochemore, M., Melichar, B., Tronícek, Z.: Directed acyclic subsequence graph: overview. J. of Discrete Algorithms **1**(3-4), 255–280 (2003)
17. Di, Z., Xia, L., Nguyen, W., Tyagi, N.: MuxProofs: Succinct Arguments for Machine Computation from Vector Lookups. In: ASIACRYPT 2024 (2025)
18. Dutta, M., Ganesh, C., Patranabis, S., Prakash, S., Singh, N.: Batching-Efficient RAM using Updatable Lookup Arguments. In: CCS 2024 (2024)
19. Eagen, L., Fiore, D., Gabizon, A.: cq: Cached quotients for fast lookups. ePrint Archive (2022), `https://ia.cr/2022/1763`

20. Feist, D., Khovratovich, D.: Fast amortized KZG proofs. ePrint Archive (2023), https://ia.cr/2023/033
21. Fenzi, G., Moghaddas, H., Nguyen, N.K.: Lattice-Based Polynomial Commitments: Towards Asymptotic and Concrete Efficiency. J. Cryptol. **37**(3), 31 (2024). https://doi.org/10.1007/S00145-024-09511-8
22. Fiat, A., Shamir, A.: How To Prove Yourself: Practical Solutions to Identification and Signature Problems. In: CRYPTO 1986 (1987)
23. Fleischhacker, N., Hall-Andersen, M., Simkin, M.: Extractable Witness Encryption for KZG Commitments and Efficient Laconic OT. In: ASIACRYPT 2024 (2025)
24. Fuchsbauer, G., Kiltz, E., Loss, J.: The Algebraic Group Model and its Applications. In: CRYPTO 2018 (2018)
25. Gabizon, A., Khovratovich, D.: flookup: Fractional decomposition-based lookups in quasi-linear time independent of table size. ePrint Archive (2022), https://ia.cr/2022/1447
26. Gabizon, A., Williamson, Z.J.: plookup: A simplified polynomial protocol for lookup tables. ePrint Archive (2020), https://ia.cr/2020/315
27. Gabizon, A., Williamson, Z.J., Ciobotaru, O.: PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge. ePrint Archive (2019), https://eprint.iacr.org/2019/953
28. GeeksforGeeks: Subsequence meaning in dsa. online (2023), https://www.geeksforgeeks.org/subsequence-meaning-in-dsa/
29. Gennaro, R., Gentry, C., Parno, B., Raykova, M.: Quadratic Span Programs and Succinct NIZKs without PCPs. In: EUROCRYPT 2013 (2013)
30. Gentry, C., Wichs, D.: Separating succinct non-interactive arguments from all falsifiable assumptions. In: STOC 2011 (2011)
31. Groth, J.: On the Size of Pairing-Based Non-interactive Arguments. In: EUROCRYPT 2016 (2016)
32. Haböck, U.: Multivariate lookups based on logarithmic derivatives. ePrint Archive (2022), https://ia.cr/2022/1530
33. Jain, A., Krenn, S., Pietrzak, K., Tentes, A.: Commitments and Efficient Zero-Knowledge Proofs from Learning Parity with Noise. In: ASIACRYPT 2012 (2012)
34. Kate, A., Zaverucha, G.M., Goldberg, I.: Constant-Size Commitments to Polynomials and Their Applications. In: ASIACRYPT 2010 (2010)
35. Klooß, M., Lai, R.W.F., Nguyen, N.K., Osadnik, M.: RoK, Paper, SISsors Toolkit for Lattice-Based Succinct Arguments. In: ASIACRYPT 2024 (2025)
36. Kothapalli, A., Setty, S., Tzialla, I.: Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. In: CRYPTO 2022 (2022)
37. Ling, S., Tang, K.H., Vu, K., Wang, H., Yan, Y.: Succinct Non-subsequence Arguments. In: SCN 2024 (2024)
38. Luo, N., Weng, C., Singh, J., Tan, G., Piskac, R., Raykova, M.: Privacy-Preserving Regular Expression Matching using Nondeterministic Finite Automata. ePrint Archive (2023), https://ia.cr/2023/643
39. Maller, M., Bowe, S., Kohlweiss, M., Meiklejohn, S.: Sonic: Zero-Knowledge SNARKs from Linear-Size Universal and Updatable Structured Reference Strings. In: CCS 2019 (2019)
40. Posen, J., Kattis, A.A.: Caulk+: Table-independent lookup arguments. ePrint Archive (2022), https://ia.cr/2022/957
41. Raymond, M., Evers, G., Ponti, J., Krishnan, D., Fu, X.: Efficient Zero Knowledge for Regular Language. In: SecureComm 2023 (2025)
42. Schwartz, J.T.: Fast Probabilistic Algorithms for Verification of Polynomial Identities. J. ACM **27**(4), 701–717 (1980)
43. Sedgewick, R., Wayne, K.: Algorithms. Addison-wesley professional (2011)
44. Setty, S.: Spartan: Efficient and General-Purpose zkSNARKs Without Trusted Setup. In: Micciancio, D., Ristenpart, T. (eds.) CRYPTO 2020 (2020)
45. Setty, S., Thaler, J., Wahby, R.: Unlocking the Lookup Singularity with Lasso. In: EUROCRYPT 2024 (2024)
46. Thakur, S.: A flexible Snark via the monomial basis. ePrint Archive (2023), https://ia.cr/2023/1255
47. Thaler, J.: Proofs, Arguments, and Zero-Knowledge. Found. Trends Priv. Secur. **4**(2-4), 117–660 (2022). https://doi.org/10.1561/3300000030
48. Tran, D.H.A., Nguyen, T.N.B., Le-Khac, N.A., Nguyen, T.D.: LogaLookup: Efficient Multivariate Lookup Argument for Accelerated Proof Generation. In: ASIACCS 2025 (2025)
49. Wahby, R.S., Tzialla, I., Shelat, A., Thaler, J., Walfish, M.: Doubly-Efficient zkSNARKs Without Trusted Setup. In: S&P 2018 (2018)
50. Xie, T., Zhang, J., Zhang, Y., Papamanthou, C., Song, D.: Libra: Succinct Zero-Knowledge Proofs with Optimal Prover Computation. In: CRYPTO 2019 (2019)
51. Xie, T., Zhang, Y., Song, D.: Orion: Zero Knowledge Proof with Linear Prover Time. In: CRYPTO 2022 (2022)

52. Zapico, A., Buterin, V., Khovratovich, D., Maller, M., Nitulescu, A., Simkin, M.: Caulk: Lookup Arguments in Sublinear Time. In: CCS 2022 (2022)
53. Zapico, A., Gabizon, A., Khovratovich, D., Maller, M., Ràfols, C.: Baloo: Nearly Optimal Lookup Arguments. ePrint Archive (2022), https://ia.cr/2022/1565
54. Zhang, C., DeStefano, Z., Arun, A., Bonneau, J., Grubbs, P., Walfish, M.: Zombie: Middleboxes that Don't Snoop. In: NSDI 2024 (2024)
55. Zhang, J., Xie, T., Zhang, Y., Song, D.: Transparent polynomial delegation and its applications to zero knowledge proof. In: S&P 2020 (2020)
56. Zhang, Y., Sun, S.F., Gu, D.: Efficient KZG-Based Univariate Sum-Check and Lookup Argument. In: PKC 2024 (2024)
57. Zippel, R.: Probabilistic algorithms for sparse polynomials. In: EUROSAM 1979 (1979)

A Preliminaries (Extended)

A.1 Univariate Sumcheck

Lemma 9 ([5]). *Let $\mathbb{K}$ be a multiplicative subgroup of $\mathbb{F}$. For $f(X) \in \mathbb{F}_{<|\mathbb{K}|}[X]$, we have $\sum_{\alpha\in\mathbb{K}} f(\alpha) = |\mathbb{K}| \cdot f(0)$.*

A.2 Lagrange's Interpolation (Extended)

Proof (Proof of Lemma 1). We have

$$\ell_{\mathbb{K}}^{(i)}(\omega_{\mathbb{K}} \cdot X) = \prod_{j\neq i, j\in[K]} \frac{\omega_{\mathbb{K}} \cdot X - \omega_{\mathbb{K}}^{j-1}}{\omega_{\mathbb{K}}^{i-1} - \omega_{\mathbb{K}}^{j-1}} = \prod_{j\neq i, j\in[K]} \frac{X - \omega_{\mathbb{K}}^{j-2}}{\omega_{\mathbb{K}}^{i-2} - \omega_{\mathbb{K}}^{j-2}}.$$

Since $\omega_{\mathbb{K}}$ is the generator of $\mathbb{K}$, $\omega_{\mathbb{K}}^{-1} = \omega_{\mathbb{K}}^{K-1}$. Therefore, if $i > 1$, $\ell_{\mathbb{K}}^{(i)}(\omega_{\mathbb{K}} \cdot X) = \ell_{\mathbb{K}}^{(i-1)}(X)$; and $\ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}} \cdot X) = \ell_{\mathbb{K}}^{(K)}(X)$. The lemma's statement hence follows. □

A.3 Succinct Non-Interactive Arguments of Knowledge (SNARKs)

A succinct non-interactive argument of knowledge (SNARK) for an NP relation $(x, w) \in \mathcal{R}$ is a non-interactive proof produced by a prover $\mathcal{P}$ and verified by a verifier $\mathcal{V}$. A SNARK should satisfy completeness, knowledge soundness, and succinctness as below. In this paper, we are interested in SNARKs with sublinear verification time.

A SNARK for an NP language $\mathcal{L}$ (w.r.t. some NP relation $\mathcal{R}$) must satisfy the following properties.

- *Completeness.* Given $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$, for any statement $x \in \mathcal{L}$, there exists a witness w s.t.

$$\Pr\left[\mathcal{V}(\mathsf{pp}, x, \pi) = 1 \,\middle|\, \begin{array}{l}\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\\ \mathcal{R}(x, w) = 1,\\ \pi \leftarrow \mathcal{P}(\mathsf{pp}, x, w)\end{array}\right] \geq 1 - \mathsf{negl}(\lambda),$$

  where $\mathsf{negl}(\lambda)$ is a negligible function of λ.
- *Knowledge soundness.* For any PPT adversary $\mathcal{A}$, there exists a PPT extractor $\mathcal{E}$ s.t. for any x, it holds that

$$\Pr\left[\begin{array}{l}\mathcal{V}(\mathsf{pp}, x, \pi^*) = 1\\ \wedge \mathcal{R}(x, w') = 0\end{array} \,\middle|\, \begin{array}{l}\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\\ \pi^\star \leftarrow \mathcal{A}(\mathsf{pp}, x),\\ w' \leftarrow \mathcal{E}(\mathsf{pp}, x, \pi^\star)\end{array}\right] \leq \mathsf{negl}(\lambda),$$

  where $\mathsf{negl}(\lambda)$ is a negligible function of λ.
- *Succinctness.* The communication between $\mathcal{P}$ and $\mathcal{V}$ is sub-linear in the size of $x \in \mathcal{L}$.
- *Sublinear verification time.* The verification time is sublinear in $|x|$.

A.4 Schwartz-Zippel Lemma

We recall the following lemma from [57,42].

Lemma 10 (Schwartz-Zippel Lemma). *Let $f(X)$ be a non-zero polynomial of degree K over $\mathbb{F}$ and S be a subset of $\mathbb{F}$. Then the probability of $f(\rho) = 0$ for $\rho \xleftarrow{\$} S$ is at most $\frac{K}{|S|}$.*

A.5 Commitment Scheme

We recall the definition of a commitment scheme without hiding property (adapted from [33]).

Syntax. A commitment scheme $\mathcal{C} = (\mathsf{C.Setup}, \mathsf{C.Com}, \mathsf{C.Verify})$ is defined as follows:

$\mathsf{C.Setup}(1^\lambda) \to \mathsf{ck}$**:** On input 1^λ, ouput the commitment key ck.
$\mathsf{C.Com}_{\mathsf{ck}}(M) \to c_M$**:** On input the commitment key ck and a message M, output the commitment c_M.
$\mathsf{C.Verify}_{\mathsf{ck}}(M, c_M) \to b$**:** On input the commitment key ck, the message M and the commitment c_M, return a bit $b \in \{0,1\}$ indicating accepted or rejected.

Security. A commitment scheme $\mathcal{C} = (\mathsf{C.Setup}, \mathsf{C.Com}, \mathsf{C.Verify})$ is secure if it satisfies completeness and (computationally) binding, defined as follows.

– *Completeness.* For any valid message M,

$$\Pr\left[\begin{array}{r}\mathsf{ck} \leftarrow \mathsf{C.Setup}(1^\lambda); c_M \leftarrow \mathsf{C.Com}_{\mathsf{ck}}(M);\\ \mathsf{C.Verify}_{\mathsf{ck}}(\mathsf{ck}, M, c_M) = 1\end{array}\right] \geq 1 - \mathsf{negl}(\lambda).$$

– *(Computationally) Binding.* For any (PPT) adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{r}\mathsf{ck} \leftarrow \mathsf{C.Setup}(1^\lambda), (c_M, M, M') \leftarrow \mathcal{A}(\mathsf{ck});\\ b_0 \leftarrow \mathsf{C.Verify}_{\mathsf{ck}}(M, c_M); b_1 \leftarrow \mathsf{C.Verify}_{\mathsf{ck}}(M', c_M);\\ b_0 = b_1 = 1 \wedge M \neq M'.\end{array}\right] \leq \mathsf{negl}(\lambda).$$

A.6 Polynomial Commitment Scheme

A polynomial commitment scheme (PCS, [34]) is a tuple of algorithms:

$$\mathsf{PCS} = (\mathsf{Setup}, \mathsf{Com}, \mathsf{Open}, \mathsf{Eval}).$$

$\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$**:** On input 1^λ, ouput the public parameter pp.
$\mathsf{Com}(\mathsf{pp}, f(X)) \to (\sigma(f), \mathsf{aux})$**:** On input pp and $f(X)$, output the polynomial commitment $\sigma(f)$ and an auxiliary information aux.
$\mathsf{Open}(\mathsf{pp}, \sigma(f), f(X), \mathsf{aux}) \to b$**:** On input pp, polynomial commitment $\sigma(f)$, polynomial $f(X)$, and an auxiliary input aux, return a bit $b \in \{0,1\}$ indicating accepted or rejected.
$\mathsf{Eval}\,\langle \mathcal{P}(f(X), \mathsf{aux}), \mathcal{V}\rangle\,(\mathsf{pp}, \sigma(f), \psi, y) \to b$**:** This is an interactive protocol between a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ with common inputs pp, $\sigma(f)$, $\psi \in \mathbb{F}$, and $y \in \mathbb{F}$. $\mathcal{P}$ additionally holds $f(X)$ and aux. After the execution, $\mathcal{V}$ outputs a bit $b \in \{0,1\}$ indicating whether $\mathcal{V}$ accepts $f(\psi) = y$.

An extractable PCS $\mathsf{PCS} = (\mathsf{Setup}, \mathsf{Com}, \mathsf{Open}, \mathsf{Eval})$ is secure if it satisfies completeness, binding, and knowledge soundness as follows.

– *Completeness.* For any polynomial $f(X)$,

$$\Pr\left[\begin{array}{r}\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda);\\ (\sigma(f), \mathsf{aux}) \leftarrow \mathsf{Com}(\mathsf{pp}, f(X));\\ \mathsf{Eval}\,\langle \mathcal{P}(f(X), \mathsf{aux}), \mathcal{V}(\mathbf{r})\rangle\,(\mathsf{pp}, \sigma(f), \psi, y) = 1;\\ y = f(\psi).\end{array}\right] \geq 1 - \mathsf{negl}(\lambda).$$

– *Binding.* For any PPT adversary $\mathcal{A}$,

$$\Pr\left[\begin{array}{r}\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda),\\ (\sigma, f_0(X), f_1(X), \mathsf{aux}_0, \mathsf{aux}_1) \leftarrow \mathcal{A}(\mathsf{pp});\\ b_0 \leftarrow \mathsf{Open}(\mathsf{pp}, \sigma, f_0(X), \mathsf{aux}_0);\\ b_1 \leftarrow \mathsf{Open}(\mathsf{pp}, \sigma, f_1(X), \mathsf{aux}_1);\\ b_0 = b_1 = 1 \wedge f_0(X) \neq f_1(X).\end{array}\right] \leq \mathsf{negl}(\lambda).$$

– *Knowledge Soundness.* Given $\mathsf{pp} \leftarrow \mathsf{Setup}(\lambda)$, Eval is a succinct argument of knowledge for NP relation:

$$\mathcal{R}_{\mathsf{Eval}} = \left\{ (\sigma(f), \psi, y; f(X), \mathsf{aux}) \,\middle|\, \begin{array}{l} f(X) \wedge f(\psi) = y \\ \wedge \mathsf{Open}(\mathsf{pp}, \sigma(f), f(X), \mathsf{aux}) = 1 \end{array} \right\}.$$

A.7 Detailed Protocols for Lookups, Permutations, and Cached Quotients

The protocols for lookups, permutations, and cached quotients as based on the following lemmas from Haböck[32].

Lemma 11. *$\{s_i\}_{i=1}^H \subseteq \{t_i\}_{i=1}^K$, for some $H, K \in \mathbb{N}$, iff there exist $(m_i)_{i=1}^K$ s.t. $\sum_{i=1}^H \frac{1}{s_i+Y} = \sum_{i=1}^K \frac{m_i}{t_i+Y}$.*

Lemma 12. *$(s_i)_{i=1}^K \bowtie (t_i)_{i=1}^K$, for some $K \in \mathbb{N}$, iff $\sum_{i=1}^K \frac{1}{s_i+Y} = \sum_{i=1}^K \frac{1}{t_i+Y}$.*

We present protocols $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ and $\Pi^{\mathbb{K}}_{\mathsf{perm}}$ in Figures 3 and 4, respectively.

For protocol $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}} = (\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}, \Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}})$, the preprocessing $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$ is presented in Figure 5 while proving phase $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$ is presented in Figure 6.

B Technical Overview and Modeling for Constructing Subsequence Scheme (Extended)

B.1 Proof of Lemma 4

Proof (Proof of Lemma 4). For $\mathsf{id}_0 = 0$, it can be simply explained as an empty string is a subsequence of itself, i.e., $\mathbf{s}[1 \dots 0] = \epsilon \lhd \epsilon = \mathbf{t}[1 \dots 0]$, and 0 is the smallest index satisfying the definition of id_0.

For some $i \in [n]$, assume that $\mathsf{id}_0, \dots, \mathsf{id}_{i-1}$ are correctly determined. We now show that $\mathsf{id}_i = \mathsf{dfa}(\mathsf{id}_{i-1}, s_i)$.

When $\mathsf{id}_{i-1} = N+1$, we know that $\mathbf{s}[1 \dots i-1] \not\lhd \mathbf{t}$ and $\mathsf{id}_i = N+1$. Clearly, $\mathsf{dfa}(\mathsf{id}_{i-1}, s_i) = \mathsf{dfa}(N+1, s_i) = N+1 = \mathsf{id}_i$.

When $\mathsf{id}_{i-1} \leq N$, it follows from Lemma 3 that $\mathsf{id}_i > \mathsf{id}_{i-1}$. Recall that id_{i-1} is the smallest index s.t. $\mathbf{s}[1 \dots i-1] \lhd \mathbf{t}[1 \dots \mathsf{id}_{i-1}]$. As $\mathsf{dfa}(\mathsf{id}_{i-1}, s_i)$ is the smallest index (say, v) s.t. $v > \mathsf{id}_{i-1}$ and either $(v \in [N] \wedge t_v = s_i)$ or $v = N+1$, we have the following two subcases.

– If $v \leq N$, then $t_v = s_i$. It must hold that $\mathbf{s}[1 \dots i] \lhd \mathbf{t}[1 \dots v]$. Hence, $\mathbf{s}[1 \dots i] \lhd \mathbf{t}$ and id_i is the smallest index satisfying $\mathbf{s}[1 \dots i] \lhd \mathbf{t}[1 \cdots \mathsf{id}_i]$. It follows that $\mathsf{id}_i \leq v$. On the other hand, clearly $\mathsf{id}_i > \mathsf{id}_{i-1}$ and $t_{\mathsf{id}_i} = s_i$. As v is the smallest index s.t. $v \geq \mathsf{id}_{i-1} + 1 \wedge t_v = s_i$, it implies that $\mathsf{id}_i \geq v$. Consequently, $\mathsf{id}_i = v = \mathsf{dfa}(\mathsf{id}_{i-1}, s_i)$.
– If $v = \mathsf{dfa}(\mathsf{id}_{i-1}, s_i) = N+1$, then $\mathbf{t}[\mathsf{id}_{i-1}+1 \dots N]$ does not contain s_i. It follows that $\mathbf{s}[1 \dots i] \not\lhd \mathbf{t}$, and $\mathsf{id}_j = N+1 = \mathsf{dfa}(\mathsf{id}_{i-1}, s_i)$.

We thus conclude the proof. □

Protocol $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$.
Common Inputs:

- *Public Parameters:* $K \in \mathbb{N}$, $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_1\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_2\}_{i=1}^{K+1}$, $z_{\mathbb{K}}(X) = X^K - 1$, and $\sigma_2(z_{\mathbb{K}}) = [\![z_{\mathbb{K}}(x)]\!]_2$.
- *Commitments to $\mathcal{P}$'s Inputs:* $\sigma_2(f_{\mathbf{t}}) = [\![f_{\mathbf{t}}(x)]\!]_2$, and $\sigma_1(f_{\mathbf{s}}) = [\![f_{\mathbf{s}}(x)]\!]_1$.

$\mathcal{P}$'s Inputs: Polynomials $f_{\mathbf{t}}(X)$ and $f_{\mathbf{s}}(X)$.
Execution:

1. $\mathcal{P}$ computes $f_{\mathbf{m}}(X)$ encoding $(m_i)_{i=1}^{K} \in \mathbb{F}^K$ s.t. $\begin{cases} \sum_{i=1}^{K} \frac{m_i}{t_i+Y} = \sum_{i=1}^{K} \frac{1}{s_i+Y} \text{ as in Lemma 11,} \\ f_{\mathbf{m}}(X) \in \mathbb{F}_{<K}[X] \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto m_i \ \forall i \in [K]. \end{cases}$
2. $\mathcal{P}$ computes and sends to $\mathcal{V}$ the commitment $\sigma_1(f_{\mathbf{m}}) = [\![f_{\mathbf{m}}(x)]\!]_1$.
3. $\mathcal{V}$ samples $\beta \xleftarrow{\$} \mathbb{F}$ and sends β to $\mathcal{P}$.
4. $\mathcal{P}$ computes, via Lagrange's interpolation (c.f. Section 2.1),

$$
\begin{aligned}
&f_{\mathbf{a}}(X) \in \mathbb{F}_{<K}[X] \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto \frac{m_i}{t_i+\beta} \quad \forall i \in [K], \\
&f_{\mathbf{b}}(X) \in \mathbb{F}_{<K}[X] \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto \frac{1}{s_i+\beta} \quad \forall i \in [K], \\
&f_{\overline{\mathbf{a}}}(X) = \frac{f_{\mathbf{a}}(X) - f_{\mathbf{a}}(0)}{X} \in \mathbb{F}_{<K-1}[X], \\
&f_{\overline{\mathbf{b}}}(X) = \frac{f_{\mathbf{b}}(X) - f_{\mathbf{b}}(0)}{X} \in \mathbb{F}_{<K-1}[X], \\
&f_{\overline{b}\text{-shifted}}(X) = f_{\overline{\mathbf{b}}}(X) \cdot X \in \mathbb{F}_{<K}[X].
\end{aligned}
$$

5. $\mathcal{P}$ sends $T := K \cdot f_{\mathbf{a}}(0)$, $\sigma_1(f_{\mathbf{a}}) := [\![f_{\mathbf{a}}(x)]\!]_1$, $\sigma_1(f_{\mathbf{b}}) := [\![f_{\mathbf{b}}(x)]\!]_1$, $\sigma_1(f_{\overline{\mathbf{a}}}) := [\![f_{\overline{\mathbf{a}}}(x)]\!]_1$, $\sigma_1(f_{\overline{\mathbf{b}}}) := [\![f_{\overline{\mathbf{b}}}(x)]\!]_1$, and $\sigma_1(f_{\overline{b}\text{-shifted}}) := [\![f_{\overline{b}\text{-shifted}}(x)]\!]_1$ to $\mathcal{V}$.
6. $\mathcal{P}$ computes $q_{\mathbf{a}}(X)$ and $q_{\mathbf{b}}(X)$ satisfying $f_{\mathbf{a}}(X) \cdot (f_{\mathbf{t}}(X) + \beta) - f_{\mathbf{m}}(X) = q_{\mathbf{a}}(X) \cdot z_{\mathbb{K}}(X)$ and $f_{\mathbf{b}}(X) \cdot (f_{\mathbf{s}}(X) + \beta) - 1 = q_{\mathbf{b}}(X) \cdot z_{\mathbb{K}}(X)$, and sends $\sigma_1(q_{\mathbf{a}}) := [\![q_{\mathbf{a}}(x)]\!]_1$ and $\sigma_1(q_{\mathbf{b}}) := [\![q_{\mathbf{b}}(x)]\!]_1$ to $\mathcal{V}$.
7. $\mathcal{V}$ samples $\gamma \xleftarrow{\$} \mathbb{F}$ and sends γ to $\mathcal{P}$.
8. $\mathcal{P}$ sends $v_{\overline{b}}^{(\gamma)} := f_{\overline{\mathbf{b}}}(\gamma)$, and $v_s^{(\gamma)} := f_{\mathbf{s}}(\gamma)$ to $\mathcal{V}$.
9. $\mathcal{V}$ computes $z_{\mathbb{K}}(\gamma) := \gamma^K - 1$, $v_b^{(\gamma)} := v_{\overline{b}}^{(\gamma)} \cdot \gamma + T/K$, and $v_{q_b}^{(\gamma)} := \frac{v_b^{(\gamma)} \cdot (v_s^{(\gamma)} + \beta) - 1}{z_{\mathbb{K}}(\gamma)}$.
10. $\mathcal{V}$ samples $\eta \xleftarrow{\$} \mathbb{F}$ and sends η to $\mathcal{P}$.
11. Both $\mathcal{P}$ and $\mathcal{V}$ determine $v := v_{\overline{b}}^{(\gamma)} + \eta \cdot v_s^{(\gamma)} + \eta^2 \cdot v_{q_b}^{(\gamma)}$.
12. $\mathcal{P}$ sends $\pi := \left[\!\!\left[\frac{f_{\overline{\mathbf{b}}}(x) + \eta \cdot f_{\mathbf{s}}(x) + \eta^2 \cdot q_{\mathbf{b}}(x) - v}{x - \gamma} \right]\!\!\right]_1$ to $\mathcal{V}$.
13. $\mathcal{V}$ computes $\mathsf{c} := \sigma_1(f_{\overline{\mathbf{b}}}) + \eta \cdot \sigma_1(f_{\mathbf{s}}) + \eta^2 \cdot \sigma_1(q_{\mathbf{b}})$ and checks whether the followings hold.

$$
\begin{aligned}
&\langle \sigma_1(f_{\mathbf{a}}), \sigma_2(f_{\mathbf{t}}) \rangle = \langle \sigma_1(q_{\mathbf{a}}), \sigma_2(z_{\mathbb{K}}) \rangle \cdot \langle \sigma_1(f_{\mathbf{m}}) - \beta \cdot \sigma_1(f_{\mathbf{a}}), [\![1]\!]_2 \rangle, \\
&\langle \sigma_1(f_{\overline{\mathbf{b}}}), [\![x]\!]_2 \rangle = \langle \sigma_1(f_{\overline{b}\text{-shifted}}), [\![1]\!]_2 \rangle, \\
&\langle \mathsf{c} - [\![v]\!]_1 + \gamma \cdot \pi, [\![1]\!]_2 \rangle = \langle \pi, [\![x]\!]_2 \rangle, \\
&\langle \sigma_1(f_{\mathbf{a}}) - [\![T/K]\!]_1, [\![1]\!]_2 \rangle = \langle \sigma_1(f_{\overline{\mathbf{a}}}), [\![x]\!]_2 \rangle.
\end{aligned}
$$

Fig. 3. Protocol $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ for lookup arguments from [32].

B.2 Binary Search Algorithm to Search for id_i based on dfa_c^{-1}

We follow the context discussed in Section 3.2 to present the binary search algorithm, in Figure 7, that looks for id_i given $s_i = c$ and id_{i-1}. We note that the binary search algorithm is a basic algorithm that can be found in any fundamental textbooks about algorithms (e.g., [43]).

We note that this algorithm will definitely halt after $\mathcal{O}(\log_2 m_c)$ times after repeating the loop to search for id_i. In fact, as discussed in Section 3.2, the consecutive intervals $\{\mathsf{dfa}_c^{-1}(j_i^{(c)})\}_{i=1}^{m_c+1}$ form a partition of $[N+1]$ according to Lemma 5. Hence, id_{i-1} must be contained in some interval among $\{\mathsf{dfa}_c^{-1}(j_i^{(c)})\}_{i=1}^{m_c+1}$.

Protocol $\Pi^{\mathbb{K}}_{\mathsf{perm}}$.
Common Inputs:

- *Public Parameters:* $K \in \mathbb{N}$, $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_1\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_2\}_{i=1}^{K+1}$, $z_{\mathbb{K}}(X) = X^K - 1$, and $\sigma_2(z_{\mathbb{K}}) = [\![z_{\mathbb{K}}(x)]\!]_2$.
- *Commitments to $\mathcal{P}$'s Inputs:* $\sigma_2(f_{\mathbf{t}}) = [\![f_{\mathbf{t}}(x)]\!]_2$ and $\sigma_1(f_{\mathbf{s}}) = [\![f_{\mathbf{s}}(x)]\!]_1$.

$\mathcal{P}$'s Inputs: Polynomials $f_{\mathbf{t}}(X)$ and $f_{\mathbf{s}}(X)$.
Execution:

1. $\mathcal{V}$ samples $\beta \xleftarrow{\$} \mathbb{F}$ and sends β to $\mathcal{P}$.
2. $\mathcal{P}$ computes polynomials $f_{\mathbf{a}}(X)$, $f_{\mathbf{b}}(X)$, $f_{\overline{\mathbf{a}}}(X)$, $f_{\overline{\mathbf{b}}}(X)$, and $f_{\overline{b}\text{-shifted}}(X)$ satisfying

$$
\begin{aligned}
&f_{\mathbf{a}}(X) \in \mathbb{F}_{<K}[X] \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto \frac{1}{t_i + \beta} \quad \forall i \in [K],\\
&f_{\mathbf{b}}(X) \in \mathbb{F}_{<K}[X] \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto \frac{1}{s_i + \beta} \quad \forall i \in [K],\\
&f_{\overline{\mathbf{a}}}(X) = \frac{f_{\mathbf{a}}(X) - f_{\mathbf{a}}(0)}{X} \in \mathbb{F}_{<K-1}[X],\\
&f_{\overline{\mathbf{b}}}(X) = \frac{f_{\mathbf{b}}(X) - f_{\mathbf{b}}(0)}{X} \in \mathbb{F}_{<K-1}[X],\\
&f_{\overline{b}\text{-shifted}}(X) = f_{\overline{\mathbf{b}}}(X) \cdot X \in \mathbb{F}_{<K}[X].
\end{aligned}
$$

3. $\mathcal{P}$ sends $T = K \cdot f_{\mathbf{a}}(0)$, $\sigma_1(f_{\mathbf{a}}) = [\![f_{\mathbf{a}}(x)]\!]_1$, $\sigma_1(f_{\mathbf{b}}) = [\![f_{\mathbf{b}}(x)]\!]_1$, $\sigma_1(f_{\overline{\mathbf{a}}}) = [\![f_{\overline{\mathbf{a}}}(x)]\!]_1$, $\sigma_1(f_{\overline{\mathbf{b}}}) = [\![f_{\overline{\mathbf{b}}}(x)]\!]_1$, and $\sigma_1(f_{\overline{b}\text{-shifted}}) = [\![f_{\overline{b}\text{-shifted}}(x)]\!]_1$ to $\mathcal{V}$.
4. $\mathcal{P}$ computes $q_{\mathbf{a}}(X)$ and $q_{\mathbf{b}}(X)$ satisfying $f_{\mathbf{a}}(X) \cdot (f_{\mathbf{t}}(X) + \beta) - 1 = q_{\mathbf{a}}(X) \cdot z_{\mathbb{K}}(X)$ and $f_{\mathbf{b}}(X) \cdot (f_{\mathbf{s}}(X) + \beta) - 1 = q_{\mathbf{b}}(X) \cdot z_{\mathbb{K}}(X)$, and sends commitments $\sigma_1(q_{\mathbf{a}}) = [\![q_{\mathbf{a}}(x)]\!]_1$ and $\sigma_1(q_{\mathbf{b}}) = [\![q_{\mathbf{b}}(x)]\!]_1$ to $\mathcal{V}$.
5. $\mathcal{V}$ samples $\gamma \xleftarrow{\$} \mathbb{F}$ and sends γ to $\mathcal{P}$.
6. $\mathcal{P}$ sends $v_{\overline{b}}^{(\gamma)} := f_{\overline{\mathbf{b}}}(\gamma)$, and $v_s^{(\gamma)} := f_{\mathbf{s}}(\gamma)$ to $\mathcal{V}$.
7. $\mathcal{V}$ computes $z_{\mathbb{K}}(\gamma) := \gamma^K - 1$, $v_b^{(\gamma)} := v_{\overline{b}}^{(\gamma)} \cdot \gamma + T/K$, and $v_{q_b}^{(\gamma)} := \frac{v_b^{(\gamma)} \cdot (v_s^{(\gamma)} + \beta) - 1}{z_{\mathbb{K}}(\gamma)}$.
8. $\mathcal{V}$ samples $\eta \xleftarrow{\$} \mathbb{F}$ and sends η to $\mathcal{P}$.
9. Both $\mathcal{P}$ and $\mathcal{V}$ can locally determine $v := v_{\overline{b}}^{(\gamma)} + \eta \cdot v_s^{(\gamma)} + \eta^2 \cdot v_{q_b}^{(\gamma)}$.
10. $\mathcal{P}$ computes $\pi := \left[\!\!\left[\frac{f_{\overline{\mathbf{b}}}(x) + \eta \cdot f_{\mathbf{s}}(x) + \eta^2 \cdot q_{\mathbf{b}}(x) - v}{x - \gamma} \right]\!\!\right]_1$ and sends π to $\mathcal{V}$.
11. $\mathcal{V}$ computes $\mathsf{c} := \sigma_1(f_{\overline{\mathbf{b}}}) + \eta \cdot \sigma_1(f_{\mathbf{s}}) + \eta^2 \cdot \sigma_1(q_{\mathbf{b}})$.
12. $\mathcal{V}$ checks whether all of the followings hold.

$$
\begin{aligned}
&\langle \sigma_1(f_{\mathbf{a}}), \sigma_2(f_{\mathbf{t}}) \rangle = \langle \sigma_1(q_{\mathbf{a}}), \sigma_2(z_{\mathbb{K}}) \rangle \cdot \left\langle [\![1]\!]_1 - \beta \cdot \sigma_1(f_{\mathbf{a}}), [\![1]\!]_2 \right\rangle,\\
&\left\langle \sigma_1(f_{\overline{\mathbf{b}}}), [\![x]\!]_2 \right\rangle = \left\langle \sigma_1(f_{\overline{b}\text{-shifted}}), [\![1]\!]_2 \right\rangle,\\
&\left\langle \mathsf{c} - [\![v]\!]_1 + \gamma \cdot \pi, [\![1]\!]_2 \right\rangle = \left\langle \pi, [\![x]\!]_2 \right\rangle,\\
&\left\langle \sigma_1(f_{\mathbf{a}}) - [\![T/K]\!]_1, [\![1]\!]_2 \right\rangle = \left\langle \sigma_1(f_{\overline{\mathbf{a}}}), [\![x]\!]_2 \right\rangle.
\end{aligned}
$$

Fig. 4. Protocol $\Pi^{\mathbb{K}}_{\mathsf{perm}}$ for permutation arguments from [32].

B.3 Proof of Lemma 6

Proof (Proof of Lemma 6). The forward direction directly follows the discussion in Section 3.2. Therefore, in this proof, for readability purposes, we simply recall the discussion to prove this lemma. The reader can directly jump to the proof of the reverse direction.

"⇒": By definition of $\mathcal{S}_{\mathsf{intv}}$ (c.f. (8)), each triple $(c, j, j') \in \mathcal{S}_{\mathsf{intv}}$ is parsed as the following two sub-forms.

- If $j' < N$, then $(c, j, j') = (c, j_{i-1}^{(c)}, j_i^{(c)} - 1)$ for some $i \in [m_c]$. Here, we know that $\mathsf{dfa}_c^{-1}(j_i^{(c)}) = [j_{i-1}^{(c)}, j_i^{(c)} - 1]$. We can recognize this case via checking whether $j' < N$ since $j' = j_i^{(c)} - 1$ and

Protocol $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$ in $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}} = (\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}, \Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}})$.
Common Inputs:

- *Public parameters:* $H, K \in \mathbb{N}$, $\mathbb{H} = \{\omega_{\mathbb{H}}^{i-1}\}_{i=1}^{H}$, $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_1\}_{i=1}^{K}$, $\{[\![x^{i-1}]\!]_2\}_{i=1}^{K+1}$, $z_{\mathbb{H}}(X) = X^H - 1$, $z_{\mathbb{K}}(X) = X^K - 1$, $\{\ell_{\mathbb{K}}^{(i)}(X)\}_{i=1}^{K}$, $\sigma_2(z_{\mathbb{K}}) = [\![z_{\mathbb{K}}(x)]\!]_2$, $\sigma_1(\ell_{\mathbb{K}}^{(i)}) = \left[\!\!\left[\ell_{\mathbb{K}}^{(i)}(x)\right]\!\!\right]_1 \ \forall i \in [K]$, and $\sigma_1(\overline{\ell}_{\mathbb{K}}^{(i)}) = \left[\!\!\left[\left(\ell_{\mathbb{K}}^{(i)}(x) - \ell_{\mathbb{K}}^{(i)}(0)\right)/x\right]\!\!\right]_1 \ \forall i \in [K]$ where $\{\ell_{\mathbb{K}}^{(i)}(X)\}_{i=1}^{K}$ is the Lagrange basis.
- *Commitments to $\mathcal{P}$'s Inputs:* $\sigma_2(f_{\mathbf{t}}) = [\![f_{\mathbf{t}}(x)]\!]_2$.

$\mathcal{P}$'s Input: Polynomial $f_{\mathbf{t}}(X)$.
Execution:
Preprocessing phase (when $f_{\mathbf{s}}(X)$ is not known), denoted by $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$:

1. For $i \in [K]$, $\mathcal{P}$ computes $q_i(X) \in \mathbb{F}_{<K}[X]$, satisfying $\ell_{\mathbb{K}}^{(i)}(X) \cdot f_{\mathbf{t}}(X) = t_i \cdot \ell_{\mathbb{K}}^{(i)}(X) + z_{\mathbb{K}}(X) \cdot q_i(X)$, and then $\sigma_1(q_i) = [\![q_i(x)]\!]_1$.
2. Return the preprocessing tuple $\mathsf{prep}(f_{\mathbf{t}}) = (q_i(X), \sigma_1(q_i))_{i=1}^{K}$.

Fig. 5. Protocol $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$ in $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}} = (\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}, \Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}})$.

$j_i^{(c)} \le N$ for $i \in [m_c]$. Hence, one can directly understand that $\mathsf{dfa}_c^{-1}(j'+1) = \mathsf{dfa}_c^{-1}(j_i^{(c)}) = [j_{i-1}^{(c)}, j_i^{(c)} - 1] = [j, j']$.
- If $j' = N+1$, then $(c, j, j') = (c, j_{m_c}^{(c)}, N+1)$. We know that $\mathsf{dfa}_c^{-1}(N+1) = [j_{m_c}^{(c)}, N+1]$. This sub-case is recognized via checking whether $j' = N+1$. Hence, we understand that $\mathsf{dfa}_c^{-1}(j') = \mathsf{dfa}_c^{-1}(N+1) = [j_{m_c}^{(c)}, N+1] = [j, j']$.

"$\Leftarrow$": According to the RHS of (9), we have the following two subcases for $(c, j, j') \in \Sigma \times [0, N+1] \times [0, N+1]$:

- *The case $j' < N \wedge \mathsf{dfa}_c^{-1}(j'+1) = [j, j']$:* By definition of dfa_c^{-1} in Section 3.2, we know that $\mathsf{dfa}_c^{-1}(j'+1) = [j_{i-1}^{(c)}, j_i^{(c)} - 1]$ for some $i \in [m_c]$. Hence, it is obvious that $(c, j, j') \in \mathcal{S}_{\mathsf{intv}}$ as $\mathcal{S}_{\mathsf{intv}}$ contains $(c, j_{i-1}^{(c)}, j_i^{(c)} - 1)$ (c.f. (8)).
- *The case $j' = N+1 \wedge \mathsf{dfa}_c^{-1}(N+1) = [j, j']$:* By definition, $\mathsf{dfa}_c^{-1}(N+1) = [j_{m_c}^{(c)}, N+1]$. Hence, $(c, j, j') = (c, j_{m_c}^{(c)}, N+1)$ which is contained in $\mathcal{S}_{\mathsf{intv}}$.

We thus conclude the proof. □

B.4 Proof of Lemma 7

Proof (Proof of Lemma 7). "$\Rightarrow$": the proof follows the discussion in Section 5.1, leading to Lemma 7. Therefore, we only prove the reverse direction here.
"$\Leftarrow$": Assume that (22) holds. We now prove that ch, lft, and rgt are well-formed according to (18). The second line of (22), i.e.,

$$\{(\mathsf{ch}_{i+1} - \mathsf{ch}_i, \mathsf{lft}_{i+1} - \mathsf{rgt}_i)\}_{i=1}^{K-1} \subseteq \{(0,1), (1, -(N+1))\}$$

says that, for each $i \in [K-1]$, one of the following two cases holds.

(i) $(\mathsf{ch}_{i+1} - \mathsf{ch}_i, \mathsf{lft}_{i+1} - \mathsf{rgt}_i) = (0,1)$. This implies that $\mathsf{ch}_{i+1} = \mathsf{ch}_i$ and $\mathsf{rgt}_i + 1 = \mathsf{lft}_{i+1}$.
(ii) $(\mathsf{ch}_{i+1} - \mathsf{ch}_i, \mathsf{lft}_{i+1} - \mathsf{rgt}_i) = (1, -(N+1))$. This implies that $\mathsf{ch}_i + 1 = \mathsf{ch}_{i+1}$ and $\mathsf{lft}_{i+1} = \mathsf{rgt}_i - (N+1)$. From the third line of (22), namely, $\{\mathsf{lft}_i\}_{i=1}^{K} \subseteq [0, N+1]$ and $\{\mathsf{rgt}_i\}_{i=1}^{K} \subseteq [0, N+1]$, we know that $\mathsf{lft}_{i+1} \in [0, N+1]$ implying $\mathsf{rgt}_i - (N+1) \in [0, N+1]$. This only holds when $\mathsf{rgt}_i = N+1$, since, otherwise, $\mathsf{rgt}_i - (N+1) < 0$ falling outside of $[0, N+1]$. Hence, in this case, we deduce that $\mathsf{rgt}_i = N+1$ and $\mathsf{lft}_{i+1} = 0$.

Protocol $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$ in $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}} = (\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}, \Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}})$.
Common Inputs: Same as those in Figure 5.
Execution:
Proving phase (when $f_{\mathbf{s}}(X)$ is known), denoted by $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$:

- *Commitment to $\mathcal{P}$'s Input (Common Input):* $\sigma_1(f_{\mathbf{s}}) = [\![f_{\mathbf{s}}(x)]\!]_1$.
- *$\mathcal{P}$'s Inputs:* $f_{\mathbf{s}}(X)$ and $\mathsf{prep}(f_{\mathbf{t}}) = (q_i(X), \sigma_1(q_i))_{i=1}^K$.

This phase works as folows.

1. $\mathcal{P}$ computes $\sigma_1(f_{\mathbf{s}}) = [\![f_{\mathbf{s}}(x)]\!]_1$.
2. $\mathcal{P}$ computes polynomial $f_{\mathbf{m}}(X)$ encoding $(m_i)_{i=1}^K \in \mathbb{F}^K$ s.t.
$\begin{cases} \sum_{i=1}^K \frac{m_i}{t_i+Y} = \sum_{i=1}^H \frac{1}{s_i+Y} \text{ as in Lemma 11,} \\ f_{\mathbf{m}}(X) \in \mathbb{F}_{<K}[X] \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto m_i \ \forall i \in [K]. \end{cases}$
3. $\mathcal{P}$ computes and sends to $\mathcal{V}$ the commitment $\sigma_1(f_{\mathbf{m}}) = [\![f_{\mathbf{m}}(x)]\!]_1$.
4. $\mathcal{V}$ samples $\beta \xleftarrow{\$} \mathbb{F}$ and sends β to $\mathcal{P}$.
5. $\mathcal{P}$ computes polynomials $f_{\mathbf{a}}(X)$, $f_{\mathbf{b}}(X)$, $f_{\overline{\mathbf{a}}}(X)$, $f_{\overline{\mathbf{b}}}(X)$, and $f_{\overline{b}\text{-}\mathsf{shifted}}(X)$ satisfying
$$f_{\mathbf{a}}(X) \in \mathbb{F}_{<K}[X] \text{ s.t. } \omega_{\mathbb{K}}^{i-1} \mapsto \frac{m_i}{t_i+\beta} \quad \forall i \in [K],$$
$$f_{\mathbf{b}}(X) \in \mathbb{F}_{<H}[X] \text{ s.t. } \omega_{\mathbb{H}}^{i-1} \mapsto \frac{1}{s_i+\beta} \quad \forall i \in [H],$$
$$f_{\overline{\mathbf{a}}}(X) = \frac{f_{\mathbf{a}}(X) - f_{\mathbf{a}}(0)}{X} \in \mathbb{F}_{<K-1}[X],$$
$$f_{\overline{\mathbf{b}}}(X) = \frac{f_{\mathbf{b}}(X) - f_{\mathbf{b}}(0)}{X} \in \mathbb{F}_{<H-1}[X],$$
$$f_{\overline{b}\text{-}\mathsf{shifted}}(X) = f_{\overline{\mathbf{b}}}(X) \cdot X^{K-1-(H-2)} \in \mathbb{F}_{<K}[X].$$
6. $\mathcal{P}$ sends $T := K \cdot f_{\mathbf{a}}(0)$, $\sigma_1(f_{\mathbf{a}}) := [\![f_{\mathbf{a}}(x)]\!]_1$, $\sigma_1(f_{\mathbf{b}}) := [\![f_{\mathbf{b}}(x)]\!]_1$, $\sigma_1(f_{\overline{\mathbf{a}}}) := [\![f_{\overline{\mathbf{a}}}(x)]\!]_1$, $\sigma_1(f_{\overline{\mathbf{b}}}) := [\![f_{\overline{\mathbf{b}}}(x)]\!]_1$, and $\sigma_1(f_{\overline{b}\text{-}\mathsf{shifted}}) := [\![f_{\overline{b}\text{-}\mathsf{shifted}}(x)]\!]_1$ to $\mathcal{V}$.
7. Let $q_{\mathbf{a}}(X)$ and $q_{\mathbf{b}}(X)$ satisfy $f_{\mathbf{a}}(X) \cdot (f_{\mathbf{t}}(X) + \beta) - f_{\mathbf{m}}(X) = q_{\mathbf{a}}(X) \cdot z_{\mathbb{K}}(X)$ and $f_{\mathbf{b}}(X) \cdot (f_{\mathbf{s}}(X) + \beta) - 1 = q_{\mathbf{b}}(X) \cdot z_{\mathbb{H}}(X)$. $\mathcal{P}$ computes and sends $\sigma_1(q_{\mathbf{a}}) = [\![q_{\mathbf{a}}(x)]\!]_1$ and $\sigma_1(q_{\mathbf{b}}) = [\![q_{\mathbf{b}}(x)]\!]_1$ to $\mathcal{V}$. Here, as explained in Section 2.4, commitment $\sigma_1(q_{\mathbf{a}})$ is computed from H out of the K elements of $\{\sigma_1(q_i)\}_{i=1}^K$ in $\mathsf{prep}(f_{\mathbf{t}})$.
8. $\mathcal{V}$ samples $\gamma \xleftarrow{\$} \mathbb{F}$ and sends γ to $\mathcal{P}$.
9. $\mathcal{P}$ sends $v_{\overline{b}}^{(\gamma)} := f_{\overline{\mathbf{b}}}(\gamma)$, and $v_s^{(\gamma)} := f_{\mathbf{s}}(\gamma)$ to $\mathcal{V}$.
10. $\mathcal{V}$ computes $z_{\mathbb{H}}(\gamma) := \gamma^H - 1$, $v_b^{(\gamma)} := v_{\overline{b}}^{(\gamma)} \cdot \gamma + T/H$, and $v_{q_b}^{(\gamma)} := \frac{v_b^{(\gamma)} \cdot (v_s^{(\gamma)} + \beta) - 1}{z_{\mathbb{H}}(\gamma)}$.
11. $\mathcal{V}$ samples $\eta \xleftarrow{\$} \mathbb{F}$ and sends η to $\mathcal{P}$.
12. Both $\mathcal{P}$ and $\mathcal{V}$ can locally determine $v := v_{\overline{b}}^{(\gamma)} + \eta \cdot v_s^{(\gamma)} + \eta^2 \cdot v_{q_b}^{(\gamma)}$.
13. $\mathcal{P}$ computes $\pi := \left[\!\!\left[\frac{f_{\overline{\mathbf{b}}}(x) + \eta \cdot f_{\mathbf{s}}(x) + \eta^2 \cdot q_{\mathbf{b}}(x) - v}{x - \gamma} \right]\!\!\right]_1$ and sends π to $\mathcal{V}$.
14. $\mathcal{V}$ computes $\mathsf{c} := \sigma_1(f_{\overline{\mathbf{b}}}) + \eta \cdot \sigma_1(f_{\mathbf{s}}) + \eta^2 \cdot \sigma_1(q_{\mathbf{b}})$.
15. $\mathcal{V}$ checks whether all of the followings hold:
$$\langle \sigma_1(f_{\mathbf{a}}), \sigma_2(f_{\mathbf{t}}) \rangle = \langle \sigma_1(q_{\mathbf{a}}), \sigma_2(z_{\mathbb{K}}) \rangle \cdot \left\langle \sigma_1(f_{\mathbf{m}}) - \beta \cdot \sigma_1(f_{\mathbf{a}}), [\![1]\!]_2 \right\rangle,$$
$$\left\langle \sigma_1(f_{\overline{\mathbf{b}}}), \left[\!\!\left[x^{K-1-(H-2)} \right]\!\!\right]_2 \right\rangle = \left\langle \sigma_1(f_{\overline{b}\text{-}\mathsf{shifted}}), [\![1]\!]_2 \right\rangle,$$
$$\left\langle \mathsf{c} - [\![v]\!]_1 + \gamma \cdot \pi, [\![1]\!]_2 \right\rangle = \left\langle \pi, [\![x]\!]_2 \right\rangle,$$
$$\left\langle \sigma_1(f_{\mathbf{a}}) - [\![T/K]\!]_1, [\![1]\!]_2 \right\rangle = \left\langle \sigma_1(f_{\overline{\mathbf{a}}}), [\![x]\!]_2 \right\rangle.$$

Fig. 6. Protocol $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$ in $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq}} = (\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}, \Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}})$.

Property of ch. From above, we deduce that ch is non-decreasing and the difference between any two consecutive entries in ch is either 0 or 1. Hence, ch_1 and ch_K are the smallest and greatest values among $\{\mathsf{ch}_i\}_{i=1}^K$. By the first line of (22), we know that $\mathsf{ch}_1 = 1$ and $\mathsf{ch}_K = |\Sigma|$. Therefore, $\{\mathsf{ch}_i\}_{i=1}^K = \Sigma$, as $\Sigma = [|\Sigma|] = \{1, \ldots, |\Sigma|\}$, and ch is a non-decreasing sequence starting from 1

Binary search algorithm for id_i based on dfa_c^{-1}.
Inputs: $s_i = c$ and id_{i-1}.
Output: id_i satisfying Definition 2.
Execution:
1. $L := 1$ and $R := m_c + 1$.
2. If $L = R$, then return $\mathsf{id}_i := j_1^{(c)}$.
3. Repeat the following until id_i is found.
   (a) $M := \lfloor (L+R)/2 \rfloor$.
   (b) If $\mathsf{dfa}_c^{-1}(j_M^{(c)})$ contains id_{i-1}, then return $\mathsf{id}_i := j_M^{(c)}$.
   (c) If id_{i-1} is on the left of the interval $\mathsf{dfa}_c^{-1}(j_M^{(c)})$, then assign $L := M + 1$; otherwise, $R := M - 1$.

Fig. 7. Binary search algorithm for id_i based on dfa_c^{-1}.

and ending at $|\Sigma|$. One see that, for each $c \in \Sigma$, there exist $l_c, r_c \in [K]$ s.t. $l_c \leq r_c$ and, for any $i \in [K]$, $\mathsf{ch}_i = c$ iff $i \in [l_c, r_c]$.

Properties of lft *and* rgt. From the above arguments, we know that

- when $\mathsf{ch}_i \neq \mathsf{ch}_{i-1}$ for $i \in [2, K]$, $\mathsf{lft}_i = 0$, and
- when $\mathsf{ch}_i \neq \mathsf{ch}_{i+1}$ for $i \in [K-1]$, $\mathsf{rgt}_i = N + 1$.

One sees that lft_1 and rgt_K are not considered. In the first line of (22), we know that $\mathsf{lft}_1 = 0$.

We now determine rgt_K. Since ch is non-decreasing and $\{\mathsf{ch}_i\}_{i=1}^K = \Sigma$, we are guaranteed that $\mathsf{ch}_K = |\Sigma|$. We then know that $r_{|\Sigma|} = K$. By the fourth line of (22), i.e., $\{\mathsf{rgt}_i - \mathsf{lft}_i\}_{i=1}^K \subseteq [0, N+1]$, we know that $\mathsf{lft}_i \leq \mathsf{rgt}_i$ for $i \in [K]$. Additionally, from the above arguments, we know that $\mathsf{rgt}_i + 1 = \mathsf{lft}_{i+1}$ for $i \in [K-1]$. Therefore,

$$0 = \mathsf{lft}_{l_{|\Sigma|}} \leq \mathsf{rgt}_{l_{|\Sigma|}} < \mathsf{lft}_{l_{|\Sigma|}+1} \leq \mathsf{rgt}_{l_{|\Sigma|}+1} < \cdots < \mathsf{lft}_{r_{|\Sigma|}} \leq \mathsf{rgt}_{r_{|\Sigma|}}$$

where $l_{|\Sigma|} = 0$ is discussed above. Therefore, $\mathsf{rgt}_{r_{|\Sigma|}}$ is the greatest value among $\mathsf{rgt}_{l_{|\Sigma|}}, \mathsf{rgt}_{l_{|\Sigma|}+1}, \ldots, \mathsf{rgt}_{r_{|\Sigma|}}$. From the third line of (22), we know that $\{\mathsf{rgt}_i\}_{i=1}^K \subseteq [0, N+1]$. Hence,

$$0 = \mathsf{lft}_{l_{|\Sigma|}} \leq \mathsf{rgt}_{l_{|\Sigma|}} < \mathsf{lft}_{l_{|\Sigma|}+1} \leq \mathsf{rgt}_{l_{|\Sigma|}+1} < \cdots < \mathsf{lft}_{r_{|\Sigma|}} \leq \mathsf{rgt}_{r_{|\Sigma|}} \leq N+1.$$

From the fifth line of (22), i.e., $\{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=1}^K = \{(t_i, i-1)\}_{i=1}^N \cup \{(c, N+1)\}_{c \in \Sigma}$, we know that $(|\Sigma|, N+1)$ exists among $\{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=1}^K$. Hence, one deduce that $r_K = \mathsf{rgt}_{r_{|\Sigma|}} = N+1$ since, otherwise, it contradicts the mentioned existence of $(|\Sigma|, N+1)$.

Putting all together. Similarly to the case of $|\Sigma|$, for each $c \in \Sigma$, we have

$$\begin{aligned} &0 = \mathsf{lft}_{l_c} \leq \mathsf{rgt}_{l_c} < \mathsf{lft}_{l_c+1} \leq \mathsf{rgt}_{l_c+1} < \cdots < \mathsf{lft}_{r_c} \leq \mathsf{rgt}_{r_c} = N+1, \text{ and} \\ &\mathsf{rgt}_i + 1 = \mathsf{lft}_{i+1} \ \forall i \in [l_c, r_c - 1]. \end{aligned} \tag{48}$$

We already show that $\{\mathsf{ch}_i\}_{i=1}^K = \Sigma$ and ch is non-decreasing. Therefore, the second line of (18) holds. Additionally, for $c \in \Sigma$, we also see that $([\mathsf{lft}_i, \mathsf{rgt}_i])_{i=l_c}^{r_c}$ are consecutive intervals forming a partition of $[0, N+1]$ according to Definition 4.

Besides, from the fifth line of (22), i.e., $\{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=1}^K = \{(t_j, j-1)\}_{j=1}^N \cup \{(c, N+1)\}_{c \in \Sigma}$. Equivalently, for each $c \in \Sigma$, we can write

$$\{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=l_c}^{r_c} = \{(t_j, j-1)\}_{j \in [N] \wedge t_j = c} \cup \{(c, N+1)\}.$$

Recall that the set $\left\{ j \in [N] \,\middle|\, t_j = c \right\} \cup \{N+1\}$ (c.f. (7)) is equal to $\{j_i^{(c)}\}_{i=1}^{m_c+1}$ where $\{j_i^{(c)}\}_{i=0}^{m_c+1}$ and m_c are defined in Section 3.2. We deduce that

$$\begin{aligned} \{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=l_c}^{r_c} &= \{(t_j, j-1)\}_{j \in [N] \wedge t_j = c} \cup \{(c, N+1)\} \\ &= \{(c, j_i^{(c)} - 1)\}_{i=1}^{m_c} \cup \{(c, N+1)\}. \end{aligned}$$

Since, as from the above arguments, $([\mathsf{lft}_i, \mathsf{rgt}_i])_{i=l_c}^{r_c}$ are consecutive intervals forming a partition of $[0, N+1]$ according to Definition 4, we straightforwardly see that

$$\{(c, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=l_c}^{r_c} = \left(\bigcup_{i=1}^{m_c} \{(c, j_{i-1}^{(c)}, j_i^{(c)} - 1)\}\right) \cup \{(c, j_{m_c}^{(c)}, N+1)\}.$$

where $j_0^{(c)} = 0$. By taking union for all $c \in \Sigma$, we see that

$$\begin{aligned}
\{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=1}^{K} &= \bigcup_{c \in \Sigma} \{(c, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=l_c}^{r_c} \\
&= \bigcup_{c \in \Sigma} \left(\left(\bigcup_{i=1}^{m_c} \{(c, j_{i-1}^{(c)}, j_i^{(c)} - 1)\} \right) \cup \{(c, j_{m_c}^{(c)}, N+1)\} \right) \\
&= \mathcal{S}_{\mathsf{intv}} \qquad \text{(c.f. (8))}.
\end{aligned}$$

We thus conclude the proof. □

B.5 Proof of Lemma 8

Proof (Proof of Lemma 8). We note that id is well-formed if (23) holds as introduced in Section 5.2. We prove this lemma as follows.

"⇒": This direction is straightfoward from the explanation leading to Lemma 8.

"⇐": Suppose that $(\mathsf{ch}, \mathsf{lft}, \mathsf{rgt})$ is the well-formed w.r.t. $\mathbf{t}$ (c.f. (18)). It follows from the first line of (25) that $\mathsf{id}_0 = 0$. The second line of (25), namely, $\{(s_i, u_i, v_i)\}_{i=1}^{n} = \{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=1}^{K}$, is already captured in the first line of (23).

The third line of (25) aims to capture that $\widetilde{\mathsf{id}}_i \in [u_i, v_i]$. By definition, $\widetilde{\mathsf{id}} = (\widetilde{\mathsf{id}}_i)_{i=1}^{n} = (\mathsf{id}_0, \ldots, \mathsf{id}_{n-1})$. Therefore, we see that $\mathsf{id}_{i-1} \in [u_i, v_i]$. Hence, the second line of (23) holds.

Regarding the last line of (25), $\{(v_i, \mathsf{id}_i)\}_{i=1}^{n} \subseteq \{(i-1, i)\}_{i=1}^{N} \cup \{(N+1, N+1)\}$, we can directly deduce that

$$(v_i \in [0, N] \wedge \mathsf{id}_i = v_i + 1) \vee (v_i = N+1 \wedge \mathsf{id}_i = N+1).$$

Hence, the last line of (23) also holds.

Thus, we conclude the proof. □

B.6 Binary Search Algorithm to Search for $(\mathsf{id}_i, u_i, v_i)$ based on $(\mathsf{ch}, \mathsf{lft}, \mathsf{rgt})$

We follow the context discussed in Section 5.2 to present the binary search algorithm, in Figure 8, that looks for (c, u_i, v_i) given $s_i = c$ and id_{i-1}. It follows that $\mathsf{id}_i := v_i + 1$ if $v_i \leq N$; otherwise, $\mathsf{id}_i := N + 1$. We note that the binary search algorithm is a basic algorithm that can be found in any fundamental textbooks about algorithms (e.g., [43]).

We note that this algorithm will definitely halt after $\mathcal{O}(\log_2 K)$ times after repeating the loop to search for id_i. We remind that $(\mathsf{ch}_j, \mathsf{lft}_j, \mathsf{rgt}_j)_{j=1}^{K}$ are arranged based on the second line of (18).

Hence, for computing $\mathsf{id} = (\mathsf{id}_i)_{i=0}^{n}$, the total time is $\mathcal{O}(n \log_2 K)$.

C Handling Preprocessing (Extended)

C.1 Polynomial Constraints for Preprocessing

This appendix is a detailed discussion of Section 6.1.

We state the formal version of Theorem 1 in this appendix (c.f. Theorem 6). We now discuss our way leading to Theorem 6.

Recall $g_{\mathsf{suf}\text{-}\Sigma}(X)$, $g_{\mathsf{rg}_N}(X)$ $f_{\mathbf{t}}(X)$, $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, and $f_{\mathsf{rgt}}(X)$ in (26). In brief, these polynomials are interpolated over the group $\mathbb{K} = \{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K}$ to encode Σ, $[0, N+1]$, $\mathbf{t}$, ch, lft, and rgt. Theorem 6 is built upon (22) via modeling each line of (22) into polynomial constraints as follows.

Polynomial constraints for the first line of (22)**.** The 1st line of this system captures $\mathsf{ch}_1 = 1$, $\mathsf{ch}_K = |\Sigma|$, and $\mathsf{lft}_1 = 0$. This can be done by simply providing the evaluations $f_{\mathsf{ch}}(\omega_{\mathbb{K}}^0) = 1$, $f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{K-1}) = |\Sigma|$, and $f_{\mathsf{lft}}(\omega_{\mathbb{K}}^0) = 0$, respectively.

Binary search algorithm for id_i based on $(\mathsf{ch}, \mathsf{lft}, \mathsf{rgt})$.
Inputs: $s_i = c$ and id_{i-1}.
Output: (u_i, v_i) s.t. $\mathsf{id}_{i-1} \in [u_i, v_i]$ and $(c, u_i, v_i) \in \{(\mathsf{ch}_j, \mathsf{lft}_j, \mathsf{rgt}_j)\}_{j=1}^{K}$.
Execution:
1. $L := 1$ and $R := K$.
2. If $L = R$, then return $(u_i, v_i) := (\mathsf{lft}_1, \mathsf{rgt}_1)$.
3. Repeat until (u_i, v_i), satisfying $u_M \leq \mathsf{id}_{i-1} \leq v_M$, is found.
   (a) $M := \lfloor (L + R)/2 \rfloor$.
   (b) If $c = \mathsf{ch}_M \wedge \mathsf{id}_{i-1} \in [u_M, v_M]$, then return $(u_i, v_i) := (u_M, v_M)$.
   (c) If $c < \mathsf{ch}_M \vee (c = \mathsf{ch}_M \wedge \mathsf{id}_{i-1} < u_M)$, then assign $L := M + 1$; otherwise, $R := M - 1$.

Fig. 8. Binary search algorithm for (c, u_i, v_i) based on $(\mathsf{ch}, \mathsf{lft}, \mathsf{rgt})$.

Polynomial constraints for the second and third lines of (22)**.** Recall the 2nd and 3rd lines of (22) in the following equations (49) and (50).

$$\{(\mathsf{ch}_{i+1} - \mathsf{ch}_i, \mathsf{lft}_{i+1} - \mathsf{rgt}_i)\}_{i=1}^{K-1} \subseteq \{(0, 1), (1, -(N + 1))\}, \tag{49}$$

$$\{\mathsf{lft}_i\}_{i=1}^{K} \subseteq [0, N + 1], \{\mathsf{rgt}_i\}_{i=1}^{K} \subseteq [0, N + 1]. \tag{50}$$

We first notice that (49) is a tuple lookup argument, which was considered in SublonK [14] and MUXProofs [17] for proving machine computations. However, proving tuple lookups may cause trouble with this result for optimizing the running time of the prover by pushing a large amount of work to preprocessing, and CQ's technique is not natural for proving tuple lookups. (SublonK manipulates CQ by introducing additional constraints and structures.)

Fortunately, Σ is the alphabet and, in reality, is usually not very large, e.g., the English alphabet has 26 letters, and the genetic alphabet has 4 chemical bases. This happens with entries of $\mathbf{t}$ and ch as they belong to Σ. On the other hand, entries of lft and rgt lie inside $[0, N + 1]$ (c.f. (50)). Therefore, by setting $\mathbb{F}$ to have sufficiently large cardinality and choosing a large enough base B in $\mathbb{F}$, we can prove (49) by proving the RHS of the equivalence (51).

$$(49) \iff \{(\mathsf{ch}_{i+1} - \mathsf{ch}_i) + B \cdot (\mathsf{lft}_{i+1} - \mathsf{rgt}_i)\}_{i=1}^{K-1} \subseteq \{B, 1 - B \cdot (N + 1)\}. \tag{51}$$

The detailed choice of B is deferred to Appendix E. Hence, we need to additionally prove that (50) holds and

$$\{\mathsf{ch}_i\}_{i=1}^{K} \subseteq \Sigma. \tag{52}$$

Otherwise, a cheating prover may use sufficiently large entries (exceeding the required ranges) to break the soundness of the argument.

Hence, we need to perform the polynomial constraints for the RHS of (51) and (52).

Polynomial constraints for the RHS of (51). First, as discussed above, we reduce the task of proving tuple lookup to proving ordinary lookup arguments, which is simpler for us to handle subsequently. The only sacrifice here is the setting of $\mathbb{F}$ to have a sufficiently large cardinality.

Second, we spot the following two challenges in its structure.

- The lookup is with respect to sequences of lengths $K - 1$ (for LHS) and 2 (for RHS) while we previously defined the group $\mathbb{K}$ of cardinality K. This makes things troublesome since, to be able to prove such lookup arguments, we need to either (i) make the respective sequences be encoded in polynomials of suitable domains (i.e., $\mathbb{K}$) or (ii) introduce additional multiplicative subgroups of $\mathbb{F}$ with appropriate cardinalities, e.g., K and 2. However, the latter option does not seem possible. We will later introduce the approach with respect to the former one.
- We need to compute the mismatching indices, e.g., computing $\mathsf{ch}_{i+1} - \mathsf{ch}_i$ or $\mathsf{lft}_{i+1} - \mathsf{rgt}_i$ that requires accessing to sequences at indices i and $i + 1$.

To encounter these issues, we define the polynomial $f_{\mathsf{adj}}(X)$[13] in (53) that encodes $\{(\mathsf{ch}_{i+1}-\mathsf{ch}_i)+B\cdot(\mathsf{lft}_{i+1}-\mathsf{rgt}_i)\}_{i=1}^{K-1}$.

$$f_{\mathsf{adj}}(X) \text{ s.t. } \begin{cases} \omega_{\mathbb{K}}^{i-1} \mapsto (\mathsf{ch}_{i+1}-\mathsf{ch}_i)+B\cdot(\mathsf{lft}_{i+1}-\mathsf{rgt}_i) & \text{if } i\in[K-1], \\ \omega_{\mathbb{K}}^{i-1} \mapsto (|\Sigma|+1-\mathsf{ch}_i)-B\cdot\mathsf{rgt}_i & \text{if } i=K. \end{cases} \tag{53}$$

Notice that $f_{\mathsf{adj}}(X)$ encodes all elements in $\{(\mathsf{ch}_{i+1}-\mathsf{ch}_i)+B\cdot(\mathsf{lft}_{i+1}-\mathsf{rgt}_i)\}_{i=1}^{K-1}$, via evaluations at $\{\omega_{\mathbb{K}}^{i-1}\}_{i=1}^{K-1}$, and an additional element $(|\Sigma|+1-\mathsf{ch}_i)-B\cdot\mathsf{rgt}_i$ via the evaluation at $\omega_{\mathbb{K}}^{K-1}$. The setting of $f_{\mathsf{adj}}(\omega_{\mathbb{K}}^{K-1})$ is to enforce it to belong to $\{B, 1-B\cdot(N+1)\}$ (c.f. the RHS of (51)) via guaranteeing $f_{\mathsf{adj}}(\omega_{\mathbb{K}}^{K-1}) = (|\Sigma|+1-\mathsf{ch}_K)-B\cdot\mathsf{rgt}_K$, which is equal to $1-B\cdot(N+1)$. In fact, by the 1st line of (22), $\mathsf{ch}_K = |\Sigma|$, and $\mathsf{rgt}_K = N+1$ as discussed in Section 5.1. Therefore, $(|\Sigma|+1-\mathsf{ch}_K)-B\cdot\mathsf{rgt}_K = 1-B\cdot(N+1)$ falls into the set $\{B, 1-B\cdot(N+1)\}$.

Due to the issue of mismatching indices, e.g., accessing ch at indices i and $i+1$, to construct $f_{\mathsf{adj}}(X)$, we additionally define the sequences

$$\begin{aligned} \widehat{\mathsf{ch}} = (\widehat{\mathsf{ch}}_1,\ldots,\widehat{\mathsf{ch}}_K) &:= (\underbrace{\mathsf{ch}_2,\mathsf{ch}_3,\ldots,\mathsf{ch}_K}_{\text{suffix of } \mathsf{ch}}, |\Sigma|+1), \text{ and} \\ \widehat{\mathsf{lft}} = (\widehat{\mathsf{lft}}_1,\ldots,\widehat{\mathsf{lft}}_K) &:= (\underbrace{\mathsf{lft}_2,\mathsf{lft}_3,\ldots,\mathsf{lft}_K}_{\text{suffix of } \mathsf{lft}}, 0). \end{aligned} \tag{54}$$

We can encode $\widehat{\mathsf{ch}}$ and $\widehat{\mathsf{lft}}$ into the following polynomials.

$$\begin{aligned} f_{\widehat{\mathsf{ch}}}(X) \quad \text{s.t.} \quad \omega_{\mathbb{K}}^{i-1} \mapsto \widehat{\mathsf{ch}}_i \quad \forall i\in[K], \text{ and} \\ f_{\widehat{\mathsf{lft}}}(X) \quad \text{s.t.} \quad \omega_{\mathbb{K}}^{i-1} \mapsto \widehat{\mathsf{lft}}_i \quad \forall i\in[K]. \end{aligned} \tag{55}$$

Thus, we see that, by setting

$$f_{\mathsf{adj}}(X) := (f_{\widehat{\mathsf{ch}}}(X) - f_{\mathsf{ch}}(X)) + B\cdot(f_{\widehat{\mathsf{lft}}}(X) - f_{\mathsf{rgt}}(X)), \tag{56}$$

(53) holds.

We encode $\{B, 1-B\cdot(N+1)\}$ by using the polynomial $g_{\mathsf{diff}}(X)$[14] in (26). Notice here that we duplicate the element $1-B\cdot(N+1)$ in the evaluations of $g_{\mathsf{diff}}(X)$ over $\omega_{\mathbb{K}}^{i-1}$ since we only need to use this polynomial to encode only two elements, namely, B and $1-B\cdot N$, and to fill up the K evaluations over $\mathbb{K}$. By (51), (56), and $g_{\mathsf{diff}}(X)$ in (26), we see that (49) can be proved by showing that

$$\begin{aligned} &\{(f_{\widehat{\mathsf{ch}}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1})) + B\cdot(f_{\widehat{\mathsf{lft}}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}))\}_{i=1}^{K} \\ &\subseteq \{g_{\mathsf{diff}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \end{aligned} \tag{57}$$

where the LHS is $\{f_{\mathsf{adj}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}$ according to (56).

Nevertheless, we need to additionally prove the well-formedness of $f_{\widehat{\mathsf{ch}}}(X)$ and $f_{\widehat{\mathsf{lft}}}(X)$, namely, showing that $f_{\widehat{\mathsf{ch}}}(X)$ and $f_{\widehat{\mathsf{lft}}}(X)$ are constructed correctly.

To show the well-formedness of $f_{\widehat{\mathsf{ch}}}(X)$, we note that

$$f_{\widehat{\mathsf{ch}}}(X) = f_{\mathsf{ch}}(\omega_{\mathbb{K}}\cdot X) + |\Sigma|\cdot\ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}}\cdot X) \tag{58}$$

which can be explained as follows. The term $f_{\mathsf{ch}}(\omega_{\mathbb{K}}\cdot X)$ encodes a left rotation of ch (c.f. Lemma 1). Specifically, $f_{\mathsf{ch}}(\omega_{\mathbb{K}}\cdot X)$ encodes $(\mathsf{ch}_2,\ldots,\mathsf{ch}_K,\mathsf{ch}_1)$. Then, by adding $|\Sigma|\cdot\ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}}\cdot X)$, we aim to transform ch_1 into $|\Sigma|+1$ since $f_{\mathsf{ch}}(\omega_{\mathbb{K}}\cdot X) = \sum_{i=2}^{K}\mathsf{ch}_i\cdot\ell_{\mathbb{K}}^{(i)}(\omega_{\mathbb{K}}\cdot X) + \mathsf{ch}_1\cdot\ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}}\cdot X)$ and $\mathsf{ch}_1 = 1$ (c.f. 1st line of (22)).

To show the well-formedness of $f_{\widehat{\mathsf{lft}}}(X)$, with a similar argument, we have

$$f_{\widehat{\mathsf{lft}}}(X) = f_{\mathsf{lft}}(\omega_{\mathbb{K}}\cdot X). \tag{59}$$

[13] The naming of $f_{\mathsf{adj}}(X)$ aims to encode values computed from "adjacent" entries by indices, e.g., $(\mathsf{ch}_{i+1}-\mathsf{ch}_i)+B\cdot(\mathsf{lft}_i-\mathsf{rgt}_i)$ is computed from $\mathsf{ch}_{i+1}-\mathsf{ch}_i$ and $\mathsf{lft}_{i+1}-\mathsf{rgt}_i$ via extracting values from $(\mathsf{ch}_i,\mathsf{lft}_r,\mathsf{rgt}_i)$ and $(\mathsf{ch}_{i+1},\mathsf{lft}_{i+1},\mathsf{rgt}_{i+1})$ at indices i and $i+1$, respectively.

[14] The naming of $g_{\mathsf{diff}}(X)$ aims to capture the values based on the "differences" computed from adjacent entries $(\mathsf{ch}_i,\mathsf{lft}_i,\mathsf{rgt}_i)$ and $(\mathsf{ch}_{i+1},\mathsf{lft}_{i+1},\mathsf{rgt}_{i+1})$.

Polynomial constraints for (50) *and* (52). We can prove (50) and (52) by using Haböck's lookup [32] for the following system.

$$\begin{cases} \{f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\ \{f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\Sigma}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}. \end{cases} \tag{60}$$

where $g_{\Sigma}(X)$ and $g_{\mathsf{rg}_N}(X)$ are defined in (26). Here, we do not use CQ-based techniques since these lookups fall inside the preprocessing part. Therefore, proving with Haböck's lookup is more efficient.

Hence, we have the following Lemma 13.

Lemma 13. *Let B and $\mathbb{F}$ be appropriately set s.t.* (51) *holds. Then, the 2rd line of* (22)*, captured by* (49)*, together are satisfied iff there exist $f_{\widehat{\mathsf{ch}}}(X), f_{\widehat{\mathsf{lft}}}(X) \in \mathbb{F}_{<K}[X]$ s.t.* (57)*,* (58)*,* (59)*, and* (60) *hold.*

The proof of Lemma 13 directly follows the above discussion.

Polynomial constraints for the fourth line of (22)**.** Recall the fourth line of (22) in the following equation (61).

$$\{\mathsf{rgt}_i - \mathsf{lft}_i\}_{i=1}^{K} \subseteq [0, N+1]. \tag{61}$$

This case is rather straightforward. One can see that $f_{\mathsf{rgt}}(X) - f_{\mathsf{lft}}(X)$ and $g_{\mathsf{rg}_N}(X)$ (c.f. (26)) encode the LHS and RHS, respectively, of (61). Hence, we have the following equivalence (61).

$$(61) \iff \{f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}. \tag{62}$$

Polynomial constraints for the fifth line of (22)**.** Recall the 5th line of (22) in the following equation (63).

$$\{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=1}^{K} = \{(t_i, i-1)\}_{i=1}^{N} \cup \{(c, N+1)\}_{c\in\Sigma}. \tag{63}$$

By setting B and $\mathbb{F}$ appropriately (i.e., B and $|\mathbb{F}|$ should be sufficiently large), this can be done by equivalently showing the RHS of the equivalence (64).

$$(63) \iff \{\mathsf{ch}_i + B\cdot\mathsf{rgt}_i\}_{i=1}^{K} = \{t_i + B\cdot(i-1)\}_{i=1}^{N} \cup \{c + B\cdot(N+1)\}_{c\in\Sigma}. \tag{64}$$

The set $\{\mathsf{ch}_i + B\cdot\mathsf{rgt}_i\}_{i=1}^{K}$ is encoded by polynomial

$$f_{\mathsf{ch}}(X) + B\cdot f_{\mathsf{rgt}}(X) \tag{65}$$

and $\{t_i + B\cdot(i-1)\}_{i=1}^{N} \cup \{c + B\cdot(N+1)\}_{c\in\Sigma}$ is encoded by the polynomial

$$f_{\mathsf{t}}(X) + g_{\mathsf{suf}\text{-}\Sigma}(X) + B\cdot g_{\mathsf{rg}_N}(X). \tag{66}$$

Notice that all K entries of the LHS of (63) are distinct. We can hence turn the RHS of (64) into the permutation

$$\begin{aligned} &(f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1}) + B\cdot f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}))_{i=1}^{K} \\ &\quad \bowtie (f_{\mathsf{t}}(\omega_{\mathbb{K}}^{i-1}) + g_{\mathsf{suf}\text{-}\Sigma}(\omega_{\mathbb{K}}^{i-1}) + B\cdot g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1}))_{i=1}^{K} \end{aligned} \tag{67}$$

whose the LHS and RHS are respectively (65) and (66). Hence, we have the following Lemma 14.

Lemma 14. *With appropriate settings of B and $\mathbb{F}$ s.t.* (64) *holds, the 5th line of* (22)*, recalled in* (63)*, is satisfied iff* (67) *holds.*

The proof of Lemma 14 directly follows the above discussion.

Putting all together. In summary, to prove system (22), we the following Theorem 6.

Theorem 6. *Let $\mathbb{F}_{<K}[X]$-polynomials $g_{\Sigma}(X)$, $g_{\mathsf{suf}\text{-}\Sigma}(X)$, $g_{\mathsf{rg}_N}(X)$, $g_{\mathsf{diff}}(X)$, $f_{\mathsf{t}}(X)$, $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, and $f_{\mathsf{rgt}}(X)$ be defined as in* (26)*. Let B and $\mathbb{F}$ be appropriately set s.t.* (51) *and* (64) *hold. Then, the constraints for preprocessing, captured by* (22)*, are satisfied iff there exist $f_{\widehat{\mathsf{ch}}}(X)$ and $f_{\widehat{\mathsf{lft}}}(X)$ s.t.*

$$
\begin{cases}
f_{\mathbf{t}}(\omega_{\mathbb{K}}^{i-1}) = 0 \ \forall i \in [N+1, K], \\
f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{0}) = 1, \ f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{K-1}) = |\Sigma|, \ f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{0}) = 0, \\
\{(f_{\widehat{\mathsf{ch}}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1})) + B \cdot (f_{\widehat{\mathsf{lft}}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}))\}_{i=1}^{K} \\
\quad \subseteq \{g_{\mathsf{diff}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\
f_{\widehat{\mathsf{ch}}}(X) = f_{\mathsf{ch}}(\omega_{\mathbb{K}} \cdot X) + |\Sigma| \cdot \ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}} \cdot X), \\
f_{\widehat{\mathsf{lft}}}(X) = f_{\mathsf{lft}}(\omega_{\mathbb{K}} \cdot X), \\
\{f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\
\{f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\
\{f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\Sigma}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\
\{f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}) - f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K} \subseteq \{g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1})\}_{i=1}^{K}, \\
(f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{i-1}) + B \cdot f_{\mathsf{rgt}}(\omega_{\mathbb{K}}^{i-1}))_{i=1}^{K} \\
\quad \bowtie (f_{\mathbf{t}}(\omega_{\mathbb{K}}^{i-1}) + g_{\mathsf{suf}\text{-}\Sigma}(\omega_{\mathbb{K}}^{i-1}) + B \cdot g_{\mathsf{rg}_N}(\omega_{\mathbb{K}}^{i-1}))_{i=1}^{K}.
\end{cases} \tag{68}
$$

Proof. The proof follows Lemmas 13 and 14, and equivalence (62). □

C.2 Proof of Theorem 3

Proof (Proof of Theorem 3). Let $\mathcal{A}$ be a PPT algebraic adversary breaking the knowledge soundness of $\Pi_{\mathsf{ssq\text{-}off}}^{\mathbb{K}}$. We show $\mathcal{A}$ has negligible probability $\mathsf{negl}(\lambda)$. Assume that $\mathcal{A}$ cannot break the evaluation binding of KZG PCS. As $\mathcal{A}$ is algebraic, we can obtain $f_{\mathbf{t}}(X)$ and $\mathsf{wit}_{\mathsf{off}}$ including $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, $f_{\mathsf{rgt}}(X)$, $f_{\widehat{\mathsf{ch}}}(X)$, $f_{\widehat{\mathsf{lft}}}(X)$, and $f_{\mathsf{comb}}(X)$ when $\mathcal{A}$ sends their commitments during the protocol. Then, we show the above functions we extracted from $\mathcal{A}$ satisfying (27).

- The following holds as the underlying PCSs satisfy binding and evaluation binding with soundness error $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$.
  - Regarding $f_{\widehat{\mathsf{ch}}}(\rho) = f_{\mathsf{ch}}(\omega_{\mathbb{K}} \cdot \rho) + |\Sigma| \cdot \ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}} \cdot \rho)$, since $\rho \xleftarrow{\$} \mathbb{F}$,
  $$f_{\widehat{\mathsf{ch}}}(X) = f_{\mathsf{ch}}(\omega_{\mathbb{K}} \cdot X) + |\Sigma| \cdot \ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}} \cdot X)$$
  holds with soundness error $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$ by Schwartz-Zippel Lemma (recalled in Appendix A.4) and the evaluation binding of KZG PCS.
  - Similarly as above,
  $$f_{\widehat{\mathsf{lft}}}(X) = f_{\mathsf{lft}}(\omega_{\mathbb{K}} \cdot X)$$
  holds.
  - $f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{0}) = \mathsf{ch}_1$ and $f_{\mathsf{lft}}(\omega_{\mathbb{K}}^{0}) = 1$.
  - $\sigma_1(f_{\mathbf{t}})$ and $\sigma_2(f_{\mathbf{t}})$ commit to the same polynomial $f_{\mathbf{t}}(X)$.
  - $f_{\mathbf{t}}(\omega_{\mathbb{K}}^{i-1}) = 0 \ \forall i \in [N+1, K]$.
  - function $f_{\mathsf{comb}}(X)$ satisfies $f_{\mathsf{comb}}(X) = f_{\mathsf{ch}}(X) + B \cdot f_{\mathsf{lft}}(X) + B^2 \cdot f_{\mathsf{rgt}}(X)$.
- The remaining lookup and permutation arguments hold with soundness error $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$, which can be straightforwardly deduced from Section 2.4.

Therefore, we conclude that $f_{\mathbf{t}}(X)$, $f_{\mathsf{ch}}(X)$, $f_{\mathsf{lft}}(X)$, $f_{\mathsf{rgt}}(X)$, $f_{\widehat{\mathsf{ch}}}(X)$, and $f_{\widehat{\mathsf{lft}}}(X)$ satisfy (27) and the adversary $\mathcal{A}$ breaks the knowledge soundness with probability $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$ where $\mathsf{negl}(\lambda)$ indicates the probability that $\mathcal{A}$ breaks the evaluation binding of KZG PCS. □

D Handling Proving Phase (Extended)

D.1 Proof of Theorem 4

Proof (Proof of Theorem 4). Let $\mathcal{A}$ be a PPT algebraic adversary breaking the knowledge soundness of $\Pi_{\mathsf{ssq\text{-}on}}^{\mathbb{H},\mathbb{K}}$. We show $\mathcal{A}$ has negligible probability $\mathsf{negl}(\lambda)$. Assume that $\mathcal{A}$ cannot break the evaluation binding of KZG PCS. As $\mathcal{A}$ is algebraic, we can obtain $f_{\mathbf{s}}(X)$, $f_{\mathsf{id}}(X)$, $f_{\widetilde{\mathsf{id}}}(X)$, $f_{\mathbf{u}}(X)$, and $f_{\mathbf{v}}(X)$, when $\mathcal{A}$ sends their commitments during the protocol.

– The following holds as the underlying PCSs satisfy binding and evaluation binding with soundness error $\mathcal{O}(K/|\mathbb{F}|)$.
  - $f_{\mathsf{id}}(\omega_{\mathbb{H}}^{n-1}) = \mathsf{eoid}$,
  - $f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}} \cdot X) = f_{\mathsf{id}}(X) - \mathsf{eoid} \cdot \ell_{\mathbb{H}}^{(n)}(X)$.
– The remaining lookup arguments ($\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$) hold with soundness error $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$ which can be straightforwardly deduced from Section 2.4.

Therefore, we conclude that $f_{\mathbf{s}}(X)$, $f_{\mathsf{id}}(X)$, $f_{\widetilde{\mathsf{id}}}(X)$, $f_{\mathbf{u}}(X)$, and $f_{\mathbf{v}}(X)$ satisfy (43) and the adversary $\mathcal{A}$ breaks the knowledge soundness with probability $\mathcal{O}(K/|\mathbb{F}| + \mathsf{negl}(\lambda))$ where $\mathsf{negl}(\lambda)$ is the probability that $\mathcal{A}$ breaks evaluation binding of KZG PCS. □

E Setting B and $\mathbb{F}$

As introduced in Section 7.1, we should choose B and $\mathbb{F}$ s.t. $B \geq 2 \cdot |\Sigma| + N$ and $|\mathbb{F}| > 2 \cdot B^2 \cdot (N+1)$ to satisfy both Theorem 1 (with formal version in Theorem 6) and Theorem 2. We now explain such a setting as follows.

As required by both theorems, we must set B and $\mathbb{F}$ s.t. the following cases hold.

The case (51). In this case, we want to guarantee that

$$\begin{aligned} &\{(\mathsf{ch}_{i+1} - \mathsf{ch}_i, \mathsf{lft}_{i+1} - \mathsf{rgt}_i)\}_{i=1}^{K-1} \subseteq \{(0,1), (1, -(N+1))\} \\ &\iff \\ &\{(\mathsf{ch}_{i+1} - \mathsf{ch}_i) + B \cdot (\mathsf{lft}_{i+1} - \mathsf{rgt}_i)\}_{i=1}^{K-1} \subseteq \{B, 1 - B \cdot (N+1)\}. \end{aligned} \tag{69}$$

Notice that, in system (22), we already enforced $\{\mathsf{ch}_i\}_{i=1}^{K} \subseteq \Sigma = [|\Sigma|]$, $\{\mathsf{lft}_i\}_{i=1}^{K} \subseteq [N+1]$, and $\{\mathsf{rgt}_i\}_{i=1}^{K} \subseteq [N+1]$. Hence, we see that $\mathsf{ch}_{i+1} - \mathsf{ch}_i \in [-|\Sigma|+1, |\Sigma|-1]$ for $i \in [K-1]$, and $\mathsf{lft}_{i+1} - \mathsf{rgt}_i \in [-N, N]$ for $i \in [K-1]$. Hence, our idea is to set B and $\mathbb{F}$ s.t., for any $x, y \in [-|\Sigma|+1, |\Sigma|-1]$ and any $u, v \in [-N, N]$, it must hold that $(x,u) \neq (y,v) \iff x + B \cdot u \neq y + B \cdot v$. Assume that $(x,u) \neq (y,v)$. Note that $x + B \cdot u \neq y + B \cdot v \iff x - y \neq B \cdot (v-u)$. We have the following sub-cases.

– If $u = v$, then $x \neq y$ because $(x,u) \neq (y,v)$, and hence it holds that $B \cdot (v-u) = 0 \neq (x-y)$ for any B. So, setting any B in this case must be fine.
– If $u \neq v$, we would like to guarantee that $x - y \neq B \cdot (v-u)$. The idea is to enforce that $|x-y| < |B \cdot (v-u)|$, hence implying $x - y \neq B \cdot (v-u)$. We see that $|x-y| \leq 2 \cdot (|\Sigma| - 1)$ and $1 \leq |v-u| \leq 2 \cdot N$ as $u \neq v$. Therefore, by setting $B > 2 \cdot (|\Sigma| - 1)$, we always have $|x-y| < |B \cdot (v-u)|$. Hence, $B \geq 2 \cdot |\Sigma| - 1$. However, since these values lie inside $\mathbb{F}$, we must guarantee that $\mathbb{F}$ is sufficiently large to contain all potential values. Notice that $|B \cdot (v-u)|$ is at most $B \cdot 2 \cdot N$. Hence, $|\mathbb{F}|$ must be at least $4 \cdot B \cdot N + 1$ to contain all possible values (0, positive, and negative values) of $B \cdot (v-u)$. Hence, $|\mathbb{F}| > 4 \cdot B \cdot N$.

The case (64). In this case, we want to guarantee that

$$\begin{aligned} &\{(\mathsf{ch}_i, \mathsf{rgt}_i)\}_{i=1}^{K} = \{(t_i, i-1)\}_{i=1}^{N} \cup \{(c, N+1)\}_{c \in \Sigma} \\ &\iff \\ &\{\mathsf{ch}_i + B \cdot \mathsf{rgt}_i\}_{i=1}^{K} = \{t_i + B \cdot (i-1)\}_{i=1}^{N} \cup \{c + B \cdot (N+1)\}_{c \in \Sigma}. \end{aligned}$$

Similar to the previous case, here we would like to set B and $\mathbb{F}$ satisfying, for any $x, y \in [|\Sigma|]$ and $u, v \in [0, N+1]$, it holds that $(x,u) \neq (y,v) \iff x + B \cdot u \neq y + B \cdot v$. Assume $(x,u) \neq (y,v)$. We would like to have that $x + B \cdot v \neq y + B \cdot v$, equivalently, $x - y \neq B \cdot (v-u)$. Here, we also consider two cases as follows.

– If $u = v$, then $x \neq y$ because $(x,u) \neq (y,v)$. Hence, $B \cdot (v-u) = 0 \neq x - y$ with any setting of B.
– If $u \neq v$, then to guarantee that $x - y \neq B \cdot (v-u)$, we enforce $|x-y| < |B \cdot (v-u)|$. Note that $|x-y| \leq |\Sigma| - 1$ and $1 \leq |v-u| \leq N+1$ as $u \neq v$. Hence, one can choose $B \geq |\Sigma|$. In this case, to be simple, we choose $|\mathbb{F}| > 2 \cdot B \cdot (N+1)$.

The case (32). In this case, we want to guarantee that

$$\begin{aligned}&\{(s_i, u_i, v_i)\}_{i=1}^{n} \subseteq \{(\mathsf{ch}_i, \mathsf{lft}_i, \mathsf{rgt}_i)\}_{i=1}^{K}\\ &\iff\\ &\{s_i + B \cdot u_i + B^2 \cdot v_i\}_{i=1}^{n} \subseteq \{\mathsf{ch}_i + B \cdot \mathsf{lft}_i + B^2 \cdot \mathsf{rgt}_i\}_{i=1}^{K}.\end{aligned}$$

Recall that $\{s_i\}_{i=1}^{n} \subseteq \Sigma$, $\{u_i\}_{i=1}^{n} \subseteq [0, N+1]$, and $\{v_i\}_{i=1}^{n} \subseteq [0, N+1]$. We would like to set B and $\mathbb{F}$ s.t., for any $x, y \in [|\Sigma|]$ and $a_1, a_2, b_1, b_2 \in [0, N+1]$, $(x, a_1, a_2) \neq (y, b_1, b_2) \iff x+B\cdot a_1+B^2\cdot a_2 \neq y+B\cdot b_1+B^2\cdot b_2$. Assume that $(x, a_1, a_2) \neq (y, b_1, b_2)$. As $x+B\cdot a_1+B^2\cdot a_2 \neq y+B\cdot b_1+B^2\cdot b_1 \iff (x-y)+B\cdot(a_1-b_1) \neq B^2\cdot(b_2-a_2)$, we have the following sub-cases.

- If $a_2 = b_2$, then it must hold that $(x, a_1) \neq (y, b_1)$ as $(x, a_1, a_2) \neq (y, b_1, b_2)$. Hence, $(x-y) + B\cdot(a_1-b_1) \neq B^2\cdot(b_2-a_2) = 0$ is equivalent to $x - y \neq B\cdot(b_1-a_1)$.
  - If $a_1 = b_1$ then it follows that $x \neq y$ as $(x, a_1) \neq (y, b_1)$. Hence, any B is satisfied.
  - If $a_1 \neq b_1$, we would like to guarantee $x-y \neq B\cdot(b_1-a_1)$ by setting $|x-y| < |B\cdot(b_1-a_1)|$. Note that $|x-y| \leq |\Sigma|-1$ and $1 \leq |b_1-a_1| \leq N+1$. Hence, B can be taken s.t. $B \geq |\Sigma|$. Additionally, we would like to set $|\mathbb{F}| > 2\cdot B\cdot(N+1)$.
- If $a_2 \neq b_2$, then to make $(x-y)+B\cdot(a_1-b_1) \neq B^2\cdot(b_2-a_2)$, we set B s.t. $|(x-y)+B\cdot(a_1-b_1)| < \left|B^2\cdot(b_2-a_2)\right|$. Since $|x-y| \leq |\Sigma|-1$ and $|a_1-b_1| \leq N+1$, the left hand side is bounded by $|\Sigma|-1+B\cdot(N+1)$. On the other hand, $1 \leq |b_2-a_2| \leq N+1$ as $a_2 \neq b_2$. Hence, to guarantee that

  $$|(x-y)+B\cdot(a_1-b_1)| < \left|B^2\cdot(b_2-a_2)\right|,$$

  we take B s.t. $|\Sigma|-1+B\cdot(N+1) < B^2$. Note that $|\Sigma|-1 \geq 1$ since, otherwise, $|\Sigma| \leq 1$ is meaningless. For this purpose, one can choose $B \geq |\Sigma|+N$. Consequently, we choose $\mathbb{F}$ s.t. $|\mathbb{F}| > 2\cdot B^2\cdot(N+1)$.

In summary, we can choose $B \geq |\Sigma|+N$ and $|\mathbb{F}| > 2\cdot B^2\cdot(N+1)$ for this case.

The case (39). In this case, we want to guarantee that

$$\begin{aligned}&\{(v_i, \mathsf{id}_i)\}_{i=1}^{n} \subseteq \{(i-1, i)\}_{i=1}^{N} \cup \{(N+1, N+1)\}\\ &\iff\\ &\{v_i + B\cdot \mathsf{id}_i\}_{i=1}^{n} \subseteq \{i-1+B\cdot i\}_{i=1}^{N} \cup \{(B+1)\cdot(N+1)\}.\end{aligned}$$

Recall that $\{v_i\}_{i=1}^{n} \subseteq [0, N+1]$ and $\{\mathsf{id}_i\}_{i=1}^{n} \subseteq [N+1]$. In this case, we would like to set B and $\mathbb{F}$ s.t., for any $x, y \in [0, N+1]$ and $u, v \in [N+1]$, it holds that $(x, u) \neq (y, v) \iff x+B\cdot u \neq y+B\cdot v \iff x-y \neq B\cdot(v-u)$. Similar to the previous cases, we have the following two cases.

- If $u = v$, then $x \neq y$ and any setting of B works.
- If $u \neq v$, then we want to set $|x-y| < |B\cdot(v-u)|$. We have that $|x-y| \leq N+1$ and $1 \leq |v-u| \leq N$ as $u \neq v$. Hence, we set $B \geq N+2$. We also can take $|\mathbb{F}| > 2\cdot B\cdot N$.

Putting all together. Since $|\Sigma| \geq 2$ and $N \geq 1$, we choose B s.t.

$$B \geq 2\cdot|\Sigma|+N.$$

This implies that $B \geq 2$. Therefore, we choose $\mathbb{F}$ s.t.

$$|\mathbb{F}| > 2\cdot B^2\cdot(N+1).$$

F Concrete Costs of Our Subsequence Scheme

Recall that $K = N + |\Sigma|$. We now estimate the concrete cost of $\widetilde{\mathcal{SSQ}}$ described in Section 7.4.

F.1 Algorithm $\mathsf{OffProve}_{\mathsf{pp}}(\sigma_1(f_{\mathsf{t}}), f_{\mathsf{t}}(X)) \to (\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}})$

Prover Time. Determine $\mathsf{ch}, \mathsf{lft}, \mathsf{rgt}$, $\widehat{\mathsf{ch}}$, and $\widehat{\mathsf{lft}}$ can be done in $\mathcal{O}(K)$ $\mathbb{F}$-operations. Encoding into polynomials and committing polynomials cost $\mathcal{O}(K)$ $\mathbb{G}_1$-, $\mathbb{G}_2$-, and $\mathbb{F}$-operations since since $\left\{\left[\!\left[\ell_{\mathbb{K}}^{(i)}\right]\!\right]_1\right\}_{i=1}^{K}$ and $\left\{\left[\!\left[\ell_{\mathbb{K}}^{(i)}\right]\!\right]_2\right\}_{i=1}^{K}$ is determined in advance in pp.

In $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$, it can be easily seen that the prover time is dominated by the preprocessings by $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$, the PCS batch evaluation $f_{\mathbf{t}}(\omega_{\mathbb{K}}^{i-1}) = 0\ \forall i \in [N+1, K]$, and the invocations to $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ and $\Pi^{\mathbb{K}}_{\mathsf{perm}}$ in $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$, and $\Pi^{\mathbb{K}}_{\mathsf{cq\text{-}prep}}$ for preprocessing $f_{\mathsf{comb}}(X)$. These cost in total $\widetilde{\mathcal{O}}(K)$ $\mathbb{G}_1$- and $\mathbb{F}$-operations.

Hence, the proving time in this algorithm $\mathsf{OffProve}$ is

$$\widetilde{\mathcal{O}}(K)\ \mathbb{G}_1\text{- and } \mathbb{F}\text{-operations, and } \mathcal{O}(K)\ \mathbb{G}_2\text{-operations}$$

according the efficiency reported in Table 3.

Size of π_{off}. Consider $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$.

- There are 12 single evaluations of polynomial commitments in $\mathbb{G}_1$ including

$$f_{\widehat{\mathsf{ch}}}(\rho), f_{\mathsf{ch}}(\omega_{\mathbb{K}} \cdot \rho), \ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}} \cdot \rho), f_{\widehat{\mathsf{lft}}}(\rho), f_{\mathsf{lft}}(\omega_{\mathbb{K}} \cdot \rho),$$
$$f_{\mathsf{ch}}(\omega_{\mathbb{K}}^0), f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{K-1}), f_{\mathsf{lft}}(\omega_{\mathbb{K}}^0), f_{\mathbf{t}}(\rho), f_{\mathsf{ch}}(\rho), f_{\mathsf{lft}}(\rho), \text{ and } f_{\mathsf{rgt}}(\rho).$$

  These single evaluations cost $12 \cdot 1 = 12$ $\mathbb{G}_1$.
- There are 2 single evaluations of PCS in $\mathbb{G}_2$ including

$$f_{\mathbf{t}}(\rho) \text{ and } f_{\mathsf{comb}}(\rho).$$

  These single evaluations cost $2 \cdot 1 = 2$ $\mathbb{G}_2$.
- There is 1 batch evaluation of PCS in $\mathbb{G}_1$, namely,

$$f_{\mathbf{t}}(\omega_{\mathbb{K}}^{i-1}) = 0\ \forall i \in [N+1, K].$$

  This batch evaluation costs 1 $\mathbb{G}_1$.
- There are 5 invocations to $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ and 1 invocation to $\Pi^{\mathbb{K}}_{\mathsf{perm}}$. Hence, these produce proofs of total size $(5+1) \cdot 8 = 48$ $\mathbb{G}_1$ and $(5+1) \cdot 3 = 18$ $\mathbb{F}$.

Hence, the size of π_{off} is

$$12 + 1 + 48 = 61\ \mathbb{G}_1, 2\ \mathbb{G}_2, 18\ \mathbb{F}.$$

Summary. In summary, prover time for computing $(\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}})$ is

$$\widetilde{\mathcal{O}}(K)\ \mathbb{G}_1, \mathbb{F}, \mathcal{O}(K)\ \mathbb{G}_2,$$

and π_{off} contains

$$61\ \mathbb{G}_1, 2\ \mathbb{G}_2, 18\ \mathbb{F}.$$

F.2 Algorithm $\mathsf{OffProve}_{\mathsf{pp}}(\sigma_1(f_{\mathsf{t}}), f_{\mathsf{t}}(X)) \to (\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \pi_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}})$

In $\Pi^{\mathbb{K}}_{\mathsf{ssq\text{-}off}}$, there are 12 single evaluations of polynomial commitments in $\mathbb{G}_1$ including

$$f_{\widehat{\mathsf{ch}}}(\rho), f_{\mathsf{ch}}(\omega_{\mathbb{K}} \cdot \rho), \ell_{\mathbb{K}}^{(1)}(\omega_{\mathbb{K}} \cdot \rho), f_{\widehat{\mathsf{lft}}}(\rho), f_{\mathsf{lft}}(\omega_{\mathbb{K}} \cdot \rho),$$
$$f_{\mathsf{ch}}(\omega_{\mathbb{K}}^0), f_{\mathsf{ch}}(\omega_{\mathbb{K}}^{K-1}), f_{\mathsf{lft}}(\omega_{\mathbb{K}}^0), f_{\mathbf{t}}(\rho), f_{\mathsf{ch}}(\rho), f_{\mathsf{lft}}(\rho), \text{ and } f_{\mathsf{rgt}}(\rho).$$

Verifying single evaluations cost $12 \cdot \mathcal{O}(1) = \mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2, \mathbb{F}$. The number of pairings will be discussed below.

There are 2 single evaluations of PCS in $\mathbb{G}_2$ including

$$f_{\mathbf{t}}(\rho) \text{ and } f_{\mathsf{comb}}(\rho).$$

Verifying these single evaluations cost $2 \cdot \mathcal{O}(1) = \mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2, \mathbb{F}$. The number of pairings will be discussed below.

There is 1 batch evaluation of PCS in $\mathbb{G}_1$, namely,

$$f_{\mathbf{t}}(\omega_{\mathbb{K}}^{i-1}) = 0 \ \forall i \in [N+1, K].$$

Verifying this batch evaluation costs $\mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2, \mathbb{F}$. The number of pairings will be discussed below.

There are 5 invocations to $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ and 1 invocation to $\Pi^{\mathbb{K}}_{\mathsf{perm}}$. Hence, verifying these evaluation proofs takes $(5+1) \cdot \mathcal{O}(1) = \mathcal{O}(1)$ $\mathbb{G}_1, \mathbb{G}_2$, and $(5+1) \cdot \mathcal{O}(\log_2 K) = \mathcal{O}(\log_2 K)$ $\mathbb{F}$. The number of pairings will be discussed below.

The number of pairings. The number of pairings is tricky for our construction. We consider the following cases for those with PCS evaluations, lookup, and permutation arguments whose commitments to witnesses are in $\mathbb{G}_1$. Looking ahead, we can categorize the pairings into the forms $\langle \cdot, [\![1]\!]_2 \rangle$, $\langle \cdot, [\![x]\!]_2 \rangle$, $\langle \cdot, [\![\mathsf{acc}(x)]\!]_2 \rangle$, and $\langle \cdot, \sigma_2(z_{\mathbb{K}}) \rangle$ as follows.

- Verification of PCS single evaluation for a polynomial commitment in $\mathbb{G}_1$ (c.f. (1)) has the form
$$\langle \sigma_1(f) - y \cdot [\![1]\!]_1, [\![1]\!]_2 \rangle = \langle \pi, [\![x]\!]_2 - \psi \cdot [\![1]\!]_2 \rangle$$
which is equivalent to
$$\langle \sigma_1(f) - y \cdot [\![1]\!]_1 + \psi \cdot \pi, [\![1]\!]_2 \rangle = \langle \pi, [\![x]\!]_2 \rangle$$
costing 2 pairings of the forms $\langle \cdot, [\![1]\!]_2 \rangle$ and $\langle \cdot, [\![x]\!]_2 \rangle$.
- Verification of PCS batch evaluation for a polynomial commitment in $\mathbb{G}_1$ (c.f. (2)) has the form
$$\langle \sigma_1(f) - [\![\mathsf{rem}(X)]\!]_1, [\![1]\!]_2 \rangle = \langle \pi_\Psi, [\![\mathsf{acc}(x)]\!]_2 \rangle$$
containing 2 pairings of the forms $\langle \cdot, [\![1]\!]_2 \rangle$ and $\langle \cdot, [\![\mathsf{acc}(x)]\!]_2 \rangle$.
- Each of the verifications for $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ and $\Pi^{\mathbb{K}}_{\mathsf{perm}}$ (c.f. Figures 3 and 4, respectively) has 1 pairing of the form $\langle \cdot, \sigma_2(z_{\mathbb{K}}) \rangle$, 4 pairings of the form $\langle \cdot, [\![1]\!]_2 \rangle$, 3 pairings of the form $\langle \cdot, [\![x]\!]_2 \rangle$, and 1 **pairing not falling into the above-mentioned forms**.

Hence, we can batch the above verification by using randomized linear combination s.t. those with the form $\langle \cdot, [\![1]\!]_2 \rangle$ (as well as $\langle \cdot, [\![x]\!]_2 \rangle$, $\langle \cdot, [\![\mathsf{acc}(x)]\!]_2 \rangle$, and $\langle \cdot, \sigma_2(z_{\mathbb{K}}) \rangle$) can be batched together into a single pairing. Hence, we have 4 pairings with these forms. Each pairing not falling into the categorized forms (in the verifications of $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ and $\Pi^{\mathbb{K}}_{\mathsf{perm}}$) is counted as 1 pairing. Hence, we have additional $5 + 1 = 6$ pairings as there are 5 and 1 invocations to $\Pi^{\mathbb{K}}_{\mathsf{lookup}}$ and $\Pi^{\mathbb{K}}_{\mathsf{perm}}$.

For the two verifications of single evaluations of PCS in $\mathbb{G}_2$, by batching as above, we additionally have 2 pairings for those of the forms $\langle [\![1]\!]_1, \cdot \rangle$ and $\langle [\![x]\!]_1, \cdot \rangle$.

Hence, in total, we have

$$4 + 6 + 2 = 12P$$

where "$12P$" is understood to be 12 pairings.

Summary. In summary, verifying π_{off} costs

$$\mathcal{O}(1)\ \mathbb{G}_1, \mathbb{G}_2, \mathcal{O}(\log_2 K)\ \mathbb{F}, 12P.$$

F.3 Algorithm $\mathsf{OnProve}_{\mathsf{pp}}(\mathsf{pp}_{\mathsf{off}}, \mathsf{wit}_{\mathsf{off}}, \mathsf{aux}_{\mathsf{off}}, f_{\mathsf{s}}(X), \sigma(f_{\mathsf{s}})) \to (\{0,1\}, \pi_{\mathsf{on}})$

Prover Time. Determining id and $\widetilde{\mathsf{id}}$ costs $\mathcal{O}(n)$ $\mathbb{F}$-operations. Determining $\mathbf{u}$ and $\mathbf{v}$ costs $\mathcal{O}(n \log_2 K)$ $\mathbb{F}$-operations by using basic binary search algorithm (e.g. [43]) detailed in Appendix B.6 to search over $(\mathsf{ch}, \mathsf{lft}, \mathsf{rgt})$. Encoding into polynomials and committing polynomials cost $\mathcal{O}(n)$ $\mathbb{G}_1$- and $\mathbb{F}$-operations.

Additionally, prover needs to encode $\mathbf{s}$, id, $\widetilde{\mathsf{id}}$, $\mathbf{u}$, and $\mathbf{v}$ into polynomials and commit to them into KZG PCS commitments over $\mathbb{G}_1$. Doing so costs $\mathcal{O}(n)$ $\mathbb{G}_1$- and $\mathbb{F}$-operations since $\left\{ \left[\!\!\left[\ell^{(i)}_{\mathbb{H}} \right]\!\!\right]_1 \right\}_{i=1}^{n}$ is determined in advance in pp.

In $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$, it can be seen that the prover time is dominated by the invocations to $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$. These invocations results in $\mathcal{O}(n)$ $\mathbb{G}_1$- and $\widetilde{\mathcal{O}}(n)$ $\mathbb{F}$-operations (c.f. Table 3).

Hence, the prover time in this algorithm $\mathsf{OnProve}$ is

$$\mathcal{O}(n)\ \mathbb{G}_1, \mathcal{O}(n \log_2 K)\ \mathbb{F},$$

according to the efficiency of binary search and in Table 3.
Size of π_{on}. Consider $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$.

- Prover sends eoid, $\sigma_1(f_{\mathsf{id}})$, $\sigma_1(f_{\widetilde{\mathsf{id}}})$, $\sigma_1(f_{\mathbf{u}})$, and $\sigma_1(f_{\mathbf{v}})$ costing $4\ \mathbb{G}_1, 1\ \mathbb{F}$.
- There are 4 single evaluations of polynomial commitments in $\mathbb{G}_1$ including

  $$f_{\mathsf{id}}(\omega_{\mathbb{H}}^{n-1}), f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}} \cdot \chi), f_{\mathsf{id}}(\chi), \text{ and } \ell^{(n)}_{\mathbb{H}}(\chi).$$

  These single evaluations cost $4 \cdot 1 = 4\ \mathbb{G}_1$.
- There are 8 invocations to $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$. According to Table 3, these produce proofs of total size $8 \cdot 8 = 64\ \mathbb{G}_1$ and $8 \cdot 3 = 24\ \mathbb{F}$.

Hence, the size of π_{on} is

$$4 + 4 + 64 = 72\ \mathbb{G}_1, 1 + 24 = 25\ \mathbb{F}.$$

Summary. In summary, prover time for computing (b, π_{on}) is

$$\mathcal{O}(n)\ \mathbb{G}_1, \mathcal{O}(n \log_2 K)\ \mathbb{F},$$

and π_{on} contains

$$72\ \mathbb{G}_1, 25\ \mathbb{F}.$$

F.4 Algorithm $\mathsf{OnVerify}_{\mathsf{pp}}(\sigma_2(f_{\mathsf{comb}}), \sigma_1(f_{\mathbf{s}}), b, \pi_{\mathsf{on}}) \to \{0,1\}$

In $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{ssq\text{-}on}}$, there are 4 single evaluations of polynomial commitments in $\mathbb{G}_1$ including

$$f_{\mathsf{id}}(\omega_{\mathbb{H}}^{n-1}), f_{\widetilde{\mathsf{id}}}(\omega_{\mathbb{H}} \cdot \chi), f_{\mathsf{id}}(\chi), \text{ and } \ell^{(n)}_{\mathbb{H}}(\chi).$$

Verifying these single evaluations cost $4 \cdot \mathcal{O}(1) = \mathcal{O}(1)\ \mathbb{G}_1, \mathbb{G}_2, \mathbb{F}$. The number of pairings will be discussed below.

There are 8 invocations to $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$. Hence, verifying these evaluation proofs takes $8 \cdot \mathcal{O}(1) = \mathcal{O}(1)\ \mathbb{G}_1, \mathbb{G}_2$, and $8 \cdot \mathcal{O}(\log_2 n) = \mathcal{O}(\log_2 n)\ \mathbb{F}$.
The number of pairings. By imitating the way in Appendix F.2, we categorize the pairings into the forms $\langle \cdot, [\![1]\!]_2 \rangle$, $\langle \cdot, [\![x]\!]_2 \rangle$, $\langle \cdot, [\![x^{K-1+(n-2)}]\!]_2 \rangle$, and $\langle \cdot, \sigma_2(z_{\mathbb{K}}) \rangle$ as follows.

- Verification of PCS single evaluation for a polynomial commitment in $\mathbb{G}_1$ (c.f. (1)) costs 2 pairings of the forms $\langle \cdot, [\![1]\!]_2 \rangle$ and $\langle \cdot, [\![x]\!]_2 \rangle$.
- Each of the verifications for $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$ (c.f. Figure 6) has 1 pairing of the form $\langle \cdot, \sigma_2(z_{\mathbb{K}}) \rangle$, 4 pairings of the form $\langle \cdot, [\![1]\!]_2 \rangle$, 2 pairings of the form $\langle \cdot, [\![x]\!]_2 \rangle$, 1 pairing of the form $\langle \cdot, [\![x^{K-1+(n-2)}]\!]_2 \rangle$, and 1 **pairing not falling into the above-mentioned forms**.

Hence, we can batch the above verification by using randomized linear combination s.t. those with the form $\langle \cdot, [\![1]\!]_2 \rangle$ (as well as $\langle \cdot, [\![x]\!]_2 \rangle$, $\langle \cdot, \sigma_2(z_{\mathbb{K}}) \rangle$, and $\langle \cdot, [\![x^{K-1+(n-2)}]\!]_2 \rangle$) can be batched together into a single pairing. Hence, we have 4 pairings with these forms. Each pairing not falling into the categorized forms (in the verifications of $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$) is counted as 1 pairing. Hence, we have 8 additional pairings as there are 8 invocations to $\Pi^{\mathbb{H},\mathbb{K}}_{\mathsf{cq\text{-}prove}}$.

Hence, in total, we have

$$4 + 8 = 12P$$

where "$12P$" is understood to be 12 pairings.
Summary. In summary, verifying π_{on} costs

$$\mathcal{O}(1)\ \mathbb{G}_1, \mathbb{G}_2, \mathcal{O}(\log_2 n)\ \mathbb{F}, 12P.$$